Dynamic and Efficient Universal Thresholdizers

Anonymous

Abstract. The Universal thresholdizers (UT), introduced by Boneh et al. CRYPTO 2018, are effective steps towards threshold cryptography as they allow for blackbox design of threshold functionalities for many standard primitives, e.g., CCA-secure public-key encryption schemes and signature schemes. They work by viewing the primitive to be thresholdized as a circuit and combining partial evaluations of the circuit by the threshold parties. This system guarantees Security - that partial evaluations do not disclose any information (with the exception of putting them together to obtain the circuit output if the parties satisfy the threshold), and Robustness – that one cannot produce invalid partial evaluations. There are two known general ideas of obtaining UT by combining threshold fully homomorphic encryption with either zero-knowledge proofs or with homomorphic signatures. However, all current constructions rely on a static approach where they work for any circuits of a fixed input value.
In this paper, we introduce a dynamic UT where evaluations are made for any input values without the need to re-run the costly onboarding phase. Additionally, we survey the existing UT construction methods, along with addressing an ambiguity in Boneh's robustness definition. Moreover, to improve the state of the art, we present two proof-based and signature-based UTs from Torus (Chillotti et al. Cryptology 2019) and Π's Batch&Pack (Baghery PKC 2025) that are both yielding more efficiency than known approaches, as well as providing the first implementation of universal thresholdizers.

Keywords: Applied Cryptography, Threshold Cryptosystem, Universal Thresholdizer, Threshold Fully Homomorphic Encryption, Lattices

1 Introduction

The threshold cryptography originates from the work on threshold secret sharing by Shamir [1] and Blakley [2] in the late 1970s and has been a long-lasting practical tool that, due to its promising applications, will be standardized by NIST [3]. These schemes addressed the problem of partitioning a secret among multiple parties such that a specified threshold of participants is required for reconstruction, and subsequent research extended these principles to specific cryptographic primitives. These constructions, however, required distinct designs for each primitive, resulting in a distributed landscape of threshold protocols with heterogeneous security properties and efficiency parameters.

To address this drawback, *Universal Thresholdizers* (UT) were presented by Boneh et al. [4] as a blackbox method to realize threshold primitives from non-threshold primitives, e.g., CCA-secure threshold public key encryption, and have progressed

since their introduction as a framework for transforming standard primitives into threshold variants [5]. This is achieved by reinterpreting the core operation of those primitives (e.g., signing or decryption algorithm) as a monotone boolean circuit and using UT to produce partial evaluations - the circuit evaluated on shares of the secret key [4]. *Correctness* of UT ensures that combining the outcomes of partial evaluations produces the same output as evaluating the circuit directly on the secret key. Additionally, UT is *compact* - where the size of the partial evaluation is linear in the size of the circuit depth and number of shares.

Universal Thresholdizers provide a unifying framework that addresses the aforementioned heterogeneity through a standardized methodology for transforming conventional primitives into threshold variants [6]. The UT comprises algorithms for setup, evaluation, verification, and combination that facilitate the distributed computation of functions representable as circuits [5]. The security model of UT encompasses two principal properties: *simulation security*, which establishes that partial evaluations disclose no information beyond the function output; the adversary cannot distinguish between real partial evaluations and fake partial evaluations [4] (i.e., the circuit outcome is simulated by a trusted 3rd party), and *robustness*, which ensures that adversarial participants cannot generate invalid partial evaluations that verification would accept.

The implementation of the UT paradigm with optimal efficiency while preserving both security and robustness remains a complex problem. The original construction by Boneh et al. [5] employed Threshold Fully Homomorphic Encryption (TFHE) in conjunction with zero-knowledge proofs with preprocessing (PZK) and commitment schemes (Com). This approach, while theoretically rigorous, incurs computational overhead, particularly for complex threshold access structures. These constraints have motivated the development of alternative constructions with different trade-offs between security, robustness, and efficiency.

Campanelli & Khoshakhlagh [7] proposed two distinct approaches to UT construction: one based on the combination of TFHE with Homomorphic Signatures (HS), and another leveraging Homomorphic Secret Sharing (HSS) with HS for two-party threshold settings. While these constructions offer computational advantages compared to the original proposal, no direct, detailed formal proofs for their security and robustness have been provided.

More recently, Ebrahimi & Yadav [6] introduced a construction based on TFHE with Key-Homomorphic Pseudo-random Functions (KHPRF), with the objective of enhancing the security guarantees of UTs. Concurrently, Cheon et al. [8] proposed a communication-efficient construction using Tree-based Secret Sharing (TreeSSS), which significantly lowers the number of secret shares needed per participant.

UT's primary application is in formalizing threshold systems during security analysis [4]; e.g., systems like Microsoft ElectionGuard [9] that are heavily based on threshold procedures can be modeled and formalized by UT for security analysis [10]. However, practical deployments of threshold cryptography often require processing *multiple distinct secret values* over time under a fixed committee structure—for instance, threshold decryption services that decrypt numerous ciphertexts, or threshold signing ceremonies that sign different messages across sessions. In such scenarios, existing static UT constructions impose a fundamental limitation: the secret value x is

irrevocably bound during the costly setup phase, necessitating complete re-execution of setup for each new value. This raises a natural open problem: *can we design a* UT *where the threshold infrastructure is established once, yet evaluations can be performed on arbitrary input values without re-running setup?* Addressing this question would enable efficient threshold-as-a-service architectures and amortize setup costs across multiple operations. We resolve this problem affirmatively in Section 4 by introducing the notion of *dynamic universal thresholdizer* (dUT). All in all, UT is a primitive that is promising and highly potential, but still evolving because of efficiency issues, and no practical implementation. Given these diverse approaches, UT requires a systematic analysis of the trade-offs between different methods for advancement, as identifying efficiency gaps in the previous work of UT is not straightforward. We initiate analysis of UT constructions in this work.

Contributions

This work presents a structured evaluation of UT constructions, providing a comprehensive taxonomy of design methodologies, security properties, and efficiency parameters. We identify precise distinctions between construction methods with implications for practical implementations. Our contributions include:

- *a survey on construction methods for* UT. For each method, we design its formalization, examine security guarantees, and quantitative efficiency analysis (e.g., computing time and space cost).
- *designing a dynamic universal thresholdizer* dUT to extend UT's further properties. All previous constructions focused on achieving a static UT where evaluations could only be performed for a fixed value, hard-coded in the setup phase. Our dynamic dUT can perform any evaluation for any value. We show how to design dUT and how they relate to static UT.
- *pushing the state-of-the-art forward* by clarifying ambiguities in UT designs and proposing new proof-based and signature-based designs from torus to construct more efficient schemes, as Tables 2 and 1 show, towards UT's evolution.

2 Preliminaries

We provide known and standard building blocks, utilizing the most commonly used notation throughout this work. Due to space constraints, we omit unnecessary details here and refer to Appendix A.

Torus Ring GSW (TRGSW) [11] Let $n,\ell \in \mathbb{N}$, and let $R = \mathbb{Z}[X]/(X^n+1)$. Let $\mathbb{T}_R = \mathbb{T}[X]/(X^n+1)$ where $\mathbb{T} = \mathbb{R}/\mathbb{Z}$. Let χ be a distribution over $\mathbb{T}_R$, and define the gadget vector $\mathbf{g} = (2^{-1},2^{-2},...,2^{-\ell})^T \in \mathbb{T}^\ell$. An encryption of $\mu \in \mathbb{B}[X]/(X^n+1)$ under secret $s \in \mathbb{B}[X]/(X^n+1)$ is a matrix $\mathbf{C} \in \mathbb{T}_R^{2\ell \times 2}$ such that $\mathbf{C}\cdot\mathbf{z} = \mu\cdot(\mathbf{g}\otimes\mathbf{z})+\mathbf{e}$, where $\mathbf{z} = (1,s)^T$, $\mathbf{e} \leftarrow\!\$\, \chi^{2\ell}$. The security of TRGSW$(n,\ell,\chi)$ encryption relies on the hardness of TRLWE(n,χ). The TRGSW scheme supports homomorphic operations through external products with TRLWE ciphertexts.

Technique	Setup	Evaluation	Verification	Combination
UT_2 [5]	$O(N{\cdot}\ell{\cdot}n{\cdot}\log q{+}N{\cdot}\lambda{+}N{\cdot}n{\cdot}\log q)$	$O(n{\cdot}\log q{+}\lambda)$	$O(n{\cdot}\log q{+}\lambda)$	$O(\ell{\cdot}N)$
Dynamic UT [Ours]	$O(N{\cdot}\ell{\cdot}n{\cdot}\log q{+}N{\cdot}\lambda{+}N{\cdot}n{\cdot}\log q)$	$O(n{\cdot}\log q{+}\lambda)$	$O(n{\cdot}\log q{+}\lambda)$	$O(\ell{\cdot}N)$
UT'_3 [5] or UT_3 [7]	$O(N{\cdot}\lambda{+}N{\cdot}n{\cdot}\log q)$	$O(\lambda{+}n{\cdot}\log q)$	$O(\lvert C\rvert{\cdot}n{\cdot}\log q{+}n{\cdot}\log q{+}\lambda)$	$O(d{\cdot}n{\cdot}\log q{+}\lvert y\rvert)$
2-UT [7]	$O(\lambda{+}\lvert x\rvert)$	$O(\lambda{+}\lvert x\rvert{+}\mathsf{poly}(\lambda))$	$O(\lambda{\cdot}\lvert C\rvert{+}\mathsf{poly}(\lambda))$	$O(\mathsf{poly}(\lambda){+}\lvert y\rvert)$
UT_4 [6]	$O(n{\cdot}m{+}N^{5.2}{+}\lambda{\cdot}n{+}n{\cdot}\lvert x\rvert)$	$O(n{\cdot}m{+}\lambda{+}\lvert\Psi\rvert{+}\lvert T_i\rvert)$	$O(n{\cdot}m{+}\lambda{+}\lvert\Psi\rvert)$	$O(t{\cdot}(\lambda{+}\lvert\Psi\rvert){+}N^{5.2})$
UT_5 [8]	$O(n{\cdot}\log q{+}(2t{-}1)^L{\cdot}\log q{+}\lambda{+}\lvert F\rvert)$	$O(n{\cdot}\log q{+}\lambda{+}\lvert F\rvert)$	$O(\lambda{+}\lvert y\rvert{+}\lvert S\rvert{\cdot}\lvert T_i\rvert{\cdot}\log q)$	$O(\lvert T\rvert{\cdot}\log q{+}\lvert S\rvert{\cdot}(\lambda{+}\lvert y\rvert))$
UT_6 [Ours]	$O(n{\cdot}\log q{+}n{\cdot}\lvert T_i\rvert{\cdot}\log q{+}N{\cdot}\lambda)$	$O(n{\cdot}\log q{+}\lambda{+}n{\cdot}\lvert T_i\rvert)$	$O(n{\cdot}\log q{+}\lambda)$	$O(n{\cdot}\lvert T\rvert{\cdot}\log q)$
UT_7 [Ours]	$O(n{\cdot}\log q{+}n{\cdot}\lvert T_i\rvert{\cdot}\log q{+}N{\cdot}\lambda)$	$O(n{\cdot}\log q{+}\lambda)$	$O(n{\cdot}\log q{+}\lambda)$	$O(n{\cdot}\lvert T\rvert{\cdot}\log q)$

Table 1: Space Consumption of Universal Thresholdizer's Construction Methods. In Tables 1 and 2, $|\Psi|$ is size of the zero-knowledge statement, $|x|$ is input size, $|y|$ is output size, s is threshold parameter in $\mathsf{TreeSSS}$, t is number of threshold shares or parties, L is tree depth or level parameter, $|F|$ is size of function representation, $|S|$ is size of signature or proof set, $|T|$ represents size of the transcript or verification data structures, and for UT_6 and UT_7, $|T_i|{=}O((2t{-}1)^L/N)$ is share size per party.

Efficient Access Structures ($\mathbb{S}$,N) [5] We consider *access structures* as all sets of authorized parties $\mathbb{A}{\subset}\mathbb{S}$ with N total parties $|\mathbb{A}|{\leq}N$. Access structures are *efficient* if they are monotone (i.e., $\forall\mathbb{A}{\subset}\mathbb{S}$ and $|\mathbb{A}|{<}N$ implies $\mathbb{A}{\cup}\{x\}{\subset}\mathbb{S}$) and they can be described using a boolean circuit $f_{\mathbb{A}}{:}\{0{,}1\}^N{\to}\{0{,}1\}$. A particular type of efficient access structures is *threshold access structures* where a threshold t defines the minimal size of the authorized set: $t{\leq}\mathbb{A}$ for $\mathbb{A}{\subset}\mathbb{S}$.

Circuit class ($\mathbb{C}$,d) [6] We use circuits of arbitrary inputs $\mathbb{C}{:}\{0{,}1\}^k{\to}\{0{,}1\}$ that are bounded by depth d. For simplicity in some parts, we use $C{\in}\mathbb{C}$.

(Threshold) Fully Homomorphic Encryption (TFHE) [12] A fully homomorphic encryption algorithm $\mathsf{FHE}{=}$ ($\mathsf{FHE.Setup}$, $\mathsf{FHE.Encrypt}$, $\mathsf{FHE.Eval}$, $\mathsf{FHE.Decrypt}$) relies on circuit depth bounds d and enables computation on encrypted data. A threshold fully homomorphic encryption scheme $\mathsf{TFHE}{=}$ (Setup, $\mathsf{Encrypt}$, Eval, $\mathsf{PartDec}$, FinDec) relies on efficient access structures ($\mathbb{S}$,N). These access structures formalize the authorized participant subsets $S{\in}\mathbb{S}$ capable of collaborative decryption, where a valid threshold scheme must satisfy both correctness and security properties. Specifically, the correctness property requires that $\forall S{\in}\mathbb{S}$, $\forall\mu{\in}\{0{,}1\}$, and for any circuit $C{\in}\mathbb{C}$ of depth at most d: $\mathsf{FinDec}(\mathsf{pk},\{\mathsf{PartDec}(\mathsf{pk},\mathsf{Eval}\ (\mathsf{pk},C,\mathrm{ct}_1,...,\mathrm{ct}_k),\ \mathsf{sk}_i)\}_{i\in S}) = C(\mu_1,\ ...,\mu_k)$ where $\mathrm{ct}_j{\leftarrow}\mathsf{Encrypt}\ (\mathsf{pk},\mu_j)$. A TFHE scheme satisfies simulation security if $\forall\lambda,d,\mathcal{A}{\in}\mathsf{S}$, $\exists$ stateful PPT algorithm $\mathcal{S}{=}(\mathcal{S}_1,\mathcal{S}_2)$ such that $\forall$ PPT adversary $\mathcal{A}$, the experiments $\mathsf{Exp}^{\mathsf{real}}_{\mathcal{A},\mathsf{TFHE}}\ (1^\lambda,1^d)$ and $\mathsf{Exp}^{\mathsf{ideal}}_{\mathcal{A},\mathsf{TFHE}}(1^\lambda,1^d)$ are computationally indistinguishable.

Static Universal Thresholdizer [5] Consider the efficient access structure ($\mathbb{S}$,N) and circuit class ($\mathbb{C}$,d). A *static universal thresholdizer* UT consists of the algorithms:

- $(\mathsf{pp},s_1,...,s_N){\leftarrow}\mathsf{Setup}(1^\lambda,1^d,(\mathbb{S},N),x)$: Given the security parameter 1^λ, a depth bound 1^d, an efficient access structure ($\mathbb{S}$,N), and a message $x{\in}\{0{,}1\}^k$, this algorithm outputs public parameters pp and a set of shares $s_1,...,s_N$.

- $y_i \leftarrow \mathsf{Eval}(\mathsf{pp}, s_i, \mathbb{C})$: On input of the public parameters p, a share s_i, and a circuit $\mathbb{C}$: $\{0,1\}^k \to \{0,1\}$ of depth at most d, the algorithm produces a partial evaluation y_i.
- $\{0,1\} \leftarrow \mathsf{Verify}(\mathsf{pp}, y_i, \mathbb{C})$: This algorithm takes as input pp, a purported partial evaluation y_i, and a circuit $\mathbb{C}$, and outputs a bit indicating whether the evaluation is valid.
- $y \leftarrow \mathsf{Combine}(\mathsf{pp}, B)$: Outputs the final evaluation y, given the collection of partial evaluations $B = \{y_i\}_{i \in S}$ from an authorized set S.

3 Related Works

In this section, we study the methods of constructing UT from different aspects, and Figure 1 presents the taxonomy of building techniques, focusing on designing procedures and constructing methods of UT. We have formalized the existing methods in Figures 12, 13, 14, 15, and 16.

The aim of UT is to compute $\mathbb{C}(x)$ from a private value $x \in \{0,1\}^k$ for some public circuit $\mathbb{C}: \{0,1\}^k \to \{0,1\}$. This is achieved by involving N parties that produce partial evaluations y_i with $i \in S$ and $S \subset \mathsf{S}$, and a mechanism to combine these partial evaluations to produce the final evaluation $y \leftarrow \mathbb{C}(x)$.

UT relies on limiting an efficient adversary's ability to distinguish between a real partial evaluation and a simulated one (for *security*) and to produce fake partial evaluations that verify (for *robustness*). Additionally, UT satisfies *compactness* - the size of the partial evaluations is reasonable (i.e., $|y_i| \leq \mathsf{poly}(\lambda, d, n)$), and *correctness* - an honest execution produces the intended outcomes.

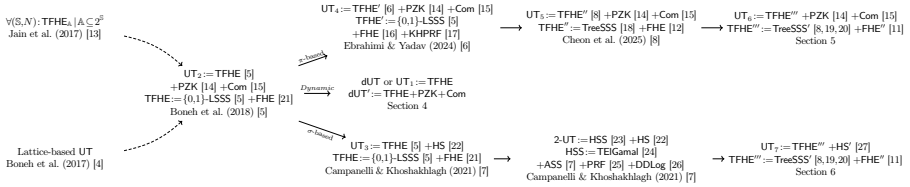

Fig. 1: Designing Techniques of Universal Thresholdizer

This static UT_2 (Figure 12) has been proposed by Boneh et al. and has been shown to be compact, correct, secure, and robust [5]. Consider $\mathsf{ct} \leftarrow \mathsf{TFHE.Enc}(\mathsf{tpk}, x)$, $\hat{\mathsf{ct}} \leftarrow \mathsf{TFHE.Eval}(\mathsf{tpk}, \mathbb{C}, \mathsf{ct})$. The idea is to have the partial evaluation $y_i = (p_i, \pi_i)$ composed from TFHE partial decryption $p_i \leftarrow \mathsf{TFHE.PartDec}(\mathsf{tsk}_i, \hat{\mathsf{ct}})$ and prove using π_i that p_i is obtained from decrypting $\hat{\mathsf{ct}}$ and that the right key was used (this is where the commitment to tsk_i in setup is needed). The security of UT trivially reduces to the simulated security of TFHE, the zero-knowledge property of the proof system PZK, and the computational-binding property of the commitment scheme Com. Robustness only holds if the experiment actually measures improperly computed partial evaluations.

In [5], another technique of constructing UT is proposed informally via $\mathsf{UT}'_3 = \mathsf{TFHE}+\mathsf{HS}$. Homomorphic signatures [28] give more compact proofs than PZKs, we

can get partial evaluations y_i whose size is independent of the original secret message x and the size of circuit C that is used for the evaluation [5]. Thus, HS are more efficient than PZK+Com in building UT from the robustness aspect. Boneh et al. do not provide the construction for this and do not state that this satisfies security.

In addition, Boneh et al. [5] proposed two separate TFHE schemes (one based on Shamir and one based on {0,1}−LSSS). In a manuscript, Cho et al. [29] have recently shown an attack that breaks the simulation security on the Shamir-based t-out-of-N TFHE of Boneh et al. [5], but {0,1}−LSSS remains secure.

To address the aforementioned limitations, [7] presented constructions based on HS more formally. Campanelli et al. [7] are the first to provide a formal construction for this, whereas Boneh et al. [5] only stated that HS can be used. Homomorphic signatures [28] give more compact proofs than NIZKs, and we can get partial evaluations y_i whose size is independent of the original secret message x and the size of the circuit $\mathbb{C}$ that is used for the evaluation. Unlike NIZK, homomorphic signatures can be constructed from the SIS problem [22]. The first procedure of [7] is constructing UT using TFHE+HS, as demonstrated in the Figure 13.

Campanelli & Khoshakhlagh [7] claim this is compact, correct, secure, and robust, but no complete formal proof has been given to UT=TFHE+HS by [7]. They define robustness but do not provide explicit proof for the specific construction, and they lack the guarantee that adversaries cannot forge valid certificates. Notice that Boneh et al [5] stated that for robustness, one can use HS instead of PZK +Com, and do not say (or imply) that HS is sufficient for security. While [7] correctly employs this standard context-hiding notion, it has not been guaranteed that context-hiding alone does not suffice for the security requirements of UT.

The other technique presented in [7] is a (2,2)-case for UT, which is subsumed by Figure 14. This type of UT is getting built upon HSS and HS as shown in Figure 14. This HSS-based construction requires simpler primitives and is more efficient than TFHE [30], and allows for a wider type of instantiations, for example, from DDH as in [23]. Despite the UT_3 that works for any threshold access structure (d,N), 2-UT only works for exactly two parties with a threshold of 2, i.e., a (2,2)-threshold scheme, (restricted setting) and cannot be extended to larger committees (limited scalability). In addition, this instantiation is still plausibly weaker than publicly verifiable NIZKs as explicitly mentioned in [7].

Ebrahimi et al. [6] improve the security of UT by designing a stronger secure TFHE variant - TFHE′, using the construction method by Boneh et al. [5]. This method improves the TFHE construction from [5] using key-homomorphic PRF techniques, and strengthens security definitions inspired by Bellare's works [31] and [32]. Additionally, they combine their techniques with Agrawal et al. (2022) [33] to achieve partially adaptive key queries. Unlike other ideas, this procedure achieves the thresholdizer by mixing {0,1}-LSSS with FHE and KHPRF. This technique works the same as Figure 12, but the main difference lies in the way of building the new TFHE as Figure 15; i.e., the difference is in the fact that the [6] satisfies a stronger notion of simulation security, and this leads to a universal thresholdizer with a stronger notion of security. The proof is exactly the same as for [5], with only accounting for the updates in the security definitions.

In some real-world applications, handling lattice-based "Almost" Key-Homomorphic PRFs can be challenging in the case of UT, as they must add flooding noise to hide the error, which increases noise parameters. There is a solution to this drawback in [33], which involves using Rényi divergence-based analysis [34] to optimize noise parameters in threshold signature schemes. This solution allows reducing the size of the q from exponential to polynomial in λ. However, Ebrahimi & Yadav [6] acknowledge that this optimization cannot be applied to their TFHE' and UT_4 constructions because the Rényi divergence procedure [34] works specifically for search-based primitives and cannot be transferred to indistinguishability-based constructions.

Cheon et al. [8] introduced a communication-efficient one-round TFHE'' construction based on a specialized linear secret sharing scheme, called TreeSSS and constructed through the iterative application of t-out-of-$(2t-1)$ Shamir secret sharing, where $t \ll N$. This construction reduces the number of required secret shares from $O(N^{5.3})$ in previous {0,1}-LSSS schemes to $O(N^{3+o(1)})$ while maintaining the compactness property of UT_5. The result matches the optimal amplification used in constructing large input majority functions from smaller ones. TreeSSS can also serve as a subroutine for constructing lattice-based t-out-of-N threshold primitives such as TFHE and threshold signatures, individually. This method is presented in Figure 16.

This technique inherits a similar limitation to Boneh's {0,1}-LSSS construction: the need for exponential offline verification costs. Both approaches require checking whether a share matrix M is correctly generated, which involves $O\binom{N}{t}$ operations, resulting in exponential time complexity for large N, so both methods share certain fundamental limitations related to probabilistic construction and offline verification costs.

4 Our Dynamic Universal Thresholdizer

Current approach to UT relies on a static structure, where the input value x is fixed and hard-coded in a privacy-preserving way in the public parameters. We introduce a dynamic approach where one can encode any values x at any time and dynamically use the UT evaluation for any circuit and any x. We can show there is a natural equivalence between the static and dynamic UT, as, instead of encoding the value x in the setup (as done by UT), we can do this at any stage by simply calling $\mathsf{ct} \leftarrow \mathsf{Encode}(x)$ and ensuring the evaluation is done over the same encoding ct.

4.1 Dynamic Universal Threshold Syntax

Based on the constructions discussed in Section 3, we bring up dUT' as a Dynamic UT can also provide robustness if it gets built by PZK+Com. One can utilize UT_5 or UT'_3's to achieve dUT' as well. This construction extends the approach delineated in Figure 2 by incorporating verification mechanisms similar to those in Figure 12, but critically, maintains a separation between the setup and encoding phases. Specifically, the $\mathsf{dUT}'.\mathsf{Setup}$ algorithm generates the necessary material without binding to any specific input value x, allowing the subsequent $\mathsf{dUT}'.\mathsf{Encode}$ to operate on arbitrary inputs without re-initializing the entire procedure.

Definition 1 (Dynamic Universal Thresholdizer). *Given the efficient access structure* (S,N) *and circuit class* $(\mathbb{C},d)$*, a* dynamic universal thresholdizer dUT *consists of the following algorithms:*

- $(\mathsf{upp},\mathsf{usk}_1,\ldots,\mathsf{usk}_N) \leftarrow \mathsf{Setup}(\lambda,(\mathsf{S},N),(\mathbb{C},d))$. *Returns the public parameters* upp *and for each party* i*, they receive an individual secret key* usk_i.
- $\mathsf{ct} \leftarrow \mathsf{Encode}(\mathsf{upp},x)$. *Encodes the private value* x *into* ct.
- $y_i \leftarrow \mathsf{Eval}(\mathsf{upp},\mathsf{usk}_i,(\mathbb{C},d),\mathsf{ct})$. *Produces the partial evaluation* y_i *over the encoding* ct *using circuit* $\mathbb{C}$ *and the secret key of user* i.
- $\{0,1\} \leftarrow \mathsf{Verify}(\mathsf{upp},y_i,(\mathbb{C},d),\mathsf{ct})$. *Verifies if the partial evaluation* y_i *is valid for circuit* $(\mathbb{C},d)$ *and encoding* ct. Evaluation correctness *ensures with overwhelming probability that* $1 \leftarrow \mathsf{Verify}(\mathsf{upp},\mathsf{Eval}(\mathsf{upp},\mathsf{usk}_i,(\mathbb{C},d),\mathsf{ct}),(\mathbb{C},d),\mathsf{ct})$.
- $y \leftarrow \mathsf{Combine}(\mathsf{upp},\{y_i\}_{i\in S},\mathsf{ct})$. *Returns the combine evaluation* y *from the partial evaluations* $\{y_i\}_{i\in S}$. Combine correctness *ensures with overwhelming probability that* $y = \mathbb{C}(x)$ *for any* $S \subset \mathsf{S}$ *and* $y_i \leftarrow \mathsf{Eval}(\mathsf{upp},\mathsf{usk}_i,(\mathbb{C},d),\mathsf{ct})$ *and* $\mathsf{ct} \leftarrow \mathsf{Encode}(\mathsf{upp},x)$.

The significance of this dynamic construction lies in its operational flexibility when evaluating multiple distinct inputs. The dUT' paradigm can be instantiated with any of the TFHE variants discussed in Section 3, inheriting their efficiency characteristics while maintaining security and robustness properties. The verification component utilizes the same PZK framework as in the static case, with the distinction that verification procedures now incorporate the encoded value ct as an explicit parameter.

The relation between Static and Dynamic Universal Thresholdizers is stated in the following remark.

Remark 1. Let $\mathsf{UT}=(\mathsf{Setup},\mathsf{Eval},\mathsf{Verify},\mathsf{Combine})$ be a secure Static Universal Thresholdizer and $\mathsf{dUT}=(\mathsf{Setup},\mathsf{Encode},\mathsf{Eval},\mathsf{Verify},\mathsf{Combine})$ be a secure Dynamic Universal Thresholdizer. Then:

(i) Any dUT can be used to construct a UT as follows:

$$\mathsf{UT.Setup}(1^\lambda,1^d,(\mathbb{S},N),x) := \left[\begin{array}{c} (\mathsf{pp}',\mathsf{sk}_1,\ldots,\mathsf{sk}_N) \leftarrow \mathsf{dUT.Setup}(1^\lambda,(\mathbb{S},N),(\mathbb{C},d)) \\ \mathsf{ct} \leftarrow \mathsf{dUT.Encode}(\mathsf{pp}',x) \\ \textbf{return } (\mathsf{pp}=(\mathsf{pp}',\mathsf{ct}),\mathsf{sk}_1,\ldots,\mathsf{sk}_N) \end{array}\right]$$

(ii) UT does not generally imply dUT without additional assumptions, as UT inherently binds the secret value x during setup, precluding the dynamic encoding of arbitrary values post-setup. Therefore, dUT is more expressive than UT.

4.2 Dynamic Universal Thresholdizer Construction and Properties

We construct dUT using only TFHE, as shown in Figure 2, which is non-robust. Recall that any construction for dUT implies a construction for UT- and thus, we also obtain a design for UT_1. It is straightforward to see that if one designs UT only from TFHE it produces an UT that is compact, correct, and secure but not robust. Consider

$\mathsf{ct} \leftarrow \mathsf{TFHE.Enc}(\mathsf{tpk},x)$, $\hat{\mathsf{ct}} \leftarrow \mathsf{TFHE.Eval}(\mathsf{tpk},\mathbb{C},\mathsf{ct})$ and $y_i \leftarrow \mathsf{TFHE.PartDec}(\mathsf{tsk}_i,\hat{\mathsf{ct}})$. The dUT security and TFHE simulated security are identical, and both ask the adversary to distinguish between real and simulated values for y_i. To obtain robustness for dUT, a method to verify the partial evaluation is needed; however, TFHE does not provide one, and as such, we can only have $1 \leftarrow \mathsf{Verify}(\mathsf{upp},y_i,(\mathbb{C},d),\mathsf{ct})$. This naturally implies that any value $z \leftarrow\$ \mathbb{Z}_q$ will also successfully verify. Here, z represents any arbitrary value, including potentially invalid partial evaluations, highlighting that without a proper verification mechanism, even maliciously crafted values would be accepted. One can obtain a trivial static equivalent for UT by creating ctin setup and adding it to the public parameters upp.

$\mathsf{dUT.Setup}(1^\lambda,(\mathsf{S},N),(\mathbb{C},d))$

1: $(\mathsf{tfhepk},\mathsf{tfhesk}_1,\ldots,\mathsf{tfhesk}_N) \leftarrow \mathsf{TFHE.Setup}(1^\lambda,1^d,\mathsf{S})$
2: $\mathsf{upp} \leftarrow \mathsf{tfhepk}$
3: $\forall i \in [N].\, \mathsf{usk}_i \leftarrow \mathsf{tfhesk}_i$
4: **return** $(\mathsf{upp},\mathsf{usk}_1,\ldots,\mathsf{usk}_N)$

$\mathsf{dUT.Encode}(\mathsf{upp},x)$

1: $\forall i \in [k].\, \mathsf{ct}_i \leftarrow \mathsf{TFHE.Encrypt}(\mathsf{upp},x_i)$
2: $\mathsf{ct} \leftarrow \{\mathsf{ct}_i\}_{i\in[k]}$
3: **return** ct

$\mathsf{dUT.Eval}(\mathsf{upp},\mathsf{usk}_i,(\mathbb{C},d),\mathsf{ct})$

1: $\hat{\mathsf{ct}} \leftarrow \mathsf{TFHE.Eval}(\mathsf{upp},\mathbb{C},\mathsf{ct})$
2: $y_i \leftarrow \mathsf{TFHE.PartDec}(\mathsf{upp},\hat{\mathsf{ct}},\mathsf{usk}_i)$
3: **return** y_i

$\mathsf{dUT.Verify}(\mathsf{upp},y_i,(\mathbb{C},d),\mathsf{ct})$

1: **return** 1 // TFHE provides no way to verify

$\mathsf{dUT.Combine}(\mathsf{upp},\{y_i\}_{i\in S},\mathsf{ct})$

1: $S \subseteq \{1,\ldots,N\} \,|\, S \in \mathsf{S}$
2: $y \leftarrow \mathsf{TFHE.FinDec}(\mathsf{upp},\{y_i\}_{i\in S})$
3: **return** y

Fig. 2: Designing the Dynamic Variant of UT (UT_1) from TFHE

Our dUT achieves the same security properties as [5], e.g., simulation security and robustness. We have slightly updated them to allow for any encoding of x, and formally define them in Figure 3. dUT can bring several applications as the separation of setup and encoding phases in dUT yields direct instantiation of threshold decryption services evaluating arbitrary ciphertexts post-setup without key redistribution (e.g. [35], [36]), and provides the platform for multi-session threshold computation protocols (e.g. [37], [38]) where circuits are evaluated over temporally distinct input datasets under fixed threshold access structures.

Definition 2 (dUT Security). *A dUT is* secure *if for any efficient adversary $\mathcal{A}$, any security parameter λ, circuit class $(\mathbb{C},d)$, access structure $(\mathbb{S},N)$ there exists an efficient simulator $\mathsf{Sim} = (\mathsf{Sim}_1,\mathsf{Sim}_2,\mathsf{Sim}_3)$ with the following advantage negligible: Assume the probabilities $P_{Real} = \Pr\left[\mathsf{Exp}^{\mathsf{dutsec}}_{\mathcal{A},\mathsf{Real}}(1^\lambda,1^d)=1\right]$ and $P_{Ideal} = \Pr\left[\mathsf{Exp}^{\mathsf{dutsec}}_{\mathcal{A},\mathsf{Ideal}}(1^\lambda,1^d)=1\right]$, then we have advantage $\mathsf{Adv}^{\mathsf{sec}}_{\mathcal{A},\mathsf{dUT}}(\lambda) = |P_{Real} - P_{Ideal}|$, where $\mathsf{Exp}^{\mathsf{dutsec}}_{\mathcal{A},b}$ defined in Figure 3. The simulator Sim_2 produces a simulated encoding ct given only the length*

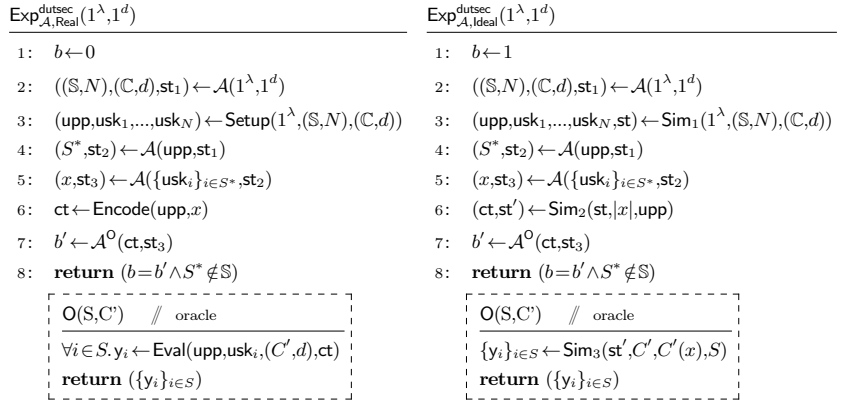

Fig. 3: Security Experiments for Dynamic Universal Thresholdizer

$|x|$ of the plaintext (but not x itself), while Sim_3 is given the computation results $C'(x)$ to simulate partial evaluations.

Definition 3 (Robustness). *A Dynamic Universal Thresholdizer scheme satisfies* robustness *if for any efficient adversary $\mathcal{A}$, any security parameter λ, circuit class $(\mathcal{C},d)$, and access structure $(\mathbb{S},N)$, the following probability is negligible*[1]*:*

$$\Pr\left[\begin{array}{l}(\mathbb{S},N)\leftarrow\mathcal{A}(1^\lambda,1^d);(pp,sk_1,...,sk_N)\leftarrow\mathsf{Setup}(1^\lambda,(\mathbb{S},N),(\mathbb{C},d));\\(x,C^*,i^*)\leftarrow\mathcal{A}(pp,sk_1,...,sk_N);ct\leftarrow\mathsf{Encode}(pp,x);\\y_{i^*}\leftarrow\mathsf{Eval}(pp,sk_{i^*},(C^*,d),ct);(p_{i^*},\pi_{i^*}):=y_{i^*};\;\; y^*\leftarrow\mathcal{A}(ct);\\(p^*,\pi^*):=y^*;\;\; b_1\leftarrow(p^*\neq p_{i^*});\\b_2\leftarrow\mathsf{Verify}(pp,y^*,(C^*,d),ct);return\ (b_1\wedge b_2)\end{array}\right]=1\leq\mathsf{negl}(\lambda)$$

As a result, as it is tangible, the property arguments, including statements of security and robustness, for the dUT and UT are straightforward modifications of each other, echoing the arguments in [5] as they are 1:1 parallel to the UT setting.

It is worth noting that by focusing on fake partial evaluations in security and robustness, as part of formalizing security, there is an assumption that an efficient simulator exists that can produce good fake partial evaluations that no efficient adversary can distinguish from real partial evaluations. This should also hold in the context of the adversary verifying the fake partial evaluations - in essence, the verification should return true on a fake partial evaluation that was produced by the simulator. We want to clarify that this does not contradict robustness, where the adversary itself has to produce those fake partial evaluations. The core difference comes from the fact that the simulator manipulates the setup phase to introduce a trapdoor that allows it to do this - this is exactly the same strategy that most zero-knowledge proofs take.

[1] Partial evaluations admit decomposition $y_i=(p_i,\pi_i)$ where p_i constitutes the share and π_i comprises auxiliary verification data instantiated via ZKP or HS.

Lemma 1. *Let* $\mathsf{UT}=(\mathsf{Setup},\mathsf{Eval},\mathsf{Verify},\mathsf{Combine})$ *be a Universal Thresholdizer with security parameter* λ*, and let* $\mathsf{dUT}'=(\mathsf{Setup}',\mathsf{Encode},\mathsf{Eval}',\mathsf{Verify}',\mathsf{Combine}')$ *be its dynamic variant constructed as follows:*

$$\mathit{dUT'.Setup'}(1^\lambda,(\mathbb{S},N),(\mathbb{C},d)) := \left[\begin{array}{c} (\mathsf{tfhepk},\mathsf{tfhesk}_1,\ldots,\mathsf{tfhesk}_N)\leftarrow\mathsf{TFHE.Setup}(1^\lambda,1^d,\mathbb{S}) \\ \forall i\in[N]:(\sigma_{V,i},\sigma_{P,i})\leftarrow\mathsf{PZK.Pre}(1^\lambda) \\ \forall i\in[N]:r_i\xleftarrow{R}\{0,1\}^\lambda;\ \ \forall i\in[N]:\mathsf{com}_i\leftarrow\mathsf{Com}(\mathsf{tfhesk}_i;r_i) \\ \textbf{\textit{return}}\ (\mathsf{pp}'=(\mathsf{tfhepk},\{\sigma_{V,i}\},\{\mathsf{com}_i\}),\{(\mathsf{tfhesk}_i,\sigma_{P,i},r_i)\}) \end{array}\right]$$

If UT *satisfies simulation security according to Definition 2 and robustness according to Definition 3, then* dUT' *also satisfies these properties with a security reduction that preserves the advantage bound up to a negligible factor.*

For simulation security, since the TFHE scheme's security guarantees hold independently of when the ciphertext is generated, the encryption of x in $\mathsf{dUT}'.\mathsf{Encode}$ maintains the same indistinguishability properties as when performed during $\mathsf{UT.Setup}$. Similarly, robustness transfers because the verification mechanism in dUT' operates on the same principle as in UT, utilizing PZK to ensure correctness of partial evaluations relative to the committed key shares.

4.3 Robustness Ambiguity in Boneh et al [5]

In the robustness experiment by Boneh et al. [5] (Definition 31), the circuit $\mathbb{C}$ used to verify the adversarial partial evaluation y_i^* is left undefined. It seems the intuition is for the adversary to also produce this circuit when it produces the partial evaluation, as it controls all user secret keys.

Additionally, the experiment was initially formalized as the adversary input $y_i^* \neq \mathsf{Eval}(\mathsf{upp},\mathsf{usk}_i,\mathbb{C})$. This should be interpreted as $y_i^* \neq y_i$ where y_i denotes all the possible outputs of $\mathsf{Eval}(\mathsf{pp},\mathsf{sk}_i,C)$, to capture the intuition of an improperly computed partial evaluation. Another approach, and the one we use in this paper, is to have split $y_i=(p_i,\pi_i)$ where there is only one true partial evaluation p_i and potentially many valid auxiliary information π_i. Thus, the adversary is restricted to produce (p_i^*,π_i^*) for $p_i^*\neq p_i$ with $(p_i,\pi_i)\leftarrow\mathsf{Eval}(\mathsf{upp},\mathsf{usk}_i,\mathbb{C})$.

We provide in Figure 4 a clarified experiment. The robustness of dUT (Definition 3) does not inherit this incompleteness.

$\mathsf{Exp}_{\mathcal{A},\mathsf{UT},\mathsf{rb}}(1^\lambda,1^d)$

1: $(x,\mathbb{A})\leftarrow\mathcal{A}(1^\lambda,1^d)$; $(\mathsf{pp},\mathsf{s}_1,\ldots,\mathsf{s}_N)\leftarrow\mathsf{UT.Setup}(1^\lambda,1^d,\mathbb{A},x)$;
2: $(\mathsf{y}_i^*,C)\leftarrow\mathcal{A}(\mathsf{pp},\mathsf{s}_1,\ldots,\mathsf{s}_N)$; $(\mathsf{p}_i,\pi_i)\leftarrow\mathsf{UT.Eval}(\mathsf{pp},\mathsf{s}_i,C)$;
3: $(\mathsf{p}_i^*,\pi_i^*):=\mathsf{y}_i^*$; **return** $\mathsf{UT.Verify}(\mathsf{pp},\mathsf{y}_i^*,C)=1\wedge(\mathsf{p}_i^*\neq\mathsf{p}_i)$

Fig. 4: Improved Robustness Experiment for UT

5 Our Proof-based UT from Torus-FHE and Batched TreeSSS

We now present our construction of a proof-based universal thresholdizer that combines Torus-based threshold fully homomorphic encryption with an optimized batched variant of tree secret sharing to improve efficiency over existing approaches.

5.1 Batched Tree Secret Sharing

The Batched&Packed technique [20] combines multiple independent secret sharings into a single protocol by (i) batching: sharing k secrets simultaneously using aggregated commitments and challenges to reduce verification overhead from $O(nk)$ to $O(n+k)$, and (ii) packing: embedding ℓ_p secrets as coefficients of a single degree-$(s+\ell_p-2)$ polynomial to reduce the number of shares per party. More on Batched&Packed in Appendix A.3.

We apply the batched and packed technique of [20] for the Π framework [19] into the TreeSSS [8], to construct TreeSSS$'$ as presented in Figure 5. Then, using Theorem 1 and Theorem 2 we state that TreeSSS$'$ is correct (Definition 9) and is private (Definition 10).

Theorem 1. *Let* $\mathsf{TreeSSS}' = (\mathsf{Share}', \mathsf{Combine}')$ *with parameters* $(N,t,s,\ell,q,L,k,\ell_p)$ *satisfying Figure 5. For all* $\{\mathsf{sk}^{(j)}\}_{j\in[k]} \subset (\mathbb{Z}_q^{\ell_p})^k$ *and all* $S \in \mathcal{A}_{N,t}$ *with* $|S| \geq t+\ell_p$*:*

$$\Pr\left[\begin{array}{l} \mathsf{Combine}'(\{(\{\mathsf{sk}_i^{(j)}\}_j, T_i, \pi_i)\}_{i\in S}) = \{\mathsf{sk}^{(j)}\}_j \\ \mid \{\{\mathsf{sk}_i^{(j)}\}_j, T_i, \pi_i\}_i \leftarrow \mathsf{Share}'(\{\mathsf{sk}^{(j)}\}_j, \mathcal{A}_{N,t}) \end{array}\right] \geq 1 - 2^{-\Omega(\lambda)}.$$

Theorem 2. *Let* $\mathsf{TreeSSS}' = (\mathsf{Share}', \mathsf{Combine}')$ *with computationally hiding commitment* $\mathcal{C}$ *and random oracle* $\mathcal{H}$*. For all unauthorized* $U \notin \mathcal{A}_{N,t}$ *with* $|U| < t+\ell_p$ *and all* $\{\mathsf{sk}_b^{(j)}\}_j \subset (\mathbb{Z}_q^{\ell_p})^k$ *for* $b \in \{0,1\}$*:*

$$\left\{\{\mathsf{sk}_i^{(j)}\}_j, T_i, \pi_i\}_{i\in U}\right\}_{\mathsf{Share}'(\{\mathsf{sk}_0^{(j)}\}_j, \mathcal{A}_{N,t})} \overset{c}{\approx} \left\{\{\mathsf{sk}_i^{(j)}\}_j, T_i, \pi_i\}_{i\in U}\right\}_{\mathsf{Share}'(\{\mathsf{sk}_1^{(j)}\}_j, \mathcal{A}_{N,t})}$$

5.2 Threshold Fully Homomorphic Encryption from Torus

Considering the Torus-based FHE (Definition 17) from Chillotti et al. [11] as Torus-FHE, the Definition 4 combines TreeSSS$'$ with Torus-FHE to achieve an efficient threshold fully homomorphic encryption as TFHE$'''$.

Definition 4 (Batched and Packed Threshold Torus-FHE). *For parameters* $n = \mathsf{poly}(\lambda)$*, ring dimension* $N_{ring} = 2^p$*, decomposition base* B_g*, gadget length* ℓ_g*, Gaussian parameters* $\alpha, \bar{\alpha}, \gamma \in \mathbb{R}_{>0}$*, threshold parameters* (N,t) *for* N *parties with threshold* t*, TreeSSS parameters* $s \geq 3$*,* $\ell = 2s-1$*,* $L \geq \log_{c_s} N + \log_s N + O(1)$ *where* $c_s = \frac{\ell}{2^{\ell-1}} \cdot \binom{\ell-1}{s-1}$*, batching parameter* $k = n$ *(number of secret key bits), packing parameter* $\ell_p \geq 1$ *such that* $N \geq 2t+\ell_p$*, prime* q *with* $(N!)^2 \leq q$*, and noise flooding bound* B_{sm} *satisfying* $\alpha + \frac{((2s-1)!)^{2L} \cdot (2s-1)^L \cdot B_{sm}}{2^\lambda} = negl(\lambda)$*, the Batched and Packed Threshold Torus-*FHE *as* $\mathsf{TFHE}''' = (\mathsf{Setup}, \mathsf{Encrypt}, \mathsf{Eval}, \mathsf{PartDec}, \mathsf{FinDec})$ *operates as:*

$\mathsf{TreeSSS}'.\mathsf{Share}'(\{\mathsf{sk}^{(j)}\}_{j\in[k]},\mathcal{A}_{N,t})$

$k\leftarrow\lceil n/\ell_p\rceil,\quad \mathrm{share}_1^{(0,j)}\leftarrow\mathsf{sk}^{(j)}\ \forall j\in[k]$
for $i=1,\ldots,L$ **do**
  for $j\in[\ell^{i-1}]$ **do**
    $\forall\kappa\in[k].\ f_j^{(i,\kappa)}(X)\xleftarrow{R}\mathbb{Z}_q[X]_{s+\ell_p-2}: f_j^{(i,\kappa)}(\alpha)=\mathrm{share}_{j,\alpha}^{(i-1,\kappa)},\alpha\in\{-(\ell_p-1),\ldots,0\}$
    $r_j^{(i)}(X)\xleftarrow{R}\mathbb{Z}_q[X]_{s+\ell_p-2}$
    for $\nu\in[\ell]$ **do**
      $\gamma^{(i)}_{\ell(j-1)+\nu}\xleftarrow{R}\mathbb{Z}_q$
      $c^{(i)}_{\ell(j-1)+\nu}\leftarrow\mathcal{C}((f_j^{(i,1)}(\nu),\ldots,f_j^{(i,k)}(\nu),r_j^{(i)}(\nu)),\gamma^{(i)}_{\ell(j-1)+\nu})$
    $(d_1^{(i)},\ldots,d_k^{(i)})\leftarrow\mathcal{H}(c^{(i)}_{\ell(j-1)+1},\ldots,c^{(i)}_{\ell j})$
    $z_j^{(i)}(X)\leftarrow r_j^{(i)}(X)+\sum_{\kappa=1}^k(d_\kappa^{(i)})^\kappa\cdot f_j^{(i,\kappa)}(X)$
    $\forall\nu\in[\ell],\kappa\in[k].\ \mathrm{share}^{(i,\kappa)}_{\ell(j-1)+\nu}\leftarrow f_j^{(i,\kappa)}(\nu)$
$\forall m\in[\ell^L].\ \kappa\xleftarrow{R}[N],T_\kappa\leftarrow T_\kappa\cup\{m\}$
$\forall i\in[N].\ \mathsf{sk}_i^{(\kappa)}\leftarrow\{\mathrm{share}_m^{(L,\kappa)}\}_{m\in T_i}\ \forall\kappa\in[k]$
$\forall i\in[N].\ \pi_i\leftarrow\{(c_m^{(L)},z^{(L)}_{\lfloor m/\ell\rfloor}(m \bmod \ell),\gamma_m^{(L)})\}_{m\in T_i}$
return $(\{\{\mathsf{sk}_i^{(\kappa)}\}_{\kappa\in[k]},T_i,\pi_i\}_{i\in[N]})$

$\mathsf{TreeSSS}'.\mathsf{Combine}'(\{(\{\mathsf{sk}_i^{(\kappa)}\}_\kappa,T_i,\pi_i)\}_{i\in S})$

if $|S|<t+\ell_p$ **then**
  return $\perp$
$(d_1^{(L)},\ldots,d_k^{(L)})\leftarrow\mathcal{H}(c_1^{(L)},\ldots,c_{\ell^L}^{(L)})$
$\forall m\in\bigcup_{i\in S}T_i.\ (c_m^{(L)},z^{(L)}_{\lfloor m/\ell\rfloor}(m \bmod \ell),\gamma_m^{(L)}):=\pi_i$
$\forall m\in\bigcup_{i\in S}T_i.\ c_m^{(L)}\stackrel{?}{=}\mathcal{C}((f^{(L,1)}_{\lfloor m/\ell\rfloor}(m \bmod \ell),\ldots,f^{(L,k)}_{\lfloor m/\ell\rfloor}(m \bmod \ell),$
  $z^{(L)}_{\lfloor m/\ell\rfloor}(m \bmod \ell)-\sum_{\kappa=1}^k(d_\kappa^{(L)})^\kappa\cdot f^{(L,\kappa)}_{\lfloor m/\ell\rfloor}(m \bmod \ell)),\gamma_m^{(L)})$
for $\iota=L,\ldots,1$ **do**
  $\forall\kappa\in[k].\ T^{(\iota,\kappa)}\leftarrow\{m_\iota\,|\,\mathrm{share}^{(\iota,\kappa)}_{m_\iota}\in\bigcup_{i\in S}T_i\}$
  $\forall m_\iota\in T^{(\iota,\kappa)},\kappa\in[k].\ S^{(\iota,\kappa)}_{m_\iota}\subseteq T^{(\iota,\kappa)}$
  $\forall m_\iota\in T^{(\iota,\kappa)},\kappa\in[k].\ \lambda_{m_\iota}^{S^{(\iota,\kappa)}_{m_\iota}}\leftarrow\prod_{\mu\in S^{(\iota,\kappa)}_{m_\iota}\setminus\{m_\iota\}}\frac{0-\mu}{m_\iota-\mu}$
$\forall\kappa\in[k],\alpha\in\{-(\ell_p-1),\ldots,0\}.\ \hat{\mathsf{sk}}_\alpha^{(\kappa)}\leftarrow\sum_{m_L\in T^{(L,\kappa)}}(\prod_{\iota=1}^L\lambda_{m_\iota}^{S^{(\iota,\kappa)}_{m_\iota}})\cdot\mathrm{share}^{(L,\kappa)}_{m_L}$
return $\{\hat{\mathsf{sk}}_\alpha^{(\kappa)}\}_{\alpha\in\{-(\ell_p-1),\ldots,0\},\kappa\in[k]}$

Fig. 5: Batched and Packed Tree Secret Sharing Scheme $\mathsf{TreeSSS}'$

– $(\mathsf{pk},\{\mathsf{sk}_i\}_{i\in[N]})\leftarrow\mathsf{Setup}(1^\lambda,1^d,\mathcal{A}_{N,t})$: *Sample the binary secret keys* $K=(K_1,\ldots,K_n)\in\mathbb{B}^n$ *and* $\bar{K}\in\mathbb{B}^{\bar{n}}$ *with* TRLWE *interpretation* $\bar{K}\in\mathbb{B}_{N_{ring}}[X]^{\bar{k}}$. *Generate bootstrapping key* $\mathsf{BK}_{K\to\bar{K},\bar{\alpha}}=(\mathsf{BK}_j\in\mathsf{TRGSW}_{\bar{K},\bar{\alpha}}(K_j))_{j\in[n]}$ *and key-switching key* $\mathsf{KS}_{\bar{K}\to K,\gamma}=(\mathsf{KS}_j^{(\tau)}\in\mathsf{TLWE}_{K,\gamma}(\bar{K}_j/2^\tau))_{j\in[\bar{n}],\tau\in[t_{ks}]}$. *Partition the* n *key bits into* $\lceil n/\ell_p\rceil$ *groups and pad each group to length* ℓ_p, *forming* $k'=\lceil n/\ell_p\rceil$ *vectors* $\{\mathsf{sk}^{(j)}\}_{j\in[k']}$ *where* $\mathsf{sk}^{(j)}=(K_{(j-1)\ell_p+1},\ldots,K_{j\ell_p})\in\mathbb{Z}_q^{\ell_p}$ *(padding with zeros if*

needed). Run $\{\{\mathsf{sk}_i^{(j)}\}_{j\in[k']},T_i,\pi_i\}_{i\in[N]} \leftarrow \mathsf{TreeSSS}'.\ \mathsf{Share}'(\{\mathsf{sk}^{(j)}\}_{j\in[k']},\mathcal{A}_{N,t})$ *to distribute secret key shares among* N *parties. Output the value of public key* $\mathsf{pk}=(\mathsf{BK},\mathsf{KS},\{\pi_i\}_{i\in[N]})$ *and secret key shares* $\mathsf{sk}_i=(\{\mathsf{sk}_i^{(j)}\}_{j\in[k']},T_i)$ *for* $i\in[N]$.

- $ct\leftarrow\mathsf{Encrypt}(\mathsf{pk},\mu)$*: For message* $\mu\in\mathbb{T}$ *where* $\mathbb{T}=\mathbb{R}/\mathbb{Z}$*, sample* $\mathbf{a}\xleftarrow{R}\mathbb{T}^n$*,* $e\leftarrow\mathcal{D}_{\mathbb{T},\alpha}$*, and output* TLWE *ciphertext* $ct=(\mathbf{a},b)\in\mathbb{T}^{n+1}$ *where* $b=\mathbf{a}\cdot K+e+\mu \pmod 1$.
- $\hat{ct}\leftarrow\mathsf{Eval}(\mathsf{pk},C,ct_1,\ldots,ct_\kappa)$*: For circuit* $C:\{0,1\}^\kappa\to\{0,1\}$ *of depth at most* d *and* TLWE *ciphertexts* $ct_1,\ldots,ct_\kappa$*, perform homomorphic evaluation via linear combinations* $\sum e_i\cdot ct_i$ *for* $e_i\in\mathbb{Z}$*, external product* $C\boxdot ct=\mathsf{Dec}_{H,\beta,\epsilon}(ct)\cdot C$ *for* TRGSW C *and* TLWE ct*, CMux gate* $CMux(C,d_1,d_0)=C\boxdot(d_1-d_0)+d_0$*, and gate bootstrapping by utilizing the function of* $\mathsf{Bootstrap}_{\mu_1}(ct,\mathsf{BK})=\mathsf{KeySwitch}(\mathsf{BootstrapTLWE}(ct,\mathsf{BK}),\mathsf{KS})$ *where* BootstrapTLWE *uses blind rotation with test vector* $v=(1+X+\cdots+X^{\bar{N}_{ring}-1})\cdot X^{\bar{N}_{ring}/2}\cdot\mu$ *to output* TLWE *sample under* $\bar{K}$*, then* KeySwitch *converts back to key* K*. Output evaluated ciphertext* $\hat{ct}\in\mathbb{T}^{n+1}$.
- $p_i\leftarrow\mathsf{PartDec}(\mathsf{sk}_i,\hat{ct})$*: For party* P_i *with secret key share* $\mathsf{sk}_i=(\{\mathsf{sk}_i^{(j)}\}_{j\in[k']},T_i)$ *and evaluated ciphertext* $\hat{ct}=(\mathbf{a},b)$*, reconstruct partial key bits* $\{\hat{K}_m^{(j)}\}_{j\in[k'],m\in T_i}$ *from shares* $\{\mathsf{sk}_i^{(j)}\}_{j\in[k']}$*, compute partial phase* $\varphi_i=\sum_{m\in T_i,j:(j-1)\ell_p<m\le j\ell_p}\mathbf{a}_m\cdot\hat{K}_m^{(j)}$ *where* $\mathbf{a}_m$ *is the* m*-th component of* $\mathbf{a}$*, sample noise flooding error* $e_i\leftarrow\mathcal{U}([-B_{sm},B_{sm}])$*, and output partial decryption* $p_i=\varphi_i+((2s-1)!)^L\cdot e_i \pmod 1$.
- $\hat{\mu}\leftarrow\mathsf{FinDec}(\{(p_i,\mathsf{sk}_i)\}_{i\in S})$*: For subset* $S\subseteq[N]$ *with* $|S|\ge t+\ell_p$ *and decryptions* $\{p_i\}_{i\in S}$*, run* $\{\hat{\mathsf{sk}}^{(j)}\}_{j\in[k']}\leftarrow\mathsf{TreeSSS}'.\ \mathsf{Combine}'(\{(\{\mathsf{sk}_i^{(j)}\}_{j\in[k']},T_i,\pi_i)\}_{i\in S})$ *to obtain recovery coefficients* $\{c_m^{S,j}\}_{m,j}$ *such that* $\hat{K}_\nu=\sum_{m\in T^{(L,j)}}c_m^{S,j}\cdot\hat{K}_m^{(j)}$ *for* $\nu=(j-1)\ell_p+1,\ldots,j\ell_p$*. Compute combined phase* $\varphi_S=b-\sum_{i\in S}c_i^S\cdot p_i \pmod 1$ *where* c_i^S *aggregates coefficients for party* P_i*'s shares, and output* $\hat{\mu}=\lfloor 2\cdot\varphi_S\rceil\in\{0,1,\perp\}$.

In the following, the compactness (Definition 19), correctness (Definition 20), and security (Definition 21) properties of the TFHE''' encryption scheme (Definition 4) are addressed by Theorems 3, 4, and 5, respectively.

Theorem 3. *Let the scheme of* $\mathsf{TFHE}'''=(\mathsf{Setup},\mathsf{Encrypt},\mathsf{Eval},\mathsf{PartDec},\mathsf{FinDec})$ *as defined in Definition 4. There exist polynomials* $\mathsf{poly}_1,\mathsf{poly}_2$ *such that for all security parameters* λ*, all depth bounds* d*, all circuits* $C:\{0,1\}^\kappa\to\{0,1\}$ *with depth at most* d*, and all messages* $\mu\in\mathbb{T}$*,* $|\hat{ct}|\le\mathsf{poly}_1(\lambda,d,n),|p_i|\le\mathsf{poly}_2(\lambda,d,n,N)$*, where* $\hat{ct}\leftarrow\mathsf{Eval}(\mathsf{pk},C,ct_1,\ldots,ct_\kappa)$ *with* $ct_j\leftarrow\mathsf{Encrypt}(\mathsf{pk},\mu_j)$ *and* $p_i\leftarrow\mathsf{PartDec}(\mathsf{sk}_i,\hat{ct})$ *for any* $i\in[N]$.

Theorem 4. *Let the scheme of* $\mathsf{TFHE}'''=(\mathsf{Setup},\mathsf{Encrypt},\mathsf{Eval},\mathsf{PartDec},\mathsf{FinDec})$ *as defined in Definition 4. For all security parameters* λ*, all depth bounds* d*, all threshold access structures* $\mathcal{A}_{N,t}$*, all authorized subsets* $S\in\mathcal{A}_{N,t}$ *with* $|S|\ge t+\ell_p$*, all circuits* $C:\{0,1\}^\kappa\to\{0,1\}$ *with depth at most* d*, and all messages* $\mu_1,\ldots,\mu_\kappa\in\{0,1\}$*:*

$$\Pr\left[\begin{array}{l}(\mathsf{pk},\{\mathsf{sk}_i\}_i)\leftarrow\mathsf{Setup}(1^\lambda,1^d,\mathcal{A}_{N,t});\forall j\in[\kappa]:ct_j\leftarrow\mathsf{Encrypt}(\mathsf{pk},\mu_j);\\ \hat{ct}\leftarrow\mathsf{Eval}(\mathsf{pk},C,ct_1,\ldots,ct_\kappa);\forall i\in S:p_i\leftarrow\mathsf{PartDec}(\mathsf{sk}_i,\hat{ct});\\ \hat{\mu}\leftarrow\mathsf{FinDec}(\{(p_i,\mathsf{sk}_i)\}_{i\in S});\hat{\mu}=C(\mu_1,\ldots,\mu_\kappa)\end{array}\right]\ge 1-\mathsf{negl}(\lambda).$$

Theorem 5. *Let* $\mathsf{TFHE}''' = (\mathsf{Setup},\mathsf{Encrypt},\mathsf{Eval},\mathsf{PartDec},\mathsf{FinDec})$ *as defined in Definition 4. Under the decisional* $\mathsf{TLWE}(n,\chi)$ *assumption, for all PPT adversaries* $\mathcal{A}$*, all security parameters* λ*, all depth bounds* d*, and all threshold access structures* $\mathcal{A}_{N,t}$*:*

$$\left|\Pr\left[\begin{array}{l}(\mathsf{pk},\{\mathsf{sk}_i\}_i)\leftarrow\mathsf{Setup}(1^\lambda,1^d,\mathcal{A}_{N,t});U\leftarrow\mathcal{A}(\mathsf{pk}):U\notin\mathcal{A}_{N,t};\\ \beta\leftarrow\{0,1\};ct\leftarrow\mathsf{Encrypt}(\mathsf{pk},\beta);\beta'\leftarrow\mathcal{A}(\{\mathsf{sk}_i\}_{i\in U},ct);\beta=\beta'\end{array}\right]-\frac{1}{2}\right|\leq\mathsf{negl}(\lambda).$$

Putting TFHE''' together with PZK and Com, we achieve UT_6, which brings more efficiency than other π-based UTs such as UT_2, UT_4, and UT_5. Figure 6 provides more details on this construction.

The universal thresholdizer UT_6 is formalized in the Figure 6. In the following, Theorems 6, 7, 8, 9 address the compactness, evaluation correctness, security, and robustness of UT_6, respectively.

Theorem 6. *Let* $\mathsf{UT}_6 = (\mathsf{Setup},\mathsf{Eval},\mathsf{Verify},\mathsf{Combine})$ *as defined in Figure 6. There exist polynomials* $\mathsf{poly}_1,\mathsf{poly}_2$ *such that for all security parameters* λ*, all depth bounds* d*, all access structures* $(\mathcal{A}_{N,t})$*, all circuits* $\mathbb{C}:\{0,1\}^k\rightarrow\{0,1\}$ *with depth at most* d*, and all messages* $x\in\{0,1\}^k$*:* $|y_i|\leq\mathsf{poly}_1(\lambda,d,n,|T_i|)$*, and* $|\mathsf{pp}|\leq\mathsf{poly}_2(\lambda,d,n,N)$*, where* $(\mathsf{pp},s_1,...,s_N)\leftarrow\mathsf{Setup}(1^\lambda,1^d,\mathcal{A}_{N,t})$ *and* $y_i\leftarrow\mathsf{Eval}(\mathsf{pp},s_i,x,\mathbb{C})$*.*

Theorem 7. *Let* $\mathsf{UT}_6 = (\mathsf{Setup},\mathsf{Eval},\mathsf{Verify},\mathsf{Combine})$ *as defined in Figure 6. For all security parameters* λ*, all depth bounds* d*, all access structures* $(\mathcal{A}_{N,t})$*, all circuits* $\mathbb{C}:\{0,1\}^k\rightarrow\{0,1\}$ *with depth at most* d*, all messages* $x\in\{0,1\}^k$*, and all parties* $i\in[N]$*:*

$$\Pr\left[\begin{array}{l}(\mathsf{pp},s_1,...,s_N)\leftarrow\mathsf{Setup}(1^\lambda,1^d,\mathcal{A}_{N,t});\\ y_i\leftarrow\mathsf{Eval}(\mathsf{pp},s_i,x,\mathbb{C});\mathsf{Verify}(\mathsf{pp},y_i,x,\mathbb{C})=1\end{array}\right]\geq 1-\mathsf{negl}(\lambda).$$

Theorem 8. *Let* $\mathsf{UT}_6 = (\mathsf{Setup},\mathsf{Eval},\mathsf{Verify},\mathsf{Combine})$ *as defined in Figure 6. Under the security of* TFHE''' *(Theorem 5), computational hiding of* Com*, and zero-knowledge of* PZK*, there exists PPT simulator* $\mathcal{S}_6 = (\mathcal{S}_{6,1},\mathcal{S}_{6,2},\mathcal{S}_{6,3})$ *such that for all PPT adversaries* $\mathcal{A}$*:*

$$\left|\Pr[\mathsf{Exp}^{real}_{\mathcal{A},\mathsf{UT}_6}(1^\lambda,1^d)=1]-\Pr[\mathsf{Exp}^{ideal}_{\mathcal{A},\mathsf{UT}_6}(1^\lambda,1^d)=1]\right|\leq\mathsf{negl}(\lambda),$$

where experiments follow Definition 2 structure.

Theorem 9. *Let* $\mathsf{UT}_6 = (\mathsf{Setup},\mathsf{Eval},\mathsf{Verify},\mathsf{Combine})$ *as defined in Figure 6. Under binding of* Com *and soundness of* PZK*, for all PPT adversaries* $\mathcal{A}$*:*

$$\Pr\left[\begin{array}{l}(\mathbb{S},N)\leftarrow\mathcal{A}(1^\lambda,1^d);(\mathsf{pp},s_1,...,s_N)\leftarrow\mathsf{Setup}(1^\lambda,1^d,(\mathbb{S},N),(\mathbb{C},d));\\ (x,C^*,i^*)\leftarrow\mathcal{A}(\mathsf{pp},s_1,...,s_N);y_{i^*}=(p_{i^*},\pi_{i^*})\leftarrow\mathsf{Eval}(\mathsf{pp},s_{i^*},x,C^*);\\ y^*=(p^*,\pi^*)\leftarrow\mathcal{A};b_1\leftarrow(p^*\neq p_{i^*});b_2\leftarrow\mathsf{Verify}(\mathsf{pp},y^*,x,C^*);(b_1\wedge b_2)\end{array}\right]\leq\mathsf{negl}(\lambda).$$

$\mathsf{UT}_6.\mathsf{Setup}(1^\lambda,1^d,\mathcal{A}_{N,t})$

$(\mathsf{fhepk},\mathsf{fhesk}) \leftarrow \mathsf{FHE}''.\mathsf{Setup}(1^\lambda,1^d)$
$k' \leftarrow \lceil |\mathsf{fhesk}|/\ell_p \rceil$
$\forall j \in [k'].\, \mathsf{sk}^{(j)} \leftarrow (\mathsf{fhesk}_{(j-1)\ell_p+1},\ldots,\mathsf{fhesk}_{j\ell_p}) \in \mathbb{Z}_q^{\ell_p}$
$(\{\{\mathrm{share}_m^{(L,j)}\}_{m\in[\ell^L]}\}_{j\in[k']},\{T_i\}_{i\in[N]},\{\pi_i^{\mathrm{tree}}\}_{i\in[N]}) \leftarrow \mathsf{TreeSSS}'.\mathsf{Share}(\{\mathsf{sk}^{(j)}\}_{j\in[k']},\mathcal{A}_{N,t})$
$\forall i \in [N].\, (\sigma_{V,i},\sigma_{P,i}) \leftarrow \mathsf{PZK}.\mathsf{Pre}(1^\lambda)$
$\forall i \in [N].\, r_i \xleftarrow{R} \{0,1\}^\lambda$
$\forall i \in [N].\, \mathsf{com}_i \leftarrow \mathsf{Com}((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},T_i);r_i)$
$\mathsf{pp} \leftarrow (\mathsf{fhepk},\{\pi_i^{\mathrm{tree}}\}_{i\in[N]},\{\sigma_{V,i}\}_{i\in[N]},\{\mathsf{com}_i\}_{i\in[N]})$
$\forall i \in [N].\, s_i \leftarrow ((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,\sigma_{P,i},r_i)$
return $(\mathsf{pp},s_1,\ldots,s_N)$

$\mathsf{UT}_6.\mathsf{Eval}(\mathsf{pp},s_i,x,\mathbb{C})$

$\forall j \in [|x|].\, \mathsf{ct}_j \leftarrow \mathsf{FHE}''.\mathsf{Encrypt}(\mathsf{fhepk},x_j)$
$\hat{\mathsf{ct}} \leftarrow \mathsf{FHE}''.\mathsf{Eval}(\mathsf{fhepk},\mathbb{C},\{\mathsf{ct}_j\}_{j\in[|x|]})$
$((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,\sigma_{P,i},r_i) \leftarrow s_i$
$\forall j \in [k'], m \in T_i.\, \hat{K}_m^{(j)} \leftarrow \mathrm{share}_m^{(L,j)}$
$\varphi_i \leftarrow \mathsf{FHE}''.\mathsf{Dec}_0(\{\hat{K}_m^{(j)}\}_{m\in T_i,j\in[k']},\hat{\mathsf{ct}})$
$e_i \xleftarrow{R} \mathcal{U}([-B_{\mathrm{sm}},B_{\mathrm{sm}}])$
$p_i \leftarrow \varphi_i + ((2s-1)!)^L \cdot e_i \bmod 1$
$\Psi_i : \exists((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,r_i,e_i):$
  $\mathsf{com}_i = \mathsf{Com}((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},T_i);r_i)$
  $\wedge\ \mathsf{TreeSSS}'.\mathsf{Verify}(\pi_i^{\mathrm{tree}},\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']}) = 1$
  $\wedge\ p_i = \mathsf{FHE}''.\mathsf{Dec}_0(\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},\hat{\mathsf{ct}}) + ((2s-1)!)^L \cdot e_i \bmod 1$
$\pi_i \leftarrow \mathsf{PZK}.\mathsf{Prove}(\sigma_{P,i},\Psi_i,((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,r_i,e_i))$
return $y_i \leftarrow (p_i,\pi_i)$

$\mathsf{UT}_6.\mathsf{Verify}(\mathsf{pp},y_i,x,\mathbb{C})$

$\forall j \in [|x|].\, \mathsf{ct}_j \leftarrow \mathsf{FHE}''.\mathsf{Encrypt}(\mathsf{fhepk},x_j)$
$\hat{\mathsf{ct}} \leftarrow \mathsf{FHE}''.\mathsf{Eval}(\mathsf{fhepk},\mathbb{C},\{\mathsf{ct}_j\}_{j\in[|x|]})$
$(p_i,\pi_i) \leftarrow y_i$
$\Psi_i : \exists((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,r_i,e_i):$
  $\mathsf{com}_i = \mathsf{Com}((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},T_i);r_i)$
  $\wedge\ \mathsf{TreeSSS}'.\mathsf{Verify}(\pi_i^{\mathrm{tree}},\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']}) = 1$
  $\wedge\ p_i = \mathsf{FHE}''.\mathsf{Dec}_0(\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},\hat{\mathsf{ct}}) + ((2s-1)!)^L \cdot e_i \bmod 1$
return $\mathsf{PZK}.\mathsf{Verify}(\sigma_{V,i},\Psi_i,\pi_i)$

$\mathsf{UT}_6.\mathsf{Combine}(\mathsf{pp},B,x,\mathbb{C})$

$B \leftarrow \{y_i\}_{i\in S} : S \subseteq [N] \wedge |S| \geq t+\ell_p$
if $|S| < t+\ell_p$ **then**
  return $\perp$
$\forall i \in S : y_i \leftarrow (p_i,\pi_i)$
$\{\hat{\mathsf{sk}}^{(j)}\}_{j\in[k']} \leftarrow \mathsf{TreeSSS}'.\mathsf{Combine}(\{((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,\pi_i^{\mathrm{tree}}\}_{i\in S})$
$\forall i \in S.\, c_i^S \leftarrow (\text{reconstruction coefficient for party } i)$
$\forall j \in [|x|].\, \mathsf{ct}_j \leftarrow \mathsf{FHE}''.\mathsf{Encrypt}(\mathsf{fhepk},x_j)$
$\hat{\mathsf{ct}} \leftarrow \mathsf{FHE}''.\mathsf{Eval}(\mathsf{fhepk},\mathbb{C},\{\mathsf{ct}_j\}_{j\in[|x|]})$
$(\mathbf{a},b) \leftarrow \hat{\mathsf{ct}}$
$\varphi_S \leftarrow b - \sum_{i\in S} c_i^S \cdot p_i \bmod 1$
$\hat{\mu} \leftarrow \mathsf{FHE}''.\mathsf{Dec}_1(\varphi_S)$
return $\hat{\mu}$

Fig. 6: Universal Thresholdizer from FHE'' [11] + $\mathsf{TreeSSS}'$ [8] + PZK [14] + Com [15]

6 Our Signature-based UT from Fully Homomorphic Signature

As a σ-based universal thresholdizer, UT_7 is an efficient alternative to UT_3, which consists of TFHE''' (Definition 4) and a different HS than the one used in [7], called HS' and demonstrated in the Definition 5. Using TFHE''' and HS', we design UT_7, presented in Figure 7.

Definition 5 (Fully-Homomorphic Signature [27]). *For security parameter λ, data-size bound N, and parameters κ_1,κ_2 chosen according to Section 3.1, the scheme* $\mathsf{HS}'=(\mathsf{KeyGen},\mathsf{Sign},\mathsf{Verify},\mathsf{Eval})$ *operates as:*

- $(\mathsf{vk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda,1^N)$*: Computes* $(\mathsf{fpk},\mathsf{fsk}) \leftarrow \mathsf{FHE.Setup}(1^{\kappa_1})$*, generates FHE encryptions* $\{\mathsf{ct}'_i \leftarrow \mathsf{FHE.Enc}(\mathsf{fpk},0)\}_{i\in[N]}$*, samples PRF keys* $K_1,K_2 \leftarrow \mathsf{PRF.KeyGen}(1^{\kappa_2})$*, obfuscates helper circuits* $\mathsf{ObfGenCRS} \leftarrow \mathsf{iO}(1^{\kappa_2},\mathsf{PubGenCRS})$ *and* $\mathsf{ObfEval} \leftarrow \mathsf{iO}(1^{\kappa_2},\mathsf{EvalNAND})$*. Outputs verification key as the value* $\mathsf{vk}=(\mathsf{fpk},\{\mathsf{ct}'_i\}_{i\in[N]},\mathsf{ObfGenCRS},\mathsf{ObfEval})$ *and signing key* $\mathsf{sk}=(K_1,K_2,\mathsf{fsk})$.
- $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk},m,i)$*: For message* $m\in\{0,1\}$ *and index* $i\in[N]$*, computes* $(\mathsf{crs}_{sim},\mathsf{td}_{sim})=\mathsf{GenCRS}(0)$ *where* GenCRS *uses* K_1 *to generate a simulated CRS via the sub-procedure* $\mathsf{NIZK.SimSetup}(1^{\kappa_2};\mathsf{PRF}(K_1,0))$*. Generates simulated proof using* $\pi \leftarrow \mathsf{NIZK.Sim}(\mathsf{crs}_{sim},\mathsf{td}_{sim},stat_{m,\mathsf{ct}'_i})$ *for the statement that* ct'_i *encrypts* m*. Outputs* $\sigma=(\mathsf{ct}'_i,\pi,0)$.
- $b \leftarrow \mathsf{Verify}(\mathsf{vk},f,y,\sigma)$*: Parses* σ *as* (ct,π,ℓ) *where* ℓ *denotes the evaluation level. Computes* $\mathsf{ct}_f=\mathsf{FHE.Eval}(\mathsf{fpk},f,\mathsf{ct}'_1,\ldots,\mathsf{ct}'_N)$ *and* $\mathsf{crs}=\mathsf{ObfGenCRS}(\ell)$*. Outputs the result of* $\mathsf{NIZK.Verify}(\mathsf{crs},stat_{y,\mathsf{ct}_f},\pi)$.
- $\sigma_f \leftarrow \mathsf{Eval}(\mathsf{vk},f,(m_1,\sigma_1),\ldots,(m_N,\sigma_N))$*: Evaluates the circuit* f *gate-by-gate using* ObfEval*. For each binary NAND gate with inputs* (σ_0,m_0) *and* (σ_1,m_1) *at level* j*, the obfuscated evaluation circuit (hardcoded with* K_2*) verifies the input proofs, computes* $\mathsf{ct}=\mathsf{FHE.Eval}(\mathsf{fpk},NAND,\mathsf{ct}_0,\mathsf{ct}_1)$ *and* $m=NAND(m_0,m_1)$*, generates* $(\mathsf{crs}'_{sim},\mathsf{td}'_{sim})=\mathsf{GenCRS}(j+1)$*, and then it produces the value of* $\pi=\mathsf{NIZK.Sim}(\mathsf{crs}'_{sim},\mathsf{td}'_{sim},stat_{m,\mathsf{ct}};\mathsf{PRF}(K_2,(m,\mathsf{ct},j+1)))$*. Returns the final signature* $\sigma_f=(\mathsf{ct}_f,\pi_f,d)$ *where* d *is the depth of* f.

In [27], it has been proven that HS' satisfies *computational signing correctness*, *computational evaluation correctness*, *weak context hiding*, and the *adaptive unforgeability* under subexponentially-secure iO, FHE, NIZK with composable zero-knowledge and proof-of-knowledge, and puncturable PRF.

HS' is generally more time-efficient as its complexity grows linearly with circuit size $|f|$ (polynomial in security parameters), whereas HS has quadratic dependence on the security parameter λ multiplied by both circuit size and depth ($\lambda^2\cdot|f|+\lambda\cdot|f|\cdot d$). By putting FHE'', $\mathsf{TreeSSS}'$, and HS' together, our signature-based universal thresholdizer can be achieved as presented in Figure 7.

Compactness, evaluation correctness, security, and robustness of UT_7 is addressed by Theorems 10, 11, 12, 13.

Theorem 10. *Let* $\mathsf{UT}_7=(\mathsf{Setup},\mathsf{PartEval},\mathsf{VfyEval},\mathsf{Combine})$ *as defined in Figure 7. There exist polynomials* $\mathsf{poly}_1,\mathsf{poly}_2$ *such that for all security parameters* λ*, all depth bounds* d*, all circuits* $\mathbb{C}$ *with depth at most* d*:* $|(y_i,\sigma_i)| \le \mathsf{poly}_1(\lambda,d,n)$*, and* $|\mathsf{pp}| \le \mathsf{poly}_2(\lambda,d,n,N)$*, where* $(\mathsf{pp},\mathsf{sk}) \leftarrow \mathsf{Setup}(1^\lambda,d,N,x)$ *and* $(y_i,\sigma_i) \leftarrow \mathsf{PartEval}(\mathsf{pp},\mathsf{sk}_i,i,\mathbb{C})$.

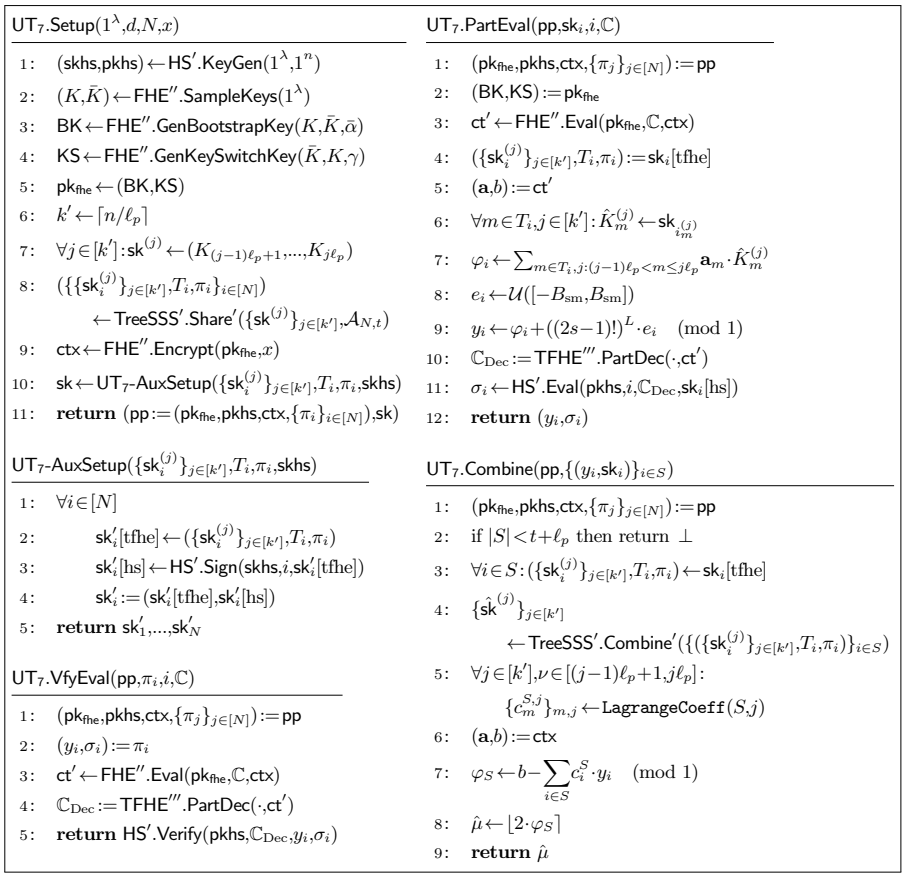

Fig. 7: Universal Thresholdizer UT_7 from FHE'' [11] +$\mathsf{TreeSSS}'$ [8] +HS' [27]

Theorem 11. *Let* $\mathsf{UT}_7 = (\mathsf{Setup},\mathsf{PartEval},\mathsf{VfyEval},\mathsf{Combine})$ *as defined in Figure 7. For all security parameters* λ*, all circuits* $\mathbb{C}$ *with depth at most* d*, and all parties* $i \in [N]$*:*

$$\Pr\left[\begin{array}{l}(\mathsf{pp},\mathsf{sk}) \leftarrow \mathsf{Setup}(1^\lambda,d,N,x); (y_i,\sigma_i) \leftarrow \mathsf{PartEval}(\mathsf{pp},\mathsf{sk}_i,i,\mathbb{C});\\ \mathit{VfyEval}(\mathsf{pp},(y_i,\sigma_i),i,\mathbb{C}) = 1\end{array}\right] \ge 1 - \mathsf{negl}(\lambda).$$

Theorem 12. *Let* $\mathsf{UT}_7 = (\mathsf{Setup},\mathsf{PartEval},\mathsf{VfyEval},\mathsf{Combine})$ *as defined in Figure 7. Under security of* TFHE''' *(Theorem 5) and context-hiding of* HS' *(Theorem 17), there exists PPT simulator* $\mathcal{S}_7 = (\mathcal{S}_{7,1},\mathcal{S}_{7,2},\mathcal{S}_{7,3})$ *such that:*

$$\left|\Pr[\mathsf{Exp}^{real}_{\mathcal{A},\mathsf{UT}_7}(1^\lambda,1^d) = 1] - \Pr[\mathsf{Exp}^{ideal}_{\mathcal{A},\mathsf{UT}_7}(1^\lambda,1^d) = 1]\right| \le \mathsf{negl}(\lambda).$$

Theorem 13. *Let* $\mathsf{UT}_7 = (\mathsf{Setup}, \mathsf{PartEval}, \mathsf{VfyEval}, \mathsf{Combine})$ *as defined in Figure 7. Under unforgeability of* HS' *(Theorem 16), for all PPT adversaries* $\mathcal{A}$*:*

$$\Pr\left[\begin{array}{l}(\mathbb{S},N)\leftarrow\mathcal{A}(1^\lambda,1^d);(\mathsf{pp},\mathsf{sk})\leftarrow\mathsf{Setup}(1^\lambda,d,N,x);\\(x,C^*,i^*)\leftarrow\mathcal{A}(\mathsf{pp},\mathsf{sk});(y_{i^*},\sigma_{i^*})\leftarrow\mathsf{PartEval}(\mathsf{pp},\mathsf{sk}_{i^*},i^*,C^*);\\(y^*,\sigma^*)\leftarrow\mathcal{A};b_1\leftarrow(y^*\neq y_{i^*});\\b_2\leftarrow\mathsf{VfyEval}(\mathsf{pp},(y^*,\sigma^*),i^*,C^*);(b_1\wedge b_2)\end{array}\right]\leq\mathsf{negl}(\lambda).$$

7 Efficiency Analysis of Design Methods

In this section, we compare the computational costs of each technique for UT construction, where Tables 2 and 1 present the summary of our study of their efficiency, and an in-depth complexity investigation is provided in Appendix A.

In UT_6, the Torus-based FHE scheme (used in UT_6 and UT_7) is more efficient than Modified GSW (used in UT_2) for both encryption and decryption operations. Modified GSW [5] exhibits total time complexity $O(n^2m^2+4nm+g\cdot n\cdot m\cdot\ell+gn\cdot m^2+4n+4m+m^2+1)$, which is dominated by the encryption phase's $O((n+1)m^2)$ matrix multiplication and the evaluation phase's $O(g(n+1)m^2)$ homomorphic multiplication operations. In contrast, Torus-based FHE achieves total time cost of $O(n+\bar{k}\bar{N}+n(\bar{k}+1)^2\ell\bar{k}\bar{N}+\bar{k}\bar{N}\cdot t\cdot n+g\cdot n(\bar{k}+1)^2\ell\bar{N}+g\cdot\bar{k}\bar{N}\cdot t\cdot n)$, where encryption requires only $O(n)$ operations and evaluation complexity is structured around ring polynomial operations and bootstrapping.

The batched and packed variant $\mathsf{TreeSSS}'$ achieves better efficiency over $\mathsf{TreeSSS}$ by reducing the per-secret communication complexity from $O(n\cdot\ell^L\cdot\log N)$ for k independent executions to $O((n+k)\cdot\ell^L\cdot\log N/(k\cdot\ell_p))$ through amortized proof aggregation (batching reduces verification overhead from $O(nk)$ to $O(n+k)$ commitments) and secret packing (embedding ℓ_p secrets per polynomial without increasing share size), yielding negligible overhead approaching plain Shamir sharing when $k,\ell_p\gg 1$, at the cost of reduced fault tolerance from optimal $t<n/2$ to $t<(n-\ell_p)/2$ due to increased reconstruction threshold $t+\ell_p$ required for packed secret recovery.

Putting FHE'' and $\mathsf{TreeSSS}'$ next to each other, we get TFHE''' that is more efficient than TFHE and TFHE', and TFHE'', as its total time cost is $O(|C|\cdot n\cdot\bar{k}\cdot\ell_g\cdot N_{\text{ring}})$ with cumulative space complexity of $O(n\cdot\bar{k}\cdot\ell_g\cdot N_{\text{ring}})$.

For UT_7, recalling the same cost of TFHE''' as in UT_6, the construction of UT_7 uses the same TFHE''' but with HS'. The homomorphic signature scheme HS' has evaluation time complexity $O(|f|\cdot\mathrm{poly}(\lambda))$ with linear scaling in circuit size, whereas HS requires $O(\lambda^2\cdot|f|+\lambda\cdot|f|\cdot d)$ operations introducing quadratic dependence on the security parameter λ and explicit depth factor d, with the advantage coming from the combined efficiency gains of TFHE''' over the TFHE construction and HS' over the HS scheme.

Furthermore, we did the first implementation[2] of universal thresholdizers focusing on UT_6 and UT_7. To validate the practical efficiency of our constructions, we implemented both UT_6 and UT_7 in `C++17`. This has been conducted on the Apple MacBook

[2] Accessible on `https://anonymous.4open.science/r/UniversalThresholdizer`.

Technique	Setup	Evaluation	Verification	Combination
UT_2 [5]	$O(\lambda{\cdot}d{+}N{\cdot}\ell{\cdot}n{+}k{\cdot}n{\cdot}\log q{+}N{\cdot}\lambda{\cdot}n{\cdot}\log q)$	$O(\lvert C\rvert{\cdot}n^2{\cdot}\log q{+}\lambda{\cdot}n{\cdot}\log q)$	$O(\lvert C\rvert{\cdot}n^2{\cdot}\log q{+}\lambda{\cdot}n{\cdot}\log q)$	$O(\ell{\cdot}N)$
Dynamic UT [Ours]	$O(\lambda{\cdot}d{+}N{\cdot}\ell{\cdot}n{+}k{\cdot}n{\cdot}\log q{+}N{\cdot}\lambda{\cdot}n{\cdot}\log q)$	$O(\lvert C\rvert{\cdot}n^2{\cdot}\log q{+}\lambda{\cdot}n{\cdot}\log q)$	$O(\lvert C\rvert{\cdot}n^2{\cdot}\log q{+}\lambda{\cdot}n{\cdot}\log q)$	$O(\ell{\cdot}N)$
UT'_3 [5] or UT_3 [7]	$O(\lambda{\cdot}d{+}N{\cdot}\ell{\cdot}n{+}n^2{\cdot}\log q{+}N{\cdot}\lambda)$	$O(\lvert C\rvert{\cdot}n^2{\cdot}\log q{+}\lambda{\cdot}n{\cdot}\log q)$	$O(\lvert C\rvert{\cdot}n^2{\cdot}\log q{+}\lambda{\cdot}n{\cdot}\log q)$	$O(d{\cdot}n{\cdot}\log q)$
2-UT [7]	$O(\lambda{+}\lvert x\rvert)$	$O(\lambda{\cdot}\lvert C\rvert{\cdot}\mathsf{poly}(\lambda))$	$O(\lambda{\cdot}\lvert C\rvert{\cdot}\mathsf{poly}(\lambda))$	$O(\mathrm{poly}(\lambda))$
UT_4 [6]	$O(2^d{\cdot}n{\cdot}m^2{\cdot}\log q{+}N^{8.4}{+}\lambda{\cdot}n{+}n{\cdot}m^2{\cdot}\log q{\cdot}\lvert x\rvert)$	$O(2^d{\cdot}n{\cdot}m^2{\cdot}\log q{\cdot}\lvert C\rvert{+}n{\cdot}m{+}\lambda{\cdot}\lvert\Psi\rvert{+}\lvert T_i\rvert)$	$O(2^d{\cdot}n{\cdot}m^2{\cdot}\log q{\cdot}\lvert C\rvert{+}\lambda{\cdot}\lvert\Psi\rvert)$	$O(t{\cdot}\lambda{\cdot}\lvert\Psi\rvert{+}N^{8.4})$
UT_5 [8]	$O(\lambda{\cdot}d{\cdot}n^2{\cdot}\log q{+}(2t{-}1)^L{\cdot}t^2{\cdot}\log q{+}\lambda{\cdot}\lvert F\rvert)$	$O(\lvert F(x)\rvert{\cdot}n^2{\cdot}\log q{+}\lambda{\cdot}\lvert F\rvert)$	$O(\lambda{\cdot}\lvert y\rvert{+}\lvert S\rvert{\cdot}(\lvert T_i\rvert{\cdot}n{\cdot}\log q))$	$O(N{+}\lvert T\rvert{\cdot}\log q{+}\lvert S\rvert{\cdot}\lambda{\cdot}\lvert y\rvert)$
UT_6 [Ours]	$O(\lambda{\cdot}d{\cdot}n{\cdot}\log q{+}n{\cdot}(2t{-}1)^L{\cdot}t{\cdot}\log q{+}N{\cdot}\lambda)$	$O(\lvert C\rvert{\cdot}n{\cdot}\log q{+}\lambda{\cdot}n{\cdot}\lvert T_i\rvert)$	$O(\lvert C\rvert{\cdot}n{\cdot}\log q{+}\lambda{\cdot}n{\cdot}\lvert T_i\rvert)$	$O(n{\cdot}(2t{-}1)^L{\cdot}t^2{\cdot}\log q)$
UT_7 [Ours]	$O(\lambda{\cdot}d{\cdot}n{\cdot}\log q{+}n{\cdot}(2t{-}1)^L{\cdot}t{\cdot}\log q{+}N{\cdot}\lambda)$	$O(\lvert C\rvert{\cdot}n{\cdot}\log q{+}n{\cdot}\lvert T_i\rvert{+}n{\cdot}\lambda)$	$O(\lvert C\rvert{\cdot}n{\cdot}\log q{+}n{\cdot}\lambda)$	$O(n{\cdot}(2t{-}1)^L{\cdot}t^2{\cdot}\log q)$

Table 2: Time Cost of Universal Thresholdizer's Construction Methods.

Air (2022) with the M2 chip, A15 Bionic (64-bit ARM-based), an 8-core CPU featuring a base clock speed of 3.49 GHz, and 16GB of memory, using Apple clang version 15.0.0 (clang-1500.3.9.4) with `-O3` optimization flags. Table 3 shows the benchmark results.

Measure \ Function	Setup	Evaluation	Verification	Combination
UT_6				
Time Cost	321.447×10^{-3} s	0.198×10^{-3} s	0.001×10^{-3} s	0.001×10^{-3} s
Space Size	864 bytes	240 bytes	3 bytes	1 bytes
UT_7				
Time Cost	296.051×10^{-3} s	0.121×10^{-3} s	0.002×10^{-3} s	0.343×10^{-3} s
Space Size	688 bytes	120 bytes	3 bytes	1 bytes

Table 3: For both constructions, the benchmark input consisted of randomly generated 3-bit boolean vectors encrypted under Torus-FHE and evaluated through `NAND` gate circuits with $N=5$ parties and threshold parameter $t=3$.

8 Conclusion

Our study establishes that existing UT constructions have challenges in providing optimal performance across dimensions of security, robustness, and efficiency. , e.g., the design of Boneh et al. [5] achieves both security and robustness at the cost of increased computational requirements, while the signature-based approach of Campanelli & Khoshakhlagh [7] offers reduced complexity but does not provide direct security proofs. The TreeSSS-based construction of Cheon et al. [8] provides superior communication efficiency for large committees, while the two-party construction using HSS is optimized for settings with exactly two participants. Our formal distinction between static and dynamic UTs, coupled with two concrete instantiations achieving asymptotic improvements in complexities, resolves open questions regarding the constructibility of efficient threshold-agnostic transformations. As future directions, apart from identifying the optimization of share distribution mechanisms in tree-based secret sharing for further asymptotic gains, utilizing silent-setup threshold schemes [39–42] can be a constructive idea to improve UT design cost; utilizing ZKPs for HE evaluations [43] to prove the correct execution of HE evaluations while protecting the server's private inputs can also strengthen existing UT security definitions.

References

1. Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979.
2. G. R. BLAKLEY. Safeguarding cryptographic keys. In *1979 International Workshop on Managing Requirements Knowledge (MARK)*, pages 313–318, 1979.
3. National Institute of Standards and Technology. Nist and multi-party threshold schemes. NIST Computer Security Resource Center News, January 2026.
4. Dan Boneh, Rosario Gennaro, Steven Goldfeder, and Sam Kim. A lattice-based universal thresholdizer for cryptographic systems. Cryptology ePrint Archive, Paper 2017/251, 2017.
5. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, pages 565–596. Springer International Publishing, 2018.
6. Ehasn Ebrahimi and Anshu Yadav. Strongly secure universal thresholdizer. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 207–239. Springer Nature Singapore, 2025.
7. Matteo Campanelli and Hamidreza Khoshakhlagh. Succinct publicly-certifiable proofs. In Avishek Adhikari, Ralf Küsters, and Bart Preneel, editors, *Progress in Cryptology – INDOCRYPT 2021*, pages 607–631, Cham, 2021. Springer International Publishing.
8. Jung Hee Cheon, Wonhee Cho, and Jiseung Kim. Improved universal thresholdizer from iterative shamir secret sharing. *Journal of Cryptology*, 38(15), 2025.
9. Josh Benaloh, Michael Naehrig, Olivier Pereira, and Dan S. Wallach. ElectionGuard: a cryptographic toolkit to enable verifiable elections. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5485–5502, Philadelphia, PA, August 2024. USENIX Association.
10. ElectionGuard - Official — electionguard.vote. `https://www.electionguard.vote/spec/`. [Accessed 20-07-2024].
11. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and et al. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology*, 33:34–91, 2020.
12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Trans. Comput. Theory*, 6(3), July 2014.
13. Aayush Jain, Peter M. R. Rasmussen, and Amit Sahai. Threshold fully homomorphic encryption. Cryptology ePrint Archive, Paper 2017/257, 2017.
14. Alfredo De Santis, Silvio Micali, and Giuseppe Persiano. Non-interactive zero-knowledge with preprocessing. In Shafi Goldwasser, editor, *Advances in Cryptology — CRYPTO' 88*, volume 403 of *LNCS*. Springer, New York, NY, 1990.
15. Manuel Blum. Coin flipping by telephone a protocol for solving impossible problems. *SIGACT News*, 15(1):23–27, January 1983.
16. Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) lwe. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, pages 97–106, 2011.
17. Dan Boneh, Kobbi Lewi, Henry Montgomery, and Ananth Raghunathan. Key homomorphic prfs and their applications. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013*, volume 8042 of *LNCS*. Springer, Berlin, Heidelberg, 2013.
18. Arvind Gupta and Sanjeev Mahajan. Using amplification to compute majority with small majority gates. *Comput. Complex.*, 6(1):46–63, January 1996.
19. Karim Baghery. Π: A unified framework for computational verifiable secret sharing. In Tibor Jager and Jia Pan, editors, *Public-Key Cryptography – PKC 2025*, volume 15677 of *Lecture Notes in Computer Science*. Springer, Cham, 2025.

20. Shahla Atapoor, Karim Baghery, Georgio Nicolas, Robi Pedersen, and Jannik Spiessens. Batched and packed (publicly) verifiable secret sharing: A unified framework and applications. Cryptology ePrint Archive, Paper 2025/2018, 2025.
21. Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013*, volume 8042 of *LNCS*. Springer, Berlin, Heidelberg, 2013.
22. Sergey Gorbunov, Vinod Vaikuntanathan, and Daniel Wichs. Leveled fully homomorphic signatures from standard lattices. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '15, page 469–477, New York, NY, USA, 2015. Association for Computing Machinery.
23. Eyal Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under ddh. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016*, volume 9814 of *LNCS*. Springer, Berlin, Heidelberg, 2016.
24. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO' 89 Proceedings*, volume 435 of *LNCS*. Springer, New York, NY, 1990.
25. Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *J. ACM*, 33(4):792–807, August 1986.
26. Ronald Cramer, Ivan Damgård, and Yuval Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In *Proceedings of the Second International Conference on Theory of Cryptography*, TCC'05, page 342–362, Berlin, Heidelberg, 2005. Springer-Verlag.
27. Romain Gay and Bogdan Ursu. On instantiating unleveled fully-homomorphic signatures from falsifiable assumptions. In Qiuliang Tang and Vanessa Teague, editors, *Public-Key Cryptography – PKC 2024*, volume 14601 of *LNCS*. Springer, Cham, 2024.
28. Dan Boneh and David M. Freeman. Homomorphic signatures for polynomial functions. In Kenneth G. Paterson, editor, *Advances in Cryptology – EUROCRYPT 2011*, volume 6632 of *LNCS*. Springer, Berlin, Heidelberg, 2011.
29. Wonhee Cho, Jiseung Kim, and Changmin Lee. (In)Security of Threshold Fully Homomorphic Encryption based on Shamir Secret Sharing. Cryptology ePrint Archive, Paper 2024/1858, 2024.
30. Eyal Boyle, Lisa Kohl, and Paul Scholl. Homomorphic secret sharing from lattices without fhe. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019*, volume 11477 of *LNCS*. Springer, Cham, 2019.
31. Mihir Bellare, Ethan Crites, Chris Komlo, Michael Maller, Stefano Tessaro, and Christina Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, volume 13510 of *LNCS*. Springer, Cham, 2022.
32. Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures: BLS and FROST. Cryptology ePrint Archive, Paper 2022/833, 2022.
33. Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-Optimal Lattice-Based Threshold Signatures, Revisited. In Mikołaj Bojańczyk, Emanuela Merelli, and David P. Woodruff, editors, *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, volume 229 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 8:1–8:20, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
34. Shi Bai, Tancrède Lepoint, Angèle Roux-Langlois, and et al. Improved security proofs in lattice-based cryptography: Using the rényi divergence rather than the statistical distance. *Journal of Cryptology*, 31:610–640, 2018.

35. David Nuñez, Isaac Agudo, and Javier Lopez. Proxy re-encryption: Analysis of constructions and its application to secure access delegation. *Journal of Network and Computer Applications*, 87:193–209, 2017.
36. Michael Egorov and MacLane Wilkison. Nucypher KMS: decentralized key management system. *CoRR*, abs/1707.06140, 2017.
37. Marcel Keller. Mp-spdz: A versatile framework for multi-party computation. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS '20, page 1575–1590, New York, NY, USA, 2020. Association for Computing Machinery.
38. Amr Aly, Kang Cong, Marcel Keller, Emmanuela Orsini, Dragos Rotaru, Onur Scherer, Paul Scholl, Nigel P. Smart, Timothée Tanguy, and Tim Wood. SCALE and MAMBA v1.14: Documentation. `https://homes.esat.kuleuven.be/~nsmart/SCALE/`, 2021. Accessed: 2025-11-09.
39. Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In *Advances in Cryptology – CRYPTO 2024: 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part VII*, page 352–386, Berlin, Heidelberg, 2024. Springer-Verlag.
40. Brent Waters and David J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. Cryptology ePrint Archive, Paper 2025/1547, 2025.
41. Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. Silent threshold encryption with one-shot adaptive security. Cryptology ePrint Archive, Paper 2025/1384, 2025.
42. Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Paper 2025/1419, 2025.
43. Zhelei Zhou, Yun Li, Yuchen Wang, Zhaomin Yang, Bingsheng Zhang, Cheng Hong, Tao Wei, and Wenguang Chen. ZHE: Efficient Zero-Knowledge Proofs for HE Evaluations . In *2025 IEEE S & P*, pages 3087–3105, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
44. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), September 2009.
45. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), September 2009.
46. Miklos Ajtai. Generating Hard Instances of Lattice Problems. Report (tr96-007), Weizmann, 1996.
47. Ruy J. P. Lopes. An ℓ_∞ version of lwe. In *International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT) 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 305–322. Springer, Berlin, Heidelberg, 2011.
48. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. Fully homomorphic encryption from lwe and R-LWE. In *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 505–525. Springer, Berlin, Heidelberg, 2012.
49. Rishab Goyal, Susan Hohenberger, Venkata Koppula, and Brent Waters. A generic approach to constructing and proving verifiable random functions. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography. TCC 2017*, volume 10678 of *LNCS*. Springer, Cham, 2017.
50. Boaz Barak, Swee-Jia Ong, and Salil Vadhan. Derandomization in cryptography. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003*, volume 2729 of *LNCS*. Springer, Berlin, Heidelberg, 2003.
51. Rob Johnson, David Molnar, Dawn Song, and David Wagner. Homomorphic signature schemes. In Bart Preneel, editor, *Topics in Cryptology — CT-RSA 2002*, volume 2271 of *LNCS*. Springer, Berlin, Heidelberg, 2002.

52. Eyal Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under ddh. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016*, volume 9814 of *LNCS*. Springer, Berlin, Heidelberg, 2016.
53. Allison Lewko and Brent Waters. Decentralizing attribute-based encryption. In Kenneth G. Paterson, editor, *Advances in Cryptology – EUROCRYPT 2011*, volume 6632 of *LNCS*. Springer, Berlin, Heidelberg, 2011.
54. Oded Goldreich. On (valiant's) polynomial-size monotone formula for majority. In Oded Goldreich, editor, *Computational Complexity and Property Testing*, volume 12050 of *LNCS*. Springer, Cham, 2020.

A Appendix

A.1 Computational Assumptions

Learning with Errors (LWE) [44] Let $n,m,q \in \mathbb{N}$, and let χ be a distribution over $\mathbb{Z}_q$. Consider $D_0 := (\mathbf{A}, \mathbf{A}^T\mathbf{s}+\mathbf{e})$ and $D_1 := (\mathbf{A},\mathbf{u})$, where $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{n\times m}$, $\mathbf{s} \leftarrow\$ \mathbb{Z}_q^n$, $\mathbf{e} \leftarrow\$ \chi^m$, $\mathbf{u} \leftarrow\$ \mathbb{Z}_q^m$. The $\mathsf{LWE}(n,m,q,\chi)$ problem is to decide between the distributions D_0 and D_1. Furthermore, if the error distribution χ is bounded by a parameter B_{lwe} (i.e., every sample from χ has an absolute value at most B_{lwe}), then solving $\mathsf{LWE}(n,m,q,\chi)$ is at least as hard as approximating worst-case problems such as GapSVP and SIVP [45] in n dimensions within the factor of $\tilde{O}\big(n \cdot q/B_{\mathsf{lwe}}\big)$.

Short Integer Solution (SIS) [46] Let $n,m,q \in \mathbb{N}$ and let $\beta > 0$. Define $\mathsf{SIS}(n,m,q,\beta)$ as the problem of finding a non-zero vector $\mathbf{z} \in \mathbb{Z}^m$ such that $\mathbf{Az} = \mathbf{0} \pmod q$ and $\|\mathbf{z}\| \le \beta$, where $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{n\times m}$.

Decisional Torus Learning With Errors (TLWE) [47] Let $n,m \in \mathbb{N}$, and let χ be a distribution over $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ (the real torus). Consider $D_0 := (\mathbf{a}, \langle \mathbf{a},\mathbf{s}\rangle + e)$ and $D_1 := (\mathbf{a},u)$, where $\mathbf{a} \leftarrow\$ \mathbb{T}^n$, $\mathbf{s} \leftarrow\$ \mathbb{B}^n$ (where $\mathbb{B} = \{0,1\}$), $e \leftarrow\$ \chi$, $u \leftarrow\$ \mathbb{T}$. The $\mathsf{TLWE}(n,\chi)$ problem is to decide between the distributions D_0 and D_1 given m independent samples. The hardness of TLWE is inherited from the standard LWE problem over $\mathbb{Z}_q$ by identifying $\mathbb{T}$ with $\mathbb{Z}_q/q$ in the limit as $q \to \infty$.

Torus Ring Learning With Errors (TRLWE) [48] Let $n \in \mathbb{N}$, and let $R = \mathbb{Z}[X]/(X^n+1)$. Define the torus polynomial ring $\mathbb{T}[X]/(X^n+1)$ where $\mathbb{T} = \mathbb{R}/\mathbb{Z}$. Let χ be a distribution over $\mathbb{T}[X]/(X^n+1)$. Consider $D_0 := (a,\, a \cdot s + e)$ and $D_1 := (a,u)$, where $a \leftarrow\$ \mathbb{T}[X]/(X^n+1)$, $s \leftarrow\$ \mathbb{B}[X]/(X^n+1)$ (where $\mathbb{B} = \{0,1\}$), $e \leftarrow\$ \chi$, $u \leftarrow\$ \mathbb{T}[X]/(X^n+1)$. The $\mathsf{TRLWE}(n,\chi)$ problem is to decide between the distributions D_0 and D_1. This problem generalizes TLWE to polynomial rings and inherits hardness from RLWE by the same torus identification.

A.2 Other Building Blocks and Procedures

Zero-Knowledge Proof with Preprocessing (PZK) [14] Here, a specialized type of zero-knowledge proof protocol $\mathsf{PZK} = (\mathsf{PZK.Pre}, \mathsf{PZK.Prove}, \mathsf{PZK.Verify})$ is

being considered where nearly all communications (except the last) are offline. Pre-processing yields σ_V for the verifier and σ_P for the prover. The online phase requires one round, a relaxed, non-interactive zero-knowledge proof. This pre-processing zero-knowledge proof (PZK) is achievable under weaker assumptions than standard NIZK (e.g., one-way functions).

Definition 6 (Zero-Knowledge Proof with Preprocessing). *Let $L \subseteq \{0,1\}^*$ be a language and $R \subseteq L \times \{0,1\}^*$ a relation. A triple of PPT algorithms $\mathsf{PZK} = (\mathsf{PZK.Pre}, \mathsf{PZK.Prove}, \mathsf{PZK.Verify})$ is said to be a zero-knowledge proof system with preprocessing if, upon executing $(\sigma_V, \sigma_P) \leftarrow \mathsf{PZK.Pre}(1^\lambda)$, it holds that for all of the values $(x,w) \in R$, $\Pr[\mathsf{PZK.Verify}(\sigma_V, x, \pi) = 1 : \pi \leftarrow \mathsf{PZK.Prove}(\sigma_P, x, w)] = 1$, while for every $x \notin L$ we have $\Pr[\exists \pi : \mathsf{PZK.Verify}(\sigma_V, x, \pi) = 1] = \mathsf{negl}(\lambda)$. Moreover, there exists a PPT simulator $\mathcal{S}$ such that for every $(x,w) \in R$ the ensembles $\{\sigma_V, \mathsf{PZK.Prove}(\sigma_P, x, w)\}$ and $\{\mathcal{S}(x)\}$ are computationally indistinguishable.*

We require PZK to fulfill three properties of completeness, soundness, and zero-knowledge, defined respectively as following: PZK satisfies completeness if $\forall (x,w) \in R$, $\Pr[\mathsf{PZK.Verify}(\sigma_V, x, \pi) = 1 : \pi \leftarrow \mathsf{PZK.Prove}(\sigma_P, x, w)] = 1$ where $(\sigma_V, \sigma_P) \leftarrow \mathsf{PZK.Pre}(1^\lambda)$. PZK satisfies soundness if $\forall x \notin L$, $\Pr[\exists \pi : \mathsf{PZK.Verify}(\sigma_V, x, \pi) = 1] = \mathsf{negl}(\lambda)$ where the probability is over PZK.Pre. PZK satisfies zero-knowledge if $\exists$ PPT algorithm $\mathcal{S}$ such that $\forall (x,w) \in R$, the distributions $\{\sigma_V, \mathsf{PZK.Prove}(\sigma_P, x, w)\}$ and $\{\mathcal{S}(x)\}$ are computationally indistinguishable.

Perfectly Binding Non-interactive Commitment (Com) [49] A non-interactive commitment scheme $\mathsf{Com} = (\mathsf{Com})$ relies on a message space $\{0,1\}^*$ and randomness space $\{0,1\}^\lambda$ to provide binding and hiding properties. The work of Blum [15] builds non-interactive commitments from injective one-way functions, and [50] builds them from regular one-way functions and a worst-case assumption relating deterministic time $2^{O(n)}$ and non-deterministic circuit complexity $2^{\Omega(n)}$. Recent work [49] constructs such schemes from LWE.

Definition 7 (Non-Interactive Commitments [15]). *A non-interactive commitment scheme is an algorithm* Com *satisfying the following conditions. For message $x \in \{0,1\}^*$ and randomness $r \in \{0,1\}^\lambda$, let $\mathsf{com} \leftarrow \mathsf{Com}(x;r)$; the scheme must fulfill Perfect Binding and Computational Hiding requirements respectively as specified below:*

- *For every security parameter $\lambda \in \mathbb{N}$, and string $\mathsf{com} \in \{0,1\}^*$, there exists at most a single $x \in \{0,1\}^*$ such that* com *is a commitment to x: Let the binding property be for any $r_0, r_1 \in \{0,1\}^\lambda$:*

$$\mathsf{Com}(x_0;r_0) = \mathsf{Com}(x_1;r_1) \implies x_0 = x_1.$$

- *For every $\lambda \in \mathbb{N}, x_0, x_1 \in \{0,1\}^{\mathsf{poly}(\lambda)}$, the following distributions are computationally indistinguishable: Let $\mathcal{D}_i = \{\mathsf{com}_i : \mathsf{com}_i \xleftarrow{R} \mathsf{Com}(x_i;r)\}_{r \xleftarrow{R} \{0,1\}^\lambda}$ for $i \in \{0,1\}$' then, $\mathcal{D}_0 \approx_c \mathcal{D}_1$.*

The Com is required to satisfy two properties, as perfect binding and computational hiding: Com satisfies perfect binding if $\forall r_0,r_1 \in \{0,1\}^\lambda$, $\mathsf{Com}(x_0;r_0) = \mathsf{Com}(x_1;r_1) \implies x_0 = x_1$. Com satisfies computational hiding if $\forall \lambda \in \mathbb{N}, x_0,x_1 \in \{0,1\}^{\mathsf{poly}(\lambda)}$, the distributions $\{\mathsf{com}_0 : \mathsf{com}_0 \leftarrow \mathsf{Com}(x_0;r)\}_{r \leftarrow \{0,1\}^\lambda}$ and $\{\mathsf{com}_1 : \mathsf{com}_1 \leftarrow \mathsf{Com}(x_1;r)\}_{r \leftarrow \{0,1\}^\lambda}$ are computationally indistinguishable.

Secret Sharing Schemes [4] Let $\mathcal{P} = \{P_1,...,P_n\}$ denote a collection of n parties, and let $\mathbb{A}$ represent a family of admissible access structures. We formalize the notion of a secret sharing scheme as follows.

Definition 8 (Secret Sharing Scheme). *A secret sharing scheme $\Pi = (\mathsf{Dist},\mathsf{Rec})$ over a secret domain $\mathcal{S}$ consists of a pair of probabilistic polynomial-time algorithms $\Pi = (\mathsf{Dist},\mathsf{Rec})$ where:*

- $\mathsf{Dist}(s,\mathcal{A}) \to (\sigma_1,...,\sigma_n)$*: The distribution algorithm takes as input a secret $s \in \mathcal{S}$ and an access structure $\mathcal{A} \in \mathbb{A}$, and outputs a collection of shares $\{\sigma_i\}_{i=1}^n$, one for each party $P_i \in \mathcal{P}$.*
- $\mathsf{Rec}(\{\sigma_i\}_{i \in I}) \to s$*: The reconstruction algorithm takes as input a subset of shares indexed by $I \subseteq [n]$ and outputs a secret value $s \in \mathcal{S}$.*

We require that any secret sharing scheme satisfies two fundamental security properties: correctness and privacy.

Definition 9 (Correctness of Secret Sharing). *A secret sharing scheme $\Pi = (\mathsf{Dist},\mathsf{Rec})$ is said to be correct if for every access structure $\mathcal{A} \in \mathbb{A}$, every authorized subset $T \in \mathcal{A}$, and every secret $s \in \mathcal{S}$, if we sample $(\sigma_1,...,\sigma_n) \leftarrow \mathsf{Dist}(s,\mathcal{A})$, then*

$$\mathsf{Rec}(\{\sigma_i\}_{i \in T}) = s \quad \textit{holds with probability } 1.$$

Definition 10 (Privacy of Secret Sharing). *A secret sharing scheme $\Pi = (\mathsf{Dist},\mathsf{Rec})$ satisfies privacy if for every unauthorized subset $U \notin \mathcal{A}$ and every pair of secrets $s_0,s_1 \in \mathcal{S}$, the following two distributions are computationally indistinguishable:*

$$\{\sigma_{0,i}\}_{i \in U} \overset{c}{\approx} \{\sigma_{1,i}\}_{i \in U},$$

where $(\sigma_{b,1},...,\sigma_{b,n}) \leftarrow \mathsf{Dist}(s_b,\mathcal{A})$ for $b \in \{0,1\}$.

Remark 2. For information-theoretic security, we require statistical indistinguishability (denoted $\overset{s}{\approx}$) or perfect indistinguishability in the privacy definition above.

The structure of access policies naturally gives rise to extremal notions of authorized and unauthorized sets, which we formalize next.

Definition 11 (Extremal Sets in Access Structures [5]). *Let $\mathcal{P} = \{P_1,...,P_n\}$ be a set of parties and $\mathcal{A}$ be a monotone access structure over $\mathcal{P}$. We define:*

1. *A subset $U \subseteq \mathcal{P}$ is called a* maximal unauthorized set *(with respect to $\mathcal{A}$) if $U \notin \mathcal{A}$, yet for every party $P_j \in \mathcal{P} \setminus U$, the augmented set $U \cup \{P_j\}$ satisfies $U \cup \{P_j\} \in \mathcal{A}$.*

2. *A subset $T \subseteq \mathcal{P}$ is called a* minimal authorized set *(with respect to $\mathcal{A}$) if $T \in \mathcal{A}$, and for every proper subset $T' \subsetneq T$, we have $T' \notin \mathcal{A}$.*

Definition 12 (Tree Secret Sharing Scheme [8,18]). *For parameters N parties, threshold t, small positive integer $s \geq 3$ with $\ell = 2s-1$, prime q such that $(N!)^2 \leq q$, and iteration number $L \geq \log_{c_s} N + \log_s N + O(1)$ where $c_s = \frac{\ell}{2^{\ell-1}} \cdot \binom{\ell-1}{s-1}$, the Tree Secret Sharing Scheme $\mathsf{TreeSSS} = (\mathsf{Share}, \mathsf{Combine})$ operates as:*

- $\{\mathsf{sk}_i\}_{i\in[N]} \leftarrow \mathsf{Share}(\mathsf{sk}, \mathcal{A}_{N,t})$*: For secret key $\mathsf{sk} \in \mathbb{Z}_q$ and threshold access structure $\mathcal{A}_{N,t}$, initialize $share_1^{(0)} = \mathsf{sk}$ as the level-0 secret share. For each level $i = 1$ to L, for each level-$(i-1)$ share $share_j^{(i-1)}$ where $j \in [\ell^{i-1}]$, apply s-out-of-ℓ Shamir secret sharing by sampling random values $r_{j,2},\ldots,r_{j,s} \xleftarrow{R} \mathbb{Z}_q$ and computing $(share_{\ell(j-1)+1}^{(i)},\ldots,share_{\ell \cdot j}^{(i)})^T = \mathbf{V}_s \cdot (share_j^{(i-1)}, r_{j,2},\ldots,r_{j,s})^T$ where $\mathbf{V}_s \in \mathbb{Z}_q^{\ell \times s}$ is the Vandermonde matrix with entries $V_{k,\kappa} = k^{\kappa-1}$ for $k \in [\ell], \kappa \in [s]$. Generate partition $\{T_i\}_{i\in[N]}$ of $[\ell^L]$ by randomly assigning each index $j \in [\ell^L]$ to party P_k where $k \xleftarrow{R} [N]$, setting $T_k = T_k \cup \{j\}$. Output $\mathsf{sk}_i = \{share_j^{(L)}\}_{j\in T_i}$ to each party P_i for $i \in [N]$.*
- $\hat{\mathsf{sk}} \leftarrow \mathsf{Combine}(\{(\mathsf{sk}_i, T_i)\}_{i\in S})$*: For subset $S \subseteq [N]$ and secret shares $\{\mathsf{sk}_i\}_{i\in S}$, output $\perp$ if $|S| < t$. Otherwise, for each level $k = L$ down to 1, compute the set $T^{(k)} = \{j_k \mid share_{j_k}^{(k)}$ can be recovered by $\bigcup_{i\in S} T_i\}$ and the reconstruction set $S_{j_k}^{(k)} = \{share_{j'_k}^{(k)} \mid share_{j'_k}^{(k)}$ recovers $share_{j_{k-1}}^{(k-1)}\}$ for each $j_{k-1} \in T^{(k-1)}$. Compute Lagrange coefficients $\lambda_{j_k}^{S_{j_k}^{(k)}} = \prod_{m \in S_{j_k}^{(k)} \setminus \{j_k\}} \frac{0-m}{j_k - m}$ for each level k and index j_k. Output $\hat{\mathsf{sk}} = \sum_{j_L \in T^{(L)}} \left(\prod_{i=1}^{L} \lambda_{j_i}^{S_{j_i}^{(i)}} \right) \cdot share_{j_L}^{(L)}$ where j_i traces the path from level-L share j_L back to the root.*

Fully Homomorphic Encryption [5] We now recall the notion of fully homomorphic encryption schemes, which enable arbitrary computation over encrypted data.

Definition 13 (Fully Homomorphic Encryption). *A fully homomorphic encryption FHE scheme $\mathcal{E}$ is defined by a quadruple of probabilistic polynomial-time algorithms $\mathcal{E} = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Compute}, \mathsf{Dec})$ specified as follows:*

- $\mathsf{Gen}(1^\lambda, 1^\ell) \to (pk, sk)$*: The key generation algorithm receives as input a security parameter λ (in unary) and a circuit depth parameter ℓ (in unary), and produces a public-secret key pair (pk, sk).*
- $\mathsf{Enc}(pk, m) \to c$*: The encryption algorithm receives as input a public key pk and a plaintext bit $m \in \{0,1\}$, and produces a ciphertext c.*
- $\mathsf{Compute}(pk, f, c_1, \ldots, c_t) \to c^*$*: The homomorphic evaluation algorithm receives as input a public key pk, a boolean circuit $f : \{0,1\}^t \to \{0,1\}$ with depth bounded by ℓ, and ciphertexts $c_1,\ldots,c_t$, and produces an evaluated ciphertext c^*.*
- $\mathsf{Dec}(pk, sk, c^*) \to m'$*: The decryption algorithm receives as input both keys pk, sk and a ciphertext c^*, and produces a plaintext bit $m' \in \{0,1,\perp\}$, where $\perp$ represents a decryption failure.*

An FHE scheme must satisfy three essential properties: output compactness, functional correctness, and semantic security.

Definition 14 (Output Compactness). *We say that a fully homomorphic encryption FHE scheme* $\mathcal{E} = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Compute}, \mathsf{Dec})$ *satisfies output compactness if there exists a fixed polynomial* $\mathsf{poly}(\cdot)$ *such that the following holds. For all security parameters* λ*, all depth bounds* ℓ*, all circuits* $f : \{0,1\}^t \to \{0,1\}$ *of depth at most* ℓ*, and all plaintext bits* $m_1,...,m_t \in \{0,1\}$*: if we sample*

$$\begin{cases} (pk,sk) \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell), \\ c_j \leftarrow \mathsf{Enc}(pk, m_j) \text{ for } j \in [t], \\ c^* \leftarrow \mathsf{Compute}(pk, f, c_1, ..., c_t), \end{cases}$$

then the bit-length of the evaluated ciphertext satisfies $|c^*| \leq \mathsf{poly}(\lambda, \ell)$.

Definition 15 (Functional Correctness). *We say that an FHE scheme* $\mathcal{E} = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Compute}, \mathsf{Dec})$ *is functionally correct if the following holds for all security parameters* λ*, all depth bounds* ℓ*, all circuits* $f : \{0,1\}^t \to \{0,1\}$ *of depth at most* ℓ*, and all plaintext bits* $m_1,...,m_t \in \{0,1\}$*. Let*

$$\begin{cases} (pk,sk) \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell), \\ c_j \leftarrow \mathsf{Enc}(pk, m_j) \text{ for } j \in [t], \\ c^* \leftarrow \mathsf{Compute}(pk, f, c_1, ..., c_t). \end{cases}$$

Then we have

$$\Pr[\mathsf{Dec}(pk, sk, c^*) = f(m_1, ..., m_t)] \geq 1 - \mathsf{negl}(\lambda),$$

where the probability is taken over the randomness of all algorithms.

Definition 16 (Semantic Security). *An FHE scheme* $\mathcal{E} = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Compute}, \mathsf{Dec})$ *is semantically secure if for all security parameters* λ*, all depth bounds* ℓ*, and all probabilistic polynomial-time adversaries* $\mathcal{A}$*, the advantage of* $\mathcal{A}$ *in the following security experiment* $\mathsf{Exp}^{IND}_{\mathcal{A},\mathcal{E}}(1^\lambda, 1^\ell)$ *is negligible in* λ*:*

1. *The challenger samples* $(pk, sk) \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell)$*, selects a random challenge bit* $\beta \xleftarrow{\$} \{0,1\}$*, and computes* $c \leftarrow \mathsf{Enc}(pk, \beta)$*. The challenger sends* (pk, c) *to the adversary* $\mathcal{A}$*.*
2. *The adversary* $\mathcal{A}$ *outputs a guess* $\beta' \in \{0,1\}$*.*
3. *The experiment outputs* 1 *if and only if* $\beta = \beta'$*.*

We require that for all PPT adversaries $\mathcal{A}$*,*

$$\left| \Pr\left[\mathsf{Exp}^{IND}_{\mathcal{A},\mathcal{E}}(1^\lambda, 1^\ell) = 1 \right] - \frac{1}{2} \right| \leq \mathsf{negl}(\lambda).$$

Definition 17 (Torus-based Fully Homomorphic Encryption [11]). *For parameters* $n=\mathsf{poly}(\lambda)$*, ring dimension* $N=2^p$*, decomposition base* B_g*, gadget length* ℓ*, and Gaussian parameters* $\alpha,\bar{\alpha},\gamma\in\mathbb{R}_{>0}$*, Torus-based Fully Homomorphic Encryption* $\mathsf{Torus\text{-}FHE}=(\mathsf{Setup},\mathsf{Encrypt},\mathsf{Eval},\mathsf{Decrypt})$ *operates as:*

- $(\mathsf{pk},\mathsf{sk})\leftarrow\mathsf{Setup}(1^\lambda,1^d)$*: Outputs public key* $\mathsf{pk}=(\mathsf{BK},\mathsf{KS})$ *and secret key* $\mathsf{sk}=K\in\mathbb{B}^n$ *where binary secret keys* $K\in\mathbb{B}^n$*,* $\bar{K}\in\mathbb{B}^{\bar{n}}$ *with* TRLWE *interpretation* $\bar{K}\in\mathbb{B}_N[X]^{\bar{k}}$*, bootstrapping key* $\mathsf{BK}_{K\to\bar{K},\bar{\alpha}}=(\mathsf{BK}_i\in\mathsf{TRGSW}_{\bar{K},\bar{\alpha}}(K_i))_{i\in[1,n]}$*, and key-switching key* $\mathsf{KS}_{\bar{K}\to K,\gamma}=(\mathsf{KS}_i^{(j)}\in\mathsf{TLWE}_{K,\gamma}\ (\bar{K}_i/2^j))_{i\in[1,\bar{n}],j\in[1,t]}$ *get generated.*
- $ct\leftarrow\mathsf{Encrypt}(\mathsf{pk},\mu)$*: For message* $\mu\in\mathbb{T}$ *where* $\mathbb{T}=\mathbb{R}/\mathbb{Z}$ *is the real torus, outputs* TLWE *ciphertext* $ct=(\mathbf{a},b)\in\mathbb{T}^{n+1}$ *where* $\mathbf{a}\xleftarrow{R}\mathbb{T}^n$*,* $e\leftarrow\mathcal{D}_{\mathbb{T},\alpha}$*, and* $b=\mathbf{a}\cdot K+e+\mu\pmod 1$.
- $\hat{ct}\leftarrow\mathsf{Eval}(\mathsf{pk},C,ct_1,...,ct_k)$*: For circuit* $C:\{0,1\}^k\to\{0,1\}$ *of depth at most* d *and* TLWE *ciphertexts* $ct_1,...,ct_k$*, performs homomorphic evaluation via linear combinations, in which for* $e_i\in\mathbb{Z}$*, computes* $\sum e_i\cdot ct_i$*, external product* $C\boxdot ct=\mathsf{Dec}_{H,\beta,\epsilon}(ct)\cdot C$ *for* TRGSW C *and* TLWE ct*, CMux gate* $CMux(C,d_1,d_0)=C\boxdot(d_1-d_0)+d_0$*, and gate bootstrapping, in which after each gate, applies* $\mathsf{Bootstrap}_{\mu_1}(ct,\mathsf{BK})=\mathsf{KeySwitch}(\mathsf{BootstrapTLWE}(ct,\mathsf{BK}),\mathsf{KS})$ *where in fact the* BootstrapTLWE *uses blind rotation with test vector* $v=(1+X+\cdots+X^{\bar{N}-1})\cdot X^{\bar{N}/2}\cdot\mu$ *to output* TLWE *sample under* $\bar{K}$*, then* KeySwitch *converts back to key* K*. Then, outputs evaluated ciphertext* $\hat{ct}\in\mathbb{T}^{n+1}$.
- $\hat{\mu}\leftarrow\mathsf{Decrypt}(\mathsf{sk},\hat{ct})$*: For* $\hat{ct}=(\mathbf{a},b)$*, computes phase* $\varphi_K(\hat{ct})=b-\mathbf{a}\cdot K\pmod 1$ *and outputs* $\hat{\mu}=\lfloor 2\cdot\varphi_K(\hat{ct})\rceil\in\{0,1,\perp\}$.

According to [11], the scheme satisfies *correctness* with probability at least $1-2^{-1}$, where for any $S\in\mathcal{A}_{N,t}$ with $|S|\geq t$, Combine outputs $\hat{\mathsf{sk}}=\mathsf{sk}$, and for any $S\notin\mathcal{A}_{N,t}$ with $|S|<t$, Combine outputs $\perp$. The scheme satisfies *privacy*, where for any $S\notin\mathcal{A}_{N,t}$ and $\mathsf{sk}_0,\mathsf{sk}_1\in\mathbb{Z}_q$, the distributions $\{\mathsf{sk}_{0,i}\}_{i\in S}$ and $\{\mathsf{sk}_{1,i}\}_{i\in S}$ are statistically indistinguishable, where $\{\mathsf{sk}_{b,i}\}_{i\in[N]}\leftarrow\mathsf{Share}(\mathsf{sk}_b,\mathcal{A}_{N,t})$ for $b\in\{0,1\}$. The total number of secret shares is $\ell^L=O(N^{\log_{c_s}\ell+\log_s\ell})$.

Definition 18 (TFHE). *Let* $(\mathbb{S},N)$ *denote an efficient access structure over* N *parties and* $(\mathbb{C},d)$ *denote a circuit class of depth bounded by* d*. A threshold fully homomorphic encryption scheme* TFHE *consists of a quintuple of probabilistic polynomial-time algorithms* $\mathsf{TFHE}=(\mathsf{Setup},\mathsf{Encrypt},\mathsf{Eval},\mathsf{PartDec},\mathsf{FinDec})$ *specified as follows:*

- $(pk,sk_1,...,sk_N)\leftarrow\mathsf{Setup}(1^\lambda,1^d,(\mathbb{S},N))$*: The setup algorithm receives as input a security parameter* λ *(in unary), a depth bound* d *(in unary), and an efficient access structure* $(\mathbb{S},N)$*, and produces a public key* pk *and secret key shares* $sk_1,...,sk_N$ *where* sk_i *is assigned to party* P_i *for* $i\in[N]$.
- $c\leftarrow\mathsf{Encrypt}(pk,\mu)$*: The encryption algorithm receives as input a public key* pk *and a plaintext bit* $\mu\in\{0,1\}$*, and produces a ciphertext* c.
- $\hat{c}\leftarrow\mathsf{Eval}(pk,C,c_1,...,c_k)$*: The homomorphic evaluation algorithm receives as input a public key* pk*, a circuit* $C:\{0,1\}^k\to\{0,1\}$ *with depth at most* d*, and ciphertexts* $c_1,...,c_k$*, and produces an evaluated ciphertext* $\hat{c}$.

- $p_i \leftarrow \mathsf{PartDec}(pk, \hat{c}, sk_i)$: *The partial decryption algorithm receives as input a public key pk, a ciphertext $\hat{c}$, and a secret key share sk_i, and produces a partial decryption p_i.*
- $\mu' \leftarrow \mathsf{FinDec}(pk, \{p_i\}_{i \in S})$: *The final decryption algorithm receives as input a public key pk and a collection of partial decryptions $\{p_i\}_{i \in S}$ from an authorized set $S \in \mathbb{S}$, and produces a plaintext bit $\mu' \in \{0,1,\perp\}$ where $\perp$ represents a decryption failure.*

According to [5], a threshold fully homomorphic encryption TFHE is required to satisfy the following properties.

Definition 19 (Ciphertext Compactness). *We say that a TFHE scheme $\Gamma = (\mathsf{Init},\mathsf{Enc},\mathsf{Eval},\mathsf{PartDec},\mathsf{FinDec})$ satisfies ciphertext compactness if there exist fixed polynomials $\mathsf{poly}_1(\cdot)$ and $\mathsf{poly}_2(\cdot)$ such that the following holds. For all security parameters λ, all depth bounds ℓ, all boolean circuits $f : \{0,1\}^t \to \{0,1\}$ with depth at most ℓ, and all plaintexts $m \in \{0,1\}$, if we generate*

$$\left[\begin{array}{c} (pk, sk_1, \ldots, sk_n) \leftarrow \mathsf{Init}(1^\lambda, 1^\ell, \mathcal{A}) \\ c_j \leftarrow \mathsf{Enc}(pk, m_j) \text{ for } j \in [t] \\ c^* \leftarrow \mathsf{Eval}(pk, f, c_1, \ldots, c_t) \\ \rho_i \leftarrow \mathsf{PartDec}(pk, c^*, sk_i) \text{ for any } i \in [n] \end{array}\right]$$

then we have $|c^*| \leq \mathsf{poly}_1(\lambda,\ell)$ and $|\rho_i| \leq \mathsf{poly}_2(\lambda,\ell,n)$ for all $i \in [n]$.

Definition 20 (Evaluation Correctness). *We say that a TFHE scheme $\Gamma = (\mathsf{Init},\mathsf{Enc},\mathsf{Eval},\mathsf{PartDec},\mathsf{FinDec})$ achieves evaluation correctness if the following condition holds for all security parameters λ, all depth bounds ℓ, all access structures $\mathcal{A}$, all authorized subsets $T \in \mathcal{A}$, all boolean circuits $f : \{0,1\}^t \to \{0,1\}$ with depth at most ℓ, and all plaintexts $m_1, \ldots, m_t \in \{0,1\}$. Suppose we sample*

$$\left[\begin{array}{c} (pk, sk_1, \ldots, sk_n) \leftarrow \mathsf{Init}(1^\lambda, 1^\ell, \mathcal{A}) \\ c_j \leftarrow \mathsf{Enc}(pk, m_j) \text{ for } j \in [t] \\ c^* \leftarrow \mathsf{Eval}(pk, f, c_1, \ldots, c_t) \end{array}\right]$$

Then we require

$$\Pr[\mathsf{FinDec}(pk, \{\mathsf{PartDec}(pk, c^*, sk_i)\}_{i \in T}) = f(m_1, \ldots, m_t)] \geq 1 - \mathsf{negl}(\lambda),$$

where the probability is taken over the randomness of all algorithms.

Definition 21 (Semantic Security). *We say that a threshold fully homomorphic encryption TFHE scheme $\Gamma = (\mathsf{Init},\mathsf{Enc},\mathsf{Eval},\mathsf{PartDec},\mathsf{FinDec})$ satisfies semantic security if for every security parameter λ, every depth bound ℓ, and every probabilistic polynomial-time adversary $\mathcal{A}$, the adversary's advantage in the following security experiment $\mathsf{Exp}^b_{\mathcal{A},\mathsf{TFHE}}(1^\lambda, 1^d)$, s.t. $b \in \{\mathsf{real},\mathsf{ideal}\}$ is negligible in λ:*

$$\Pr\left[\begin{array}{l}\mathcal{A}\in\mathbb{S}\leftarrow\mathcal{A}(1^\lambda,1^\ell)\\(pk,sk_1,...,sk_n)\leftarrow\mathsf{Init}(1^\lambda,1^\ell,\mathcal{A})\\U\subseteq\{P_1,...,P_n\}\leftarrow\mathcal{A}(pk)\ \textit{where}\ U\notin\mathcal{A}\\\beta\xleftarrow{\$}\{0,1\};c\leftarrow\mathsf{Enc}(pk,\beta)\\\beta'\leftarrow\mathcal{A}(\{sk_i\}_{i\in U},c)\\\textbf{\textit{return}}\ (\beta=\beta')\end{array}\right]-\frac{1}{2}\leq\mathsf{negl}(\lambda).$$

Homomorphic Signature [7] A homomorphic signature scheme [51] enables party P_1 to sign message x with secret key $\mathsf{sk}_{\mathrm{hs}}$ and provide this signature to untrusted party P_2. P_2 can compute $y=f(x)$ and derive signature $\sigma_{f,y}$ certifying y as valid output of computation f over x. Signature $\sigma_{f,y}$ must be short and size-independent of x. Anyone can verify tuple $(f,y,\sigma_{f,y})$ using P_1's public key $\mathsf{vk}_{\mathrm{hs}}$ to confirm $y=f(x)$ without requiring x.

Definition 22 (Homomorphic Signature Scheme [22, 28]). *Let $\mathbb{F}$ be a family of admissible functions. A homomorphic signature scheme Σ for the function family $\mathbb{F}$ consists of a quadruple of probabilistic polynomial-time algorithms $\Sigma=$ (KeyGen,Sign,Verify,Eval) specified as follows:*

- $\mathsf{KeyGen}(1^\lambda)\rightarrow(sk_{sig},pk_{sig})$*: The key generation algorithm receives as input a security parameter λ (in unary), and produces a signing key sk_{sig} and a verification key pk_{sig}. The verification key implicitly defines the family $\mathbb{F}$ of admissible functions over which homomorphic operations can be performed.*
- $\mathsf{Sign}(sk_{sig},\tau,m)\rightarrow\sigma$*: The signing algorithm receives as input a signing key sk_{sig}, a label τ (used to bind the signature to a specific dataset or context), and a message m, and produces a signature σ.*
- $\mathsf{Verify}(pk_{sig},\tau,m,\sigma,f)\rightarrow b$*: The verification algorithm receives as input a verification key pk_{sig}, a label τ, a message m, a signature σ, and a function $f\in\mathbb{F}$, and outputs a decision bit $b\in\{0,1\}$ indicating whether the signature is accepted (1) or rejected (0).*
- $\mathsf{Eval}(pk_{sig},\tau,f,\sigma)\rightarrow\sigma_{f,y}$*: The homomorphic evaluation algorithm receives as input a verification key pk_{sig}, a label τ, a function $f\in\mathbb{F}$, and a signature σ on some message m, and produces an evaluated signature $\sigma_{f,y}$ where $y=f(m)$.*

The homomorphic signature scheme enables the following functionality: A trusted party holding sk_{sig} can sign a message m to produce σ. An untrusted third party, given only the public verification key pk_{sig} and the signature σ, can homomorphically compute $\sigma_{f,y}$ for any function $f\in\mathbb{F}$, which serves as a valid signature certifying that $y=f(m)$, without requiring knowledge of m or access to sk_{sig}. Any verifier can then check the validity of $(f,y,\sigma_{f,y})$ using only pk_{sig} and the label τ, becoming convinced that y is indeed the correct output of f applied to the originally signed message.

Definition 23 (Correctness). *An HS scheme* $\Sigma=(\mathsf{KeyGen},\mathsf{Sign},\mathsf{Verify},\mathsf{Eval})$ *satisfies correctness if the following two conditions hold for all security parameters* λ*. Let* $(sk_{sig},pk_{sig})\leftarrow\mathsf{KeyGen}(1^\lambda)$*.*
(Base Correctness): *For every label* τ *and every message* m*, we have*

$$\Pr\left[\begin{array}{l}\sigma\leftarrow\mathsf{Sign}(sk_{sig},\tau,m)\\ \textbf{\textit{return}}\ \mathsf{Verify}(pk_{sig},\tau,m,\sigma,\mathsf{id})=1\end{array}\right]\geq 1-\mathsf{negl}(\lambda),$$

where id *denotes the identity function.*
(Evaluation Correctness): *For every label* τ*, every message* m*, and every function* $f\in\mathbb{F}$*, we have*

$$\Pr\left[\begin{array}{l}\sigma\leftarrow\mathsf{Sign}(sk_{sig},\tau,m)\\ \sigma_{f,y}\leftarrow\mathsf{Eval}(pk_{sig},\tau,f,\sigma)\\ \textbf{\textit{return}}\ \mathsf{Verify}(pk_{sig},\tau,f(m),\sigma_{f,y},f)=1\end{array}\right]\geq 1-\mathsf{negl}(\lambda).$$

Definition 24 (Unforgeability). *An HS scheme* $\Sigma=(\mathsf{KeyGen},\mathsf{Sign},\mathsf{Verify},\mathsf{Eval})$ *is unforgeable if for every probabilistic polynomial-time adversary* $\mathcal{A}$*, the probability that* $\mathcal{A}$ *succeeds in the following experiment is negligible in* λ*:*

$$\Pr\left[\begin{array}{l}(sk_{sig},pk_{sig})\leftarrow\mathsf{KeyGen}(1^\lambda)\\ (f^*,y^*,\sigma^*)\leftarrow\mathcal{A}^{\mathsf{Sign}(sk_{sig},\cdot,\cdot)}(pk_{sig})\\ b\leftarrow\mathsf{Verify}(pk_{sig},\tau^*,y^*,\sigma^*,f^*)=1\\ \textbf{\textit{return}}\ (b=1)\wedge(f^*(m)\neq y^*)\end{array}\right]\leq\mathsf{negl}(\lambda),$$

where $\mathcal{A}$ *has oracle access to the signing algorithm and* m *denotes the message signed under label* τ^* *during the query phase. The adversary wins if it outputs a valid signature for a function-output pair* (f^*,y^*) *where* $y^*\neq f^*(m)$*.*

Definition 25 (Context-Hiding). *An HS scheme* $\Sigma=(\mathsf{KeyGen},\mathsf{Sign},\mathsf{Verify},\mathsf{Eval})$ *satisfies the context-hiding property if there exists a probabilistic polynomial-time simulator algorithm* $\mathcal{S}$ *such that for all security parameters* λ*, all labels* τ*, all messages* m*, and all functions* $f\in\mathbb{F}$*, the following two distributions are computationally indistinguishable:*

$$\left\{\begin{array}{l}(sk_{sig},pk_{sig})\leftarrow\mathsf{KeyGen}(1^\lambda)\\ \sigma_m\leftarrow\mathsf{Sign}(sk_{sig},\tau,m)\\ \sigma_{f,y}\leftarrow\mathsf{Eval}(pk_{sig},\tau,f,\sigma_m)\\ \textbf{\textit{return}}\ \sigma_{f,y}\end{array}\right\}\overset{c}{\approx}\left\{\begin{array}{l}(sk_{sig},pk_{sig})\leftarrow\mathsf{KeyGen}(1^\lambda)\\ \widetilde{\sigma}\leftarrow\mathcal{S}(sk_{sig},\tau,f,f(m))\\ \textbf{\textit{return}}\ \widetilde{\sigma}\end{array}\right\}.$$

Intuitively, context-hiding ensures that an evaluated signature reveals nothing beyond the function f *and its output* $f(m)$*, hiding information about the original message* m*.*

Definition 26 (Compactness). *An HS scheme* $\Sigma=(\mathsf{KeyGen},\mathsf{Sign},\mathsf{Verify},\mathsf{Eval})$ *satisfies compactness if there exists a fixed polynomial* $\mathsf{poly}(\cdot,\cdot)$ *such that for all security parameters* λ*, all functions* $f\in\mathbb{F}$*, all labels* τ*, and all messages* m*, when we compute*

$$\begin{bmatrix}(sk_{sig},pk_{sig})\leftarrow\mathsf{KeyGen}(1^\lambda)\\ \sigma_m\leftarrow\mathsf{Sign}(sk_{sig},\tau,m)\\ \sigma_{f,y}\leftarrow\mathsf{Eval}(pk_{sig},\tau,f,\sigma_m)\end{bmatrix}$$

the size of the evaluated signature satisfies $|\sigma_{f,y}|\leq\mathsf{poly}(\lambda,\mathsf{depth}(f))$*, where* $\mathsf{depth}(f)$ *denotes the circuit depth of function* f*. In particular, the signature size grows at most linearly with the circuit depth and is independent of the input size or circuit size.*

Homomorphic Secret Sharing (HSS) [7, 52] Homomorphic Secret Sharing enables a party to share a secret x among multiple parties, where each party can perform computations on their share without interaction. These computations yield shares of the result, which can be combined to obtain the actual result of the computation on the original secret.

HSS [52] enables a party to share a secret x among multiple parties, where each party can perform computations on their share without interaction. These computations yield shares of the result, which can be combined to obtain the actual result of the computation on the original secret.

Definition 27 (HSS [23]). *A homomorphic secret sharing scheme for function family* $\mathcal{F}$ *consists of* $\mathsf{HSS}=(\mathsf{Share},\mathsf{Eval}_1,\mathsf{Eval}_2,\mathsf{Combine})$.

- $(\mathsf{share}_1,\mathsf{share}_2)\leftarrow\mathsf{Share}(1^\lambda,x)$ *produces two secret shares* share_1 *and* share_2 *of a secret* $x\in\{0,1\}^*$*, given the security parameter* 1^λ*.*
- $y_b\leftarrow\mathsf{Eval}_b(\mathsf{share}_b,C)$*: For* $b\in\{1,2\}$*, party* b *produces an evaluation share* y_b *of the computation* $C(x)$ *using only their share* share_b *and circuit* $C\in\mathcal{F}$*.*
- $y\leftarrow\mathsf{Combine}(y_1,y_2)$*. The combination algorithm takes evaluation shares* y_1 *and* y_2 *and outputs the final result* $y\in\{0,1\}^*$ *such that* $y=C(x)$*.*

Linear Secret Sharing Scheme ({0,1}-LSSS) [5] We define the special case of a linear secret sharing scheme where the reconstruction coefficients are always binary [5] and the combining algorithm consists of linear operations.

Definition 28. *Let* $P=\{P_1,...,P_N\}$ *be a set of parties. The class of access structure* $\{0,1\}$-LSSS_N *is the collection of access structures* $\mathbb{A}\in\mathsf{LSSS}_N$ *for which there exists an efficient linear secret sharing scheme* $\mathsf{SS}=(\mathsf{SS.Share},\mathsf{SS.Combine})$ *over the secret space* $K=\mathbb{Z}_p$ *satisfying the following property: Let* k *be a shared secret and* $\{w_j\}_{j\in T_i}$ *be the share of party* P_i *for* $i\in[N]$*. Then, for every set* $S\in\mathbb{A}$*, there exists a subset* $T\subseteq\bigcup_{i\in S}T_i$ *such that* $k=\sum_{j\in T}w_j$*.*

For any special linear secret sharing scheme SS, and any minimal valid share set $T\subseteq[l]$, we have that $\sum_{j\in T}w_j=k$.

A.3 Batched and Packed Secret Sharing Framework

We require this technique because TFHE necessitates sharing n independent key bits (n is the lattice dimension), and applying TreeSSS naively would generate $O(n\cdot(2t-1)^L)$ total shares with $O(n)$ separate commitment phases. Batching the n key bits reduces commitment overhead by factor $\Theta(n)$, while packing ℓ_p bits per polynomial reduces share count by factor ℓ_p, yielding asymptotic improvement from $O(n\cdot N^{3+2.3/\log s})$ to $O(N^{3+2.3/\log s})$ shares for key distribution—important for practical deployment at scale.

For parameters n parties, threshold t, batching parameter $k \geq 1$ (number of independent sharings), packing parameter $\ell \geq 1$ (number of secrets per polynomial) such that $n \geq 2t+\ell$, prime q, security parameter λ, the Batched and Packed VSS Framework (k,ℓ)-BΠ = (Share,Verify,Reconstruct) operates as:

- $(\{f_i^{(j)}\}_{i\in[n],j\in[k]},\pi_{\mathsf{share}}) \leftarrow \mathsf{Share}(n,t,\{f_\alpha^{(j)}\}_{\alpha\in\{-(\ell-1),\ldots,0\},j\in[k]})$: For $k\times\ell$ secrets $\{f_\alpha^{(j)}\}_{\alpha\in\{-(\ell-1),\ldots,0\},j\in[k]} \subset \mathbb{Z}_q$, sample k random polynomials $f^{(1)}(x),\ldots,f^{(k)}(x) \in \mathbb{Z}_q[x]_{t+\ell-1}$ of degree $t+\ell-1$ subject to $f^{(j)}(\alpha)=f_\alpha^{(j)}$ for $\alpha\in\{-(\ell-1),\ldots,0\}$ and $j\in[k]$, and one masking polynomial $r(x)\in\mathbb{Z}_q[x]_{t+\ell-1}$ of degree $t+\ell-1$. For $i=1,\ldots,n$: compute shares $f_i^{(j)}=f^{(j)}(i)$ for $j\in[k]$ and $r_i=r(i)$, sample auxiliary randomness $\gamma_i \xleftarrow{R} \mathbb{Z}_q$, and compute commitment $c_i=\mathcal{C}((f_i^{(1)},\ldots,f_i^{(k)},r_i),\gamma_i)$ using vector commitment scheme $\mathcal{C}$. Compute batched challenge $(d_1,\ldots,d_k)\leftarrow \mathcal{H}(c_1,\ldots,c_n)$ from random oracle $\mathcal{H}:\{0,1\}^*\to\mathbb{Z}_q^k$. Set aggregated proof polynomial $z(x)=r(x)+\sum_{j=1}^k d_j\cdot f^{(j)}(x)$ and proof $\pi_{\mathsf{share}}=(c_1,\ldots,c_n,z(x))$. Send shares $(\{f_i^{(j)}\}_{j\in[k]},\gamma_i)$ privately to party P_i and broadcast π_{share}.
- $b \leftarrow \mathsf{Verify}(n,t,\pi_{\mathsf{share}},\{f_i^{(j)}\}_{j\in[k]},\gamma_i)$: Given proof $\pi_{\mathsf{share}}=(c_1,\ldots,c_n,z(x))$ and individual shares $(\{f_i^{(j)}\}_{j\in[k]},\gamma_i)$, party P_i checks whether $\deg(z(x)) \leq t+\ell-1$. If so, recompute $(d_1,\ldots,d_k)\leftarrow\mathcal{H}(c_1,\ldots,c_n)$ and verify $c_i \stackrel{?}{=} \mathcal{C}((f_i^{(1)},\ldots,f_i^{(k)},z(i)-\sum_{j=1}^k d_j f_i^{(j)}),\gamma_i)$. If verification fails, broadcast complaint against dealer. Output false if more than t parties complain; otherwise, if a party P_i complains, dealer broadcasts $(f_i^{(1)},\ldots,f_i^{(k)},\gamma_i)$ for public verification. Output true if all non-complained shares verify correctly, false otherwise.
- $\{f_\alpha^{(j)}\}_{\alpha\in\{-(\ell-1),\ldots,0\},j\in[k]} \leftarrow \mathsf{Reconstruct}(\pi_{\mathsf{share}},\{(\{f_i^{(j)}\}_{j\in[k]},\gamma_i)\}_{i\in Q})$: For qualified subset $Q\subseteq[n]$ with $|Q|\geq t+\ell$, each party $P_i\in Q$ broadcasts share values $(\{f_i^{(j)}\}_{j\in[k]},\gamma_i)$. A party P_i is confirmed if $c_i=\mathcal{C}((f_i^{(1)},\ldots,f_i^{(k)},z(i)-\sum_{j=1}^k d_j f_i^{(j)}),\gamma_i)$ where $(d_1,\ldots,d_k)\leftarrow\mathcal{H}(c_1,\ldots,c_n)$. For each $j\in[k]$, consider $\{f_i^{(j)}\}_{i\in Q'}$ of any $t+\ell$ confirmed parties Q' and interpolate polynomial $f^{(j)}(x)$ of degree $t+\ell-1$ passing through these points using Lagrange coefficients $\lambda_{\alpha,i}^{Q'}=\prod_{m\in Q'\setminus\{i\}}\frac{\alpha-m}{i-m}$ for reconstruction points $\alpha\in\{-(\ell-1),\ldots,0\}$. Output $f_\alpha^{(j)}=f^{(j)}(\alpha)=\sum_{i\in Q'}\lambda_{\alpha,i}^{Q'}\cdot f_i^{(j)}$ for all $\alpha\in\{-(\ell-1),\ldots,0\}$ and $j\in[k]$, or false if fewer than $t+\ell$ valid shares exist.

In [20], it has been proven that the framework satisfies *completeness* where honest dealer and parties enable successful reconstruction, *verifiability* ensuring unique

reconstruction of $k \times \ell$ secrets from any qualified subset under binding of $\mathcal{C}$ and collision-resistance of $\mathcal{H}$ in the random oracle model, and *privacy* where any subset of at most t parties learns no information about the secrets under hiding of $\mathcal{C}$. The communication complexity is $O(n+k)$ commitments and $O(t+\ell)$ polynomial coefficients, achieving amortized cost of $O((n+k)/(k \cdot \ell))$ per secret. When $\ell = 1$ (non-packed), optimal resilience $t < n/2$ is maintained; when $\ell > 1$ (packed), resilience reduces to $t < (n-\ell)/2$ but efficiency improves by factor ℓ.

A.4 Simulation Security of TFHE‴

Theorem 14. *Let* $\mathsf{TFHE}''' = (\mathsf{Setup}, \mathsf{Encrypt}, \mathsf{Eval}, \mathsf{PartDec}, \mathsf{FinDec})$ *as defined in Definition 4. Under the privacy of* $\mathsf{TreeSSS}'$ *and noise flooding with parameter* B_{sm}*, there exists a PPT simulator* $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ *such that for all PPT adversaries* $\mathcal{A}$*, all security parameters* λ*, all depth bounds* d*, and all threshold access structures* $\mathcal{A}_{N,t}$*:*

$$\left\{\mathsf{Expt}_{\mathcal{A}}^{real}(1^\lambda, 1^d)\right\} \overset{c}{\approx} \left\{\mathsf{Expt}_{\mathcal{A}}^{ideal}(1^\lambda, 1^d)\right\},$$

where the experiments are defined as follows:

$\mathsf{Expt}_{\mathcal{A}}^{real}(1^\lambda, 1^d)$:

$(\mathsf{pk}, \{\mathsf{sk}_i\}_i) \leftarrow \mathsf{Setup}(1^\lambda, 1^d, \mathcal{A}_{N,t})$;
$U^* \leftarrow \mathcal{A}(\mathsf{pk}) : U^* \notin \mathcal{A}_{N,t}$;
$(\mu_1, \ldots, \mu_\kappa, C) \leftarrow \mathcal{A}(\{\mathsf{sk}_i\}_{i \in U^*})$;
$\forall j : ct_j \leftarrow \mathsf{Encrypt}(\mathsf{pk}, \mu_j)$;
$\hat{ct} \leftarrow \mathsf{Eval}(\mathsf{pk}, C, ct_1, \ldots, ct_\kappa)$;
For queries (S, C'):
  send $\{\mathsf{PartDec}(\mathsf{sk}_i, \hat{ct})\}_{i \in S}$ *to* $\mathcal{A}$;
return $\mathcal{A}'s$ *output*;

$\mathsf{Expt}_{\mathcal{A}}^{ideal}(1^\lambda, 1^d)$:

$(\mathsf{pk}, \{\mathsf{sk}_i\}_i, \mathsf{st}) \leftarrow \mathcal{S}_1(1^\lambda, 1^d, \mathcal{A}_{N,t})$;
$U^* \leftarrow \mathcal{A}(\mathsf{pk}) : U^* \notin \mathcal{A}_{N,t}$;
$(\mu_1, \ldots, \mu_\kappa, C) \leftarrow \mathcal{A}(\{\mathsf{sk}_i\}_{i \in U^*})$;
$\forall j : ct_j \leftarrow \mathsf{Encrypt}(\mathsf{pk}, \mu_j)$;
$\hat{ct} \leftarrow \mathsf{Eval}(\mathsf{pk}, C, ct_1, \ldots, ct_\kappa)$;
For queries (S, C'):
  $\{p_i\}_{i \in S} \leftarrow \mathcal{S}_2(\mathsf{st}, C', C'(\mu_1, \ldots, \mu_\kappa), S)$;
  send $\{p_i\}_{i \in S}$ *to* $\mathcal{A}$;
return $\mathcal{A}'s$ *output.*

Proof. Construct simulator $\mathcal{S}=(\mathcal{S}_1,\mathcal{S}_2)$ as follows. $\mathcal{S}_1(1^\lambda,1^d,\mathcal{A}_{N,t})$ executes Setup honestly to generate $(\mathsf{pk},\{\mathsf{sk}_i\}_i)$ and stores $\mathsf{st}=(\mathsf{pk},\{\mathsf{sk}_i\}_i)$. For query (S,C'), $\mathcal{S}_2(\mathsf{st},C',y,S)$ where $y=C'(\mu_1,...,\mu_\kappa)$ simulates partial decryptions as follows: sample target phase $\varphi_{\text{target}} \leftarrow y+e_{\text{small}}$ where $e_{\text{small}} \leftarrow \mathcal{D}_{\mathbb{T},\bar{\sigma}}$, compute Lagrange coefficients $\{c_i^S\}_{i\in S}$ from $\mathsf{TreeSSS}'.\mathsf{Combine}'$, sample $\{e_i\}_{i\in S\setminus\{i^*\}}$ where $e_i \leftarrow \mathcal{U}([-B_{\text{sm}},B_{\text{sm}}])$, set $p_i=\varphi_i'+((2s-1)!)^L e_i$ for arbitrary φ_i' for $i\neq i^*$, and choose p_{i^*} such that $\varphi_{\text{target}}=b-\sum_{i\in S}c_i^S p_i$ where b is from $\hat{\text{ct}}$. Output $\{p_i\}_{i\in S}$. By noise flooding, the distribution of $\{p_i\}_{i\in S}$ is statistically close to honest partial decryptions: the noise terms $\{((2s-1)!)^L e_i\}_{i\in S}$ dominate any information about the actual partial phases $\{\varphi_i\}_{i\in S}$. Specifically, the statistical distance is bounded by $\frac{((2s-1)!)^{2L}(2s-1)^L\bar{\sigma}}{B_{\text{sm}}} \leq 2^{-\lambda}$ by choice of B_{sm}. Combined with Theorem 2 ensuring shares $\{\mathsf{sk}_i\}_{i\in U^*}$ reveal no information about K, the simulated experiment is computationally indistinguishable from the real experiment. □

A.5 Analysis of HS′ Properties

Theorem 15. *Let* $\mathsf{HS}'=(\mathsf{KeyGen},\mathsf{Sign},\mathsf{Verify},\mathsf{Eval})$ *as defined in Definition 5. For all security parameters* λ*, all data-size bounds* N*, all labels* τ*, all messages* $m\in\{0,1\}$*, all indices* $i\in[N]$*, and all circuits* $f:\{0,1\}^N\to\{0,1\}$ *composed of NAND gates:*

$$\Pr\left[\begin{array}{l}(\mathsf{vk},\mathsf{sk})\leftarrow\mathsf{KeyGen}(1^\lambda,1^N);\\ \sigma\leftarrow\mathsf{Sign}(\mathsf{sk},m,i);\\ \mathit{Verify}(\mathsf{vk},\mathsf{id},m,\sigma)=1\end{array}\right]\geq 1-\mathsf{negl}(\lambda),$$

$$\Pr\left[\begin{array}{l}(\mathsf{vk},\mathsf{sk})\leftarrow\mathsf{KeyGen}(1^\lambda,1^N);\\ \forall j\in[N]:\sigma_j\leftarrow\mathsf{Sign}(\mathsf{sk},m_j,j);\\ \sigma_f\leftarrow\mathsf{Eval}(\mathsf{vk},f,(m_1,\sigma_1),...,(m_N,\sigma_N));\\ \mathit{Verify}(\mathsf{vk},f,f(m_1,...,m_N),\sigma_f)=1\end{array}\right]\geq 1-\mathsf{negl}(\lambda).$$

Proof. For signing correctness, $\mathsf{Sign}(\mathsf{sk},m,i)$ generates $(\mathsf{crs}_{\text{sim}},\mathsf{td}_{\text{sim}})=\mathsf{GenCRS}(0)$ using $\mathsf{NIZK.SimSetup}$ with PRF-generated randomness $\mathsf{PRF}(K_1,0)$, and produces simulated proof $\pi\leftarrow\mathsf{NIZK.Sim}(\mathsf{crs}_{\text{sim}},\mathsf{td}_{\text{sim}},\text{stat}_{m,\mathsf{ct}_i'})$ for statement that ct_i' encrypts m. Output $\sigma=(\mathsf{ct}_i',\pi,0)$. For verification, Verify parses $\sigma=(\mathsf{ct},\pi,0)$, computes $\mathsf{ct}_{\mathsf{id}}=\mathsf{FHE.Eval}(\mathsf{fpk},\mathsf{id},\mathsf{ct}_1',...,\mathsf{ct}_N')=\mathsf{ct}_i'$, generates $\mathsf{crs}=\mathsf{ObfGenCRS}(0)$, and verifies $\mathsf{NIZK.Verify}(\mathsf{crs},\text{stat}_{m,\mathsf{ct}_i'},\pi)$. By construction, $\mathsf{GenCRS}(0)$ via $\mathsf{ObfGenCRS}$ produces the same CRS as GenCRS in signing, and by completeness of NIZK, $\mathsf{NIZK.Verify}$ accepts simulated proofs with overwhelming probability $1-\mathsf{negl}(\lambda)$.

For evaluation correctness, Eval processes circuit f gate-by-gate. For each NAND gate at level j with inputs (σ_0,m_0) and (σ_1,m_1), the obfuscated circuit $\mathsf{ObfEval}$ (hardcoded with K_2) first verifies input proofs at level $j-1$ via $\mathsf{NIZK.Verify}$, computes $\mathsf{ct}=\mathsf{FHE.Eval}(\mathsf{fpk},\text{NAND},\mathsf{ct}_0,\mathsf{ct}_1)$ and $m=\text{NAND}(m_0,m_1)$, generates $(\mathsf{crs}_{\text{sim}}',\mathsf{td}_{\text{sim}}')=\mathsf{GenCRS}(j+1)$ using $\mathsf{PRF}(K_2,(m,\mathsf{ct},j+1))$, produces $\pi=\mathsf{NIZK.Sim}(\mathsf{crs}_{\text{sim}}',\mathsf{td}_{\text{sim}}',\text{stat}_{m,\mathsf{ct}})$. By FHE evaluation correctness, ct_f encrypts $f(m_1,\dots,m_N)$. Final verification $\mathsf{Verify}(\mathsf{vk},f,y,\sigma_f)$ computes $\mathsf{ct}_f=\mathsf{FHE.Eval}(\mathsf{fpk},f,\mathsf{ct}_1',...,\mathsf{ct}_N')$, generates $\mathsf{crs}=\mathsf{ObfGenCRS}(d)$

where $d = \mathsf{depth}(f)$, and verifies $\mathsf{NIZK.Verify}(\mathsf{crs}, \mathrm{stat}_{y,\mathsf{ct}_f}, \pi_f)$. Since $\mathsf{GenCRS}(d)$ produces consistent CRS and NIZK completeness holds, verification succeeds with probability $1 - |f| \cdot \mathsf{negl}(\lambda) = 1 - \mathsf{negl}(\lambda)$ where $|f|$ denotes circuit size. □

Theorem 16. *Let* $\mathsf{HS}' = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify}, \mathsf{Eval})$ *as defined in Definition 5. Under subexponentially-secure indistinguishability obfuscation* iO*, subexponentially-secure FHE, composably zero-knowledge and extractable NIZK, and puncturable PRF, for all PPT adversaries* $\mathcal{A}$ *and all security parameters* λ*:*

$$\Pr\left[\begin{array}{l} (\mathsf{vk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda, 1^N); \\ (f^*, y^*, \sigma^*, \tau^*) \leftarrow \mathcal{A}^{\mathsf{Sign}(\mathsf{sk},\cdot,\cdot)}(\mathsf{vk}); \\ b \leftarrow \mathsf{Verify}(\mathsf{vk}, f^*, y^*, \sigma^*); \\ \{m_j\}_{j\in[N]} = \textit{messages signed under } \tau^*; \\ (b=1) \wedge (f^*(m_1,\ldots,m_N) \neq y^*) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

Proof. Assume adversary $\mathcal{A}$ succeeds with non-negligible probability ϵ. Construct sequence of hybrid experiments $\{\mathcal{H}_\ell\}_{\ell=0}^{d^*+1}$ where $d^* = \mathsf{depth}(f^*)$.

$\mathcal{H}_0$: Real unforgeability experiment where KeyGen generates $(\mathsf{vk},\mathsf{sk})$ honestly, $\mathcal{A}$ queries Sign oracle to obtain signatures on messages $\{m_j\}_{j\in[N]}$ under label τ^*, and outputs forgery $(f^*, y^*, \sigma^*, \tau^*)$ with $y^* \neq f^*(m_1,\ldots,m_N)$.

$\mathcal{H}_\ell$ for $\ell \in [d^*]$: Replace $\mathsf{ObfEval}$ with obfuscation of modified circuit that: (i) for levels $< \ell$, behaves as before; (ii) at level ℓ, uses punctured PRF key $K_{2,\{(m_\ell^*, \mathsf{ct}_\ell^*, \ell)\}}$ where $(m_\ell^*, \mathsf{ct}_\ell^*)$ is the ℓ-th gate evaluation in forgery path, hardcodes real NIZK proof π_ℓ^* for $(m_\ell^*, \mathsf{ct}_\ell^*)$, and uses real CRS crs_ℓ instead of simulated; (iii) for levels $> \ell$, uses simulated proofs as before. By security of puncturable PRF, $K_{2,\{(m_\ell^*, \mathsf{ct}_\ell^*, \ell)\}}$ is indistinguishable from K_2 at all points except $(m_\ell^*, \mathsf{ct}_\ell^*, \ell)$. By subexponential security of iO with security parameter $\kappa_2 = (|\mathsf{ct}| + N + \log N + 2\log^2 \lambda)^{1/\varepsilon}$ chosen such that $2^{\kappa_2^\varepsilon} = \mathsf{poly}(\lambda)$, the obfuscations are indistinguishable with advantage $2^{-\kappa_2^\varepsilon} = \mathsf{negl}(\lambda)$. Hence $|\Pr[\mathcal{H}_{\ell-1}] - \Pr[\mathcal{H}_\ell]| \leq \mathsf{negl}(\lambda)$.

$\mathcal{H}_{d^*+1}$: All levels use real CRS and real NIZK proofs. If $\mathcal{A}$ outputs valid forgery $\sigma^* = (\mathsf{ct}^*, \pi^*, d^*)$ with $\mathsf{Verify}(\mathsf{vk}, f^*, y^*, \sigma^*) = 1$ but $y^* \neq f^*(m_1, \ldots, m_N)$, then $\mathsf{ct}_f^* = \mathsf{FHE.Eval}(\mathsf{fpk}, f^*, \mathsf{ct}'_1, \ldots, \mathsf{ct}'_N)$ encrypts $f^*(m_1,\ldots,m_N) \neq y^*$ by FHE correctness. However, $\mathsf{NIZK.Verify}(\mathsf{crs}_{d^*}, \mathrm{stat}_{y^*,\mathsf{ct}_f^*}, \pi^*) = 1$ implies ct_f^* encrypts y^* by extractability of NIZK with knowledge extractor $\mathcal{E}$. This contradicts semantic security of FHE: construct FHE distinguisher that generates $(\mathsf{fpk},\mathsf{fsk})$, receives challenge ciphertext encrypting either $f^*(m_1,\ldots,m_N)$ or y^*, embeds in verification key, runs $\mathcal{A}$, extracts witness from π^* using $\mathcal{E}$, and distinguishes based on extracted plaintext. Since FHE is subexponentially secure, extraction succeeds with probability at most $\mathsf{negl}(\lambda)$.

By hybrid argument over d^*+1 hybrids, $|\Pr[\mathcal{H}_0] - \Pr[\mathcal{H}_{d^*+1}]| \leq (d^*+1) \cdot \mathsf{negl}(\lambda) = \mathsf{negl}(\lambda)$ for polynomial $d^* \leq \mathsf{poly}(\lambda)$. Hence $\epsilon = \Pr[\mathcal{H}_0] \leq \mathsf{negl}(\lambda)$. □

Theorem 17. *Let* $\mathsf{HS}' = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify}, \mathsf{Eval})$ *as defined in Definition 5. Under composably zero-knowledge NIZK and pseudorandomness of PRF, there exists PPT simulator* $\mathcal{S}_{ctx}$ *such that for all security parameters* λ*, all* N*, all* $\{m_j\}_{j\in[N]} \in \{0,1\}^N$*,*

and all circuits $f:\{0,1\}^N\to\{0,1\}$*:*

$$\left\{\begin{array}{l}(\mathsf{vk},\mathsf{sk})\leftarrow\mathsf{KeyGen}(1^\lambda,1^N);\\ \forall j:\sigma_j\leftarrow\mathsf{Sign}(\mathsf{sk},m_j,j);\\ \sigma_f\leftarrow\mathsf{Eval}(\mathsf{vk},f,(m_1,\sigma_1),...,(m_N,\sigma_N));\\ \textbf{\textit{return}}\ \sigma_f\end{array}\right\}$$
$$\stackrel{c}{\approx}$$
$$\left\{\begin{array}{l}(\mathsf{vk},\mathsf{sk})\leftarrow\mathsf{KeyGen}(1^\lambda,1^N);\\ \widetilde{\sigma}_f\leftarrow\mathcal{S}_{ctx}(\mathsf{sk},f,f(m_1,...,m_N));\\ \textbf{\textit{return}}\ \widetilde{\sigma}_f\end{array}\right\}.$$

Proof. Construct simulator $\mathcal{S}_{\mathrm{ctx}}(\mathsf{sk},f,y)$ that operates as follows: parse $\mathsf{sk}=(K_1,K_2,\mathsf{fsk})$ and set $d=\mathsf{depth}(f)$. Generate fresh FHE ciphertext $\mathsf{ct}_f\leftarrow\mathsf{FHE.Enc}(\mathsf{fpk},y)$ encrypting output y directly. Generate simulated CRS $(\mathsf{crs}_{\mathrm{sim}},\mathsf{td}_{\mathrm{sim}})=\mathsf{GenCRS}(d)$ using $\mathsf{NIZK.SimSetup}(1^{\kappa_2};\mathsf{PRF}(K_1,d))$. Produce the value of the simulated proof $\pi_f\leftarrow\mathsf{NIZK.Sim}(\mathsf{crs}_{\mathrm{sim}},\mathsf{td}_{\mathrm{sim}},\mathrm{stat}_{y,\mathsf{ct}_f})$ using trapdoor $\mathsf{td}_{\mathrm{sim}}$. Output $\widetilde{\sigma}_f=(\mathsf{ct}_f,\pi_f,d)$.

Real evaluated signature: Eval computes $\mathsf{ct}_f=\mathsf{FHE.Eval}(\mathsf{fpk},f,\mathsf{ct}'_1,...,\mathsf{ct}'_N)$ where each ct'_j encrypts m_j. By FHE evaluation correctness, ct_f encrypts $f(m_1,...,m_N)=y$. At each gate evaluation at level $j\in[d]$, $\mathsf{ObfEval}$ generates CRS via $\mathsf{GenCRS}(j+1)$ using $\mathsf{PRF}(K_2,(\cdot,\cdot,j+1))$ and produces simulated proof. Final signature $\sigma_f=(\mathsf{ct}_f,\pi_f,d)$ where π_f is simulated proof for $\mathrm{stat}_{y,\mathsf{ct}_f}$.

Simulated signature: $\mathcal{S}_{\mathrm{ctx}}$ generates fresh $\mathsf{ct}_f\leftarrow\mathsf{FHE.Enc}(\mathsf{fpk},y)$ and simulated proof π_f for same statement $\mathrm{stat}_{y,\mathsf{ct}_f}$.

Indistinguishability follows from: (i) semantic security of FHE ensures that the $\mathsf{FHE.Eval}(\mathsf{fpk},f,\mathsf{ct}'_1,...,\mathsf{ct}'_N)\stackrel{c}{\approx}\mathsf{FHE.Enc}(\mathsf{fpk},f(m_1,...,m_N))$ since both encrypt same plaintext y; (ii) composable zero-knowledge of NIZK ensures simulated proofs from $\mathsf{NIZK.Sim}$ are indistinguishable from real proofs for true statements; (iii) PRF security ensures $\mathsf{PRF}(K_2,\cdot)$ outputs are pseudorandom, making CRS generation indistinguishable. By composition of security reductions, $|\Pr[\mathcal{D}(\sigma_f)=1]-\Pr[\mathcal{D}(\widetilde{\sigma}_f)=1]|\le\mathsf{negl}(\lambda)$ for any PPT distinguisher $\mathcal{D}$. □

Theorem 18. *Let* $\mathsf{HS}'=(\mathsf{KeyGen},\mathsf{Sign},\mathsf{Verify},\mathsf{Eval})$ *as defined in Definition 5. There exists polynomial* $\mathsf{poly}(\cdot,\cdot)$ *such that for all security parameters* λ*, all* N*, all circuits* $f:\{0,1\}^N\to\{0,1\}$*, and all messages* $\{m_j\}_{j\in[N]}$*:* $|\sigma_f|\le\mathsf{poly}(\lambda,\mathsf{depth}(f))$*, where* $\sigma_f\leftarrow\mathsf{Eval}(\mathsf{vk},f,(m_1,\sigma_1),\ldots,(m_N,\sigma_N))$ *with* $(\mathsf{vk},\mathsf{sk})\leftarrow\mathsf{KeyGen}(1^\lambda,1^N)$ *and* $\sigma_j\leftarrow\mathsf{Sign}(\mathsf{sk},m_j,j)$*.*

Proof. Parse evaluated signature $\sigma_f=(\mathsf{ct}_f,\pi_f,d)$ where $d=\mathsf{depth}(f)$. By FHE compactness (Definition 14), $|\mathsf{ct}_f|\le\mathsf{poly}_{\mathrm{FHE}}(\kappa_1,d)$ where $\kappa_1=(N+\log N+2\log^2\lambda)^{1/\varepsilon}$ is FHE security parameter. Since $\kappa_1=\mathsf{poly}(\lambda,N)$, we have $|\mathsf{ct}_f|=\mathsf{poly}(\lambda,N,d)$. For NIZK proof π_f, by succinctness of NIZK, $|\pi_f|\le\mathsf{poly}_{\mathrm{NIZK}}(\kappa_2,|\mathrm{stat}_{y,\mathsf{ct}_f}|)$ where $\kappa_2=(|\mathsf{ct}|+N+\log N+2\log^2\lambda)^{1/\varepsilon}$ and statement size $|\mathrm{stat}_{y,\mathsf{ct}_f}|=O(|\mathsf{ct}_f|+1)=\mathsf{poly}(\lambda,N,d)$.

Hence $|\pi_f| = \mathsf{poly}(\lambda, N, d)$. Depth value d requires $O(\log d)$ bits. Total signature size $|\sigma_f| = |\mathsf{ct}_f| + |\pi_f| + O(\log d) = \mathsf{poly}(\lambda, N, d)$. Since N is fixed data-size bound (not dependent on circuit size $|f|$), and choosing $\mathsf{poly}(\lambda,d) = \mathsf{poly}(\lambda,N,d)$ to absorb N-dependence, we obtain $|\sigma_f| \leq \mathsf{poly}(\lambda, \mathsf{depth}(f))$ independent of circuit size $|f|$. □

A.6 Correctness and Compactness of UT

According to [5], a static UT is required to satisfy the other properties as well, such as evaluation and verification correctness, and compactness.

Definition 29 (Correctness). *A static universal thresholdizer scheme* UT *satisfies correctness if for all* $\lambda \in \mathbb{N}$*, all efficient access structures* $(\mathbb{S},N)$*, all* $x \in \{0,1\}^k$*, all circuits* $C : \{0,1\}^k \to \{0,1\}$ *of depth at most* d*, and all authorized sets* $S \in \mathbb{S}$*:*

$$\Pr\left[\begin{array}{ll} (\mathsf{p},s_1,\ldots,s_N) \leftarrow \mathsf{Setup}(1^\lambda,1^d,(\mathbb{S},N),x) & \\ \forall i \in S : y_i \leftarrow \mathsf{Eval}(\mathsf{p},s_i,C) & : y = C(x) \\ y \leftarrow \mathsf{Combine}(\mathsf{p},\{y_i\}_{i\in S}) & \end{array}\right] = 1 - \mathsf{negl}(\lambda)$$

Moreover, for all honestly generated partial evaluations: $\Pr[\mathsf{Verify}(\mathsf{p},y_i,C)=1]=1$.

Definition 30 (Compactness). *A static universal thresholdizer scheme* UT *satisfies compactness if there exists a polynomial* $\mathsf{poly}(\lambda)(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$*, all efficient access structures* $(\mathbb{S},N)$*, all* $x \in \{0,1\}^k$*, and all circuits* C *of depth at most* d*:* $|y_i| \leq \mathsf{poly}(\lambda)(\lambda,d,N)$*, where* $y_i \leftarrow \mathsf{Eval}(\mathsf{p},s_i,C)$ *for honestly generated parameters and shares.*

A.7 Robustness by Boneh. et al. [5]

Definition 31 (Incomplete Robustness [5]). *A* UT *is robust if for all* λ*, and depth bound* d*, the following holds. For any PPT adversary* $\mathcal{A}$*, the experiment* $\mathsf{Expt}_{\mathcal{A},\mathrm{UT},\mathrm{rb}}(1^\lambda,1^d)$ *in Figure 8 outputs 1 with negligible probability.*

$\mathsf{Exp}_{\mathcal{A},\mathsf{UT},\mathsf{rb}}(1^\lambda,1^d)$

1: $(x,\mathbb{A}) \leftarrow \mathcal{A}(1^\lambda,1^d)$
2: $(\mathsf{pp},\mathsf{s}_1,\ldots,\mathsf{s}_N) \leftarrow \mathsf{UT.Setup}(1^\lambda,1^d,\mathbb{A},x)$
3: $\mathsf{y}_i^* \leftarrow \mathcal{A}(\mathsf{pp},\mathsf{s}_1,\ldots,\mathsf{s}_N)$
4: return $\mathsf{y}_i^* \neq \mathsf{UT.Eval}(\mathsf{pp},\mathsf{s}_i,C) \wedge \mathsf{UT.Verify}(\mathsf{pp},\mathsf{y}_i^*,C)=1$

Fig. 8: Incomplete Robustness Experiment for UT

A.8 Security and Robustness by Campanelli & Khoshakhlagh [7]

The security properties of UT as defined in [7] are as follows:

Definition 32 (Security). *A UT scheme satisfies security with respect to circuit sampler D if for all $\lambda \in \mathbb{N}$, there exists a PPT algorithm $\mathcal{S} = (\mathcal{S}_S, \mathcal{S}_E)$ such that for any PPT adversary $\mathcal{A}$, the following experiments $\mathsf{Exp}^{\mathsf{real}}_{\mathcal{A},\mathsf{UT}}(1^\lambda, d, N)$ and $\mathsf{Exp}^{\mathsf{ideal}}_{\mathcal{A},\mathsf{UT}}(1^\lambda, d, N)$ are computationally indistinguishable:*

$\mathsf{Exp}^{\mathsf{real}}_{\mathcal{A},\mathsf{UT}}(1^\lambda, d, N)$

1: $x \leftarrow \mathcal{A}(1^\lambda, d, N)$
2: $(\mathsf{pp}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N) \leftarrow \mathsf{UT.Setup}(1^\lambda, d, N, x)$
3: $\mathcal{C} \leftarrow \mathcal{A}(\mathsf{pp})$, where $|\mathcal{C}| = d - 1$
4: $\{\mathsf{sk}_i\}_{i \in \mathcal{C}} \rightarrow \mathcal{A}$
5: $\forall C \in \mathcal{Q} : \{y_i \leftarrow \mathsf{UT.Eval}(\mathsf{pp}, \mathsf{sk}_i, C)\}_{i \in [N]} \leftarrow \mathcal{O}^{\mathsf{real}}_{\mathsf{UT}}(C)$
6: $b \leftarrow \mathcal{A}^{\mathcal{O}^{\mathsf{real}}_{\mathsf{UT}}}(\mathsf{pp}, \{\mathsf{sk}_i\}_{i \in \mathcal{C}})$
7: **return** b

Fig. 9: Real-World Experiment for UT

$\mathsf{Exp}^{\mathsf{ideal}}_{\mathcal{A},\mathsf{UT}}(1^\lambda, d, N)$

1: $x \leftarrow \mathcal{A}(1^\lambda, d, N)$
2: $(\mathsf{pp}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N) \leftarrow \mathsf{Sim}_S(1^\lambda, d, N)$
3: $\mathcal{C} \leftarrow \mathcal{A}(\mathsf{pp})$, where $|\mathcal{C}| = d - 1$
4: $\{\mathsf{sk}_i\}_{i \in \mathcal{C}} \rightarrow \mathcal{A}$
5: $\forall C \in \mathcal{Q} : \{y_i\}_{i \in [N]} \leftarrow \mathsf{Sim}_E(\mathsf{trapd}, C, C(x)) \leftarrow \mathcal{O}^{\mathsf{ideal}}_{\mathsf{UT}}(C)$
6: $b \leftarrow \mathcal{A}^{\mathcal{O}^{\mathsf{ideal}}_{\mathsf{UT}}}(\mathsf{pp}, \{\mathsf{sk}_i\}_{i \in \mathcal{C}})$
7: **return** b

Fig. 10: Ideal-World Experiment for UT

Definition 33 (Robustness). *A UT scheme satisfies robustness if for all $\lambda \in \mathbb{N}$, for any PPT adversary $\mathcal{A}$, the probability that the following experiment $\mathsf{Expt}_{\mathcal{A},\mathsf{robust}}(1^\lambda)$ outputs 1 is negligible in λ:*

$\mathsf{Exp}^{\mathsf{pe}}_{\mathcal{A},\mathsf{UT}}(1^\lambda,d,N)$

1: $x \leftarrow \mathcal{A}(1^\lambda)$
2: $(\mathsf{pp},\mathsf{sk}_1,\ldots,\mathsf{sk}_N) \leftarrow \mathsf{UT.Setup}(1^\lambda,d,N,x)$
3: $(\mathsf{pp},\mathsf{sk}_1,\ldots,\mathsf{sk}_N) \rightarrow \mathcal{A}$
4: $\pi_i^* \leftarrow \mathcal{A}(\mathsf{pp},\mathsf{sk}_1,\ldots,\mathsf{sk}_N)$
5: $\pi_i \leftarrow \mathsf{UT.PartEval}(\mathsf{pp},\mathsf{sk}_i,i,C)$
6: **return** $(\pi_i^* \neq \pi_i) \wedge (\mathsf{UT.VfyEval}(\mathsf{pp},\pi_i^*,i,C)=1)$

Fig. 11: Partial Evaluation for Robustness of UT

A.9 More on Existing UT Constructions

In this section, we show our formalizations of each construction of UT, along with details on complexity analysis.

Boneh et al. [5], $\mathsf{UT_2} = \mathsf{TFHE} + \mathsf{PZK} + \mathsf{Com}$ The developers of this technique define the linear secret sharing scheme with binary reconstruction coefficients, and this class includes access structures defined by monotone Boolean formulas [53]. They then combine it with FHE [21] to achieve TFHE. SS.Share processes $k \in \mathbb{Z}_q$ in $O(1)$ time. For threshold access with N parties, formula size is $O(N^{5.2})$ [54]. Traversal takes $O(|C|)$ time. Share generation for $O(|C|)$ nodes: `AND` gates and `OR` gates cost $O(1)$, yielding $O(N^{5.2})$ total nodes and space. So, SS.Share time and space are $O(N^{5.2})$. SS.Combine processes input in $O(|S|)$ time. Valid share sets examined up to $\ell = O(N^{4.2})$, recursive algorithm takes $O(\ell^2)$ worst-case time. Secret reconstruction sums shares in $O(\ell)$ time with $O(\ell)$ space for valid sets. Therefore, SS.Combine time is $O(N^{8.4})$ worst-case, and space is $O(N^{4.2})$. Overall {0,1}-LSSS time complexity is $O(N^{8.4})$, and space is $O(N^{5.2})$.

According to De Santis et al. [14], the computational complexities of the PZK protocol are as follows. The preprocessing algorithm $\mathsf{PZK.Pre}(1^\lambda)$ requires $O(\lambda)$ time and space cost. The proof generation algorithm $\mathsf{PZK.Prove}(\sigma_P,x,w)$ has time $O(\lambda \cdot |\Psi|)$ and space cost $O(\lambda + |\Psi|)$. Similarly, the verification algorithm $\mathsf{PZK.Verify}(\sigma_V,x,\pi)$ exhibits time $O(\lambda \cdot |\Psi|)$ and space $O(\lambda + |\Psi|)$. Consequently, the overall time and space complexity of the PZK are $O(\lambda \cdot |\Psi|)$ and $O(\lambda + |\Psi|)$, respectively.

Based on Blum [15], $\mathsf{Com}(m;r)$ takes $O(\lambda \cdot |m|)$ time and $O(\lambda + |m|)$ space. $\mathsf{C.Verify}(m,com,r)$ takes $O(\lambda \cdot |m|)$ time and $O(\lambda + |m|)$ space. So, the commitment scheme's time and space costs are $O(\lambda \cdot |m|)$ and $O(\lambda + |m|)$.

From Gentry et al. [21], FHE.Setup takes $O(1)$ for parameter selection, $O(nm)$ for matrix $A \in \mathbb{Z}_q^{n \times m}$, $O(n)$ for secret key s and error e, and $O(n^2 m)$ for public key $\tilde{A} = [A; s^T A + ce^T]$ with $O(nm)$ space. Therefore, FHE.Setup time is $O(n^2 m)$ and space is $O(nm)$. FHE.Encrypt takes $O(m^2)$ for random matrix $R \in \{0,1\}^{m \times m}$ and $O(nm^2)$ for ciphertext $ct = \tilde{A}R + \mu G$ (where μ is the message) with $O(nm)$ space. So,

FHE.Encrypt time is $O(nm^2)$ and space is $O(nm)$. FHE.Eval takes $O(nm)$ for addition. Multiplication involves $O(m^2 \log q)$ for gadget decomposition G^{-1} and $O(nm^2)$ for matrix multiplication, yielding $O(2^d nm^2 \log q)$ for depth d circuits with $O(nm)$ space. Hence, FHE.Eval time is $O(2^d nm^2 \log q)$ and space is $O(nm)$. FHE.Decrypt takes $O(nm)$ for inner product $\langle sk, ct \cdot w_m \rangle$ and $O(1)$ for rounding with $O(1)$ space. Thus, FHE.Decrypt time is $O(nm)$ and space is $O(1)$. Overall, FHE time complexity is $O(2^d nm^2 \log q)$, and space is $O(nm)$.

Putting {0,1}-LSSS and FHE together, the TFHE of Boneh et al. [5] consists of $O(2^d nm^2 \log q + N^{8.4})$ time, and $O(nm + N^{5.2})$ space complexity.

$\mathsf{UT}_2.\mathsf{Setup}(1^\lambda, 1^d, \mathbb{A}, x)$

1: $(\mathsf{tfhepk}, \mathsf{tfhesk}_1, ..., \mathsf{tfhesk}_N) \leftarrow \mathsf{TFHE.Setup}(1^\lambda, 1^d, \mathbb{A})$
2: $\forall i \in [k].\, \mathsf{ct}_i \leftarrow \mathsf{TFHE.Encrypt}(\mathsf{tfhepk}, x_i)$
3: $\forall i \in [N].\, (\sigma_{V,i}, \sigma_{P,i}) \leftarrow \mathsf{PZK.Pre}(1^\lambda)$
4: $\forall i \in [N].\, r_i \xleftarrow{R} \{0,1\}^\lambda$
5: $\forall i \in [N].\, \mathsf{com}_i \leftarrow \mathsf{Com}(\mathsf{tfhesk}_i; r_i)$
6: $\mathsf{pp} \leftarrow (\mathsf{tfhepk}, \{\mathsf{ct}_i\}_{i\in[k]}, \{\sigma_{V,i}\}_{i\in[N]}, \{\mathsf{com}_i\}_{i\in[N]})$
7: $\forall i \in [N].\, s_i \leftarrow (\mathsf{tfhesk}_i, \sigma_{P,i}, r_i)$
8: **return** $(\mathsf{pp}, s_1, ..., s_N)$

$\mathsf{UT}_2.\mathsf{Combine}(\mathsf{pp}, B)$

1: $B \leftarrow \{y_i\}_{i\in S} \,|\, (S \subseteq \{1,...,N\}) \vee (S \in \mathbb{A})$
2: $\forall i \in S: y_i \leftarrow (p_i, \pi_i)$
3: $y \leftarrow \mathsf{TFHE.FinDec}(\mathsf{tfhepk}, \{p_i\}_{i\in S})$
4: **return** y

$\mathsf{UT}_2.\mathsf{Eval}(\mathsf{pp}, s_i, \mathbb{C})$

1: $\hat{\mathsf{ct}} \leftarrow \mathsf{TFHE.Eval}(\mathsf{tfhepk}, \mathbb{C}, \{\mathsf{ct}_i\}_{i=1}^k)$
2: $p_i \leftarrow \mathsf{TFHE.PartDec}(\mathsf{tfhepk}, \hat{\mathsf{ct}}, \mathsf{tfhesk}_i)$
3: $\Psi_i : \exists (\mathsf{tfhesk}_i, r_i) \,|\, \mathsf{com}_i \leftarrow \mathsf{Com}(\mathsf{tfhesk}_i; r_i)$
$\wedge\ p_i \leftarrow \mathsf{TFHE.PartDec}(\mathsf{tfhepk}, \hat{\mathsf{ct}}, \mathsf{tfhesk}_i)$
4: $\pi_i \leftarrow \mathsf{PZK.Prove}(\sigma_{P,i}, \Psi_i, (\mathsf{tfhesk}_i, r_i))$
5: $y_i \leftarrow (p_i, \pi_i)$
6: **return** y_i

$\mathsf{UT}_2.\mathsf{Verify}(\mathsf{pp}, y_i, \mathbb{C})$

1: $\hat{\mathsf{ct}} \leftarrow \mathsf{TFHE.Eval}(\mathsf{tfhepk}, \mathbb{C}, \{\mathsf{ct}_i\}_{i=1}^k)$
2: $\Psi_i : \exists (\mathsf{tfhesk}_i, r_i) \,|\, \mathsf{com}_i \leftarrow \mathsf{Com}(\mathsf{tfhesk}_i; r_i)$
$\wedge\ p_i \leftarrow \mathsf{TFHE.PartDec}(\mathsf{tfhepk}, \hat{\mathsf{ct}}, \mathsf{tfhesk}_i)$
3: $y_i \leftarrow (p_i, \pi_i)$
4: **return** $\mathsf{PZK.Verify}(\sigma_{V,i}, \Psi_i, \pi_i)$

Fig. 12: Building the UT_2 from TFHE+PZK+C, Boneh et al. [5]

Campanelli et al [7], UT_3 = TFHE+HS This method adapts TFHE from [5] directly, so we recall its cumulative complexity with $O(2^d nm^2 \log q + N^{8.4})$ for time and $O(nm + N^{5.2})$ for space from Section A.9.

For HS, this technique uses a context-hiding homomorphic signature $\mathsf{HS} = (\mathsf{HS.Setup}, \mathsf{HS.Sign}, \mathsf{HS.Verify}, \mathsf{HS.Eval})$ from [22], which is based on lattices. Considering t and s as time and space cost, we have the following complexities.

- $\mathsf{HS.Setup}(1^\lambda)$ generates lattice parameters $(t{:}O(\lambda))$ and performs SIS-based key generation $\big(t{:}O(\lambda^2),\ s{:}O(\lambda^2)$ for keys$\big)$. Thus, HS.Setup time is $O(\lambda^2)$ and space is $O(\lambda^2)$.
- $\mathsf{HS.Sign}(sk_{hs}, \tau, m)$ computes hash $\big(t : O(|m| + |\tau|)\big)$, samples short vector $\big(t{:}O(\lambda^2)\big)$, and performs matrix-vector multiplication $\big(t{:}O(\lambda^2)\big)$, with $O(\lambda^2 +$

$|m| + |\tau|)$ space. Therefore, HS.Sign time is $O(\lambda^2 + |m| + |\tau|)$ and space is $O(\lambda^2+|m|+|\tau|)$.

- HS.Verify$(pk_{hs}, \tau, m, \sigma, f)$ computes hash $\big(t : O(|m| + |\tau|)\big)$, verifies function evaluation $\big(t : O(\lambda|f|)\big)$, and performs matrix-vector multiplication with norm check $\big(t : O(\lambda^2)\big)$, with $O(\lambda^2 + |m| + |\tau| + |f|)$ space. Hence, HS.Verify time is $O(\lambda^2+\lambda|f|+|m|+|\tau|)$ and space is $O(\lambda^2+|m|+|\tau|+|f|)$.
- HS.Eval(pk_{hs},τ,f,σ) evaluates function on signatures $\big(t : O(\lambda|f|d)\big)$ and performs matrix-vector operations $\big(t : O(\lambda^2\ |f|)\big)$, with $O(\lambda^2 \log d+|\tau|+|f|)$ space. Thus, HS.Eval time is $O(\lambda|f|(\lambda+d))$ and space is $O(\lambda^2 \log d+|\tau|+|f|)$.
- Context-hiding simulator requires $O(\lambda^2+|f|)$ time and $O(\lambda^2+|f|)$ space, ensuring $\sigma_{f,y}$ reveals only $y = f(x)$.

$\mathsf{UT}_3.\mathsf{Setup}(1^\lambda,d,N,x)$

1: $(\mathsf{skhs},\mathsf{pkhs}) \leftarrow \mathsf{HS.Setup}(1^\lambda)$
2: $(\mathsf{skfhe},\mathsf{pkfhe}) \leftarrow \mathsf{TFHE.KeyGenAux}(1^\lambda,d,N)$
3: $\mathsf{ctx} \leftarrow \mathsf{TFHE.Enc}(\mathsf{pkfhe},x)$
4: $\mathsf{sk} \leftarrow \mathsf{UT-AuxSetup}(\mathsf{skfhe},\mathsf{skhs})$
5: **return** $(\mathsf{pp} := (\mathsf{pkfhe},\mathsf{pkhs},\mathsf{ctx}),\mathsf{sk})$

$\mathsf{UT}_3\text{-}\mathsf{AuxSetup}(\mathsf{skfhe},\mathsf{skhs})$

1: $\forall i \in \{1,...,N\}$
2: $\quad \mathsf{sk}'_i[\mathsf{fhe}] \leftarrow \mathsf{SS}(d,N,\mathsf{skfhe})$
3: $\quad \mathsf{sk}'_i[\mathsf{hs}] \leftarrow \mathsf{HS.Sign}(\mathsf{skhs},\text{“}i\text{”},\mathsf{sk}'_i[\mathsf{fhe}])$
4: $\quad \mathsf{sk}'_i := (\mathsf{sk}'_i[\mathsf{fhe}],\mathsf{sk}'_i[\mathsf{hs}])$
5: **return** $\mathsf{sk}'_1,...,\mathsf{sk}'_N$

$\mathsf{UT}_3.\mathsf{PartEval}(\mathsf{pp} := (\mathsf{pkfhe},\mathsf{pkhs},\mathsf{ctx}),\mathsf{ski},i,\mathbb{C})$

1: $\mathsf{ct}' \leftarrow \mathsf{TFHE.Eval}(\mathsf{pkfhe},\mathsf{ctx},\mathbb{C})$
2: $y_i \leftarrow \mathsf{TFHE.PartDec}(\mathsf{ski}[\mathsf{fhe}],\mathsf{ct}')$
3: $\sigma_i \leftarrow \mathsf{HS.Eval}(\mathsf{pkhs},\text{“}i\text{”},\mathbb{C}_{\mathsf{Dec}},\mathsf{ski}[\mathsf{hs}])$
$\qquad\qquad | \ \mathbb{C}_{\mathsf{Dec}} := \mathsf{TFHE.PartDec}(\cdot,\mathsf{ct}')$
4: **return** (y_i,σ_i)

$\mathsf{UT}_3.\mathsf{VfyEval}(\mathsf{pp} := (\mathsf{pkfhe},\mathsf{pkhs}),\pi_i = (y_i,\sigma_i),i,\mathbb{C})$

1: $\mathsf{ct}' \leftarrow \mathsf{TFHE.Eval}(\mathsf{pkfhe},\mathsf{ctx},\mathbb{C})$
2: **return** $\mathsf{HS.Verify}(\mathsf{pkhs},\text{“}i\text{”},y_i,\sigma_i,\mathbb{C}_{\mathsf{Dec}})$
$\qquad\qquad | \ \mathbb{C}_{\mathsf{Dec}} := \mathsf{TFHE.PartDec}(\cdot,\mathsf{ct}')$

$\mathsf{UT}_3.\mathsf{Combine}(\mathsf{pp} := (\mathsf{pkfhe},\mathsf{pkhs}),y_1,...,y_d)$

1: **return** $\mathsf{TFHE.Dec}(\mathsf{pkfhe},\{y_1,...,y_d\})$

Fig. 13: Universal Thresholdizer from TFHE+HS, Campanelli & Khoshakhlagh [7]

Campanelli et al [7], 2-UT = HSS+HS In this technique, they use the same HS as Figure 13, so we recall its total time and space complexity as $O(\lambda^2 \cdot |f| + \lambda \cdot |f| \cdot d)$ and $O(\lambda^2 \cdot \log d + |m| + |\tau| + |f|)$, respectively.

For HSS, they adapted the 2-party Homomorphic Secret Sharing from [23], and for the lattice-based instantiation, they refer to [30].

- HSS.Share$(1^\lambda, x)$ generates LWE-based random matrices of dimension $n \times m$ $\big(t : O(nm)\big)$, samples error vectors $\big(t : O(n)\ \big)$, and performs matrix operations to encode the secret $\big(t : O(n^2)\ \big)$, with $O(nm)$ space for storing shares. Thus, HSS.Share time is $O(n^2+nm)$ and space is $O(nm)$.

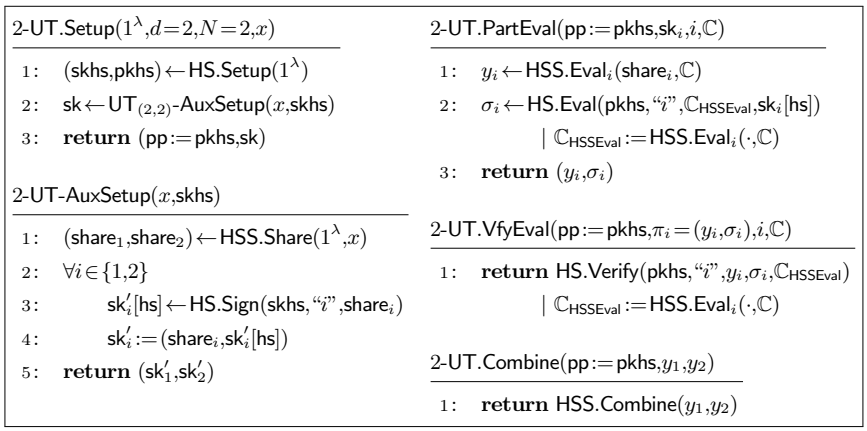

Fig. 14: Two-Party UT from HSS+HS, Campanelli & Khoshakhlagh [7]

- HSS.Eval$_b$(share$_b$,C) for $b \in \{1,2\}$ performs homomorphic operations on the share according to circuit C: addition operations $\big(t : O(n)\big)$, multiplication $\big(O(n^2 \log q)$ time due to noise management$\big)$, resulting in $O(2^d n^2 \log q)$ time for a circuit of depth d, with $O(n)$ space for the evaluation share. Therefore, HSS.Eval$_b$ time complexity is $O(2^d n^2 \log q)$ and space is $O(n)$.
- HSS.Combine(y_1,y_2) performs bit-wise operations on the shares $\big(t : O(n)\big)$ and final decoding $\big(t : O(1)\big)$, with $O(1)$ space for the result. Thus, HSS.Combine time is $O(n)$ and space is $O(1)$.
- Overall, the HSS scheme has time complexity $O(2^d n^2 \log q)$ dominated by evaluation, and space complexity $O(nm)$ dominated by the shares.

Overall time complexity of HSS+HS Figure 14 is dominated by $O(\lambda \cdot |C| \cdot \mathsf{poly}(\lambda))$ from PartEval and VfyEval operations, while space complexity includes $O(\lambda + |x|)$ for setup, $O(\lambda + |x| + \mathsf{poly}(\lambda))$ for partial evaluation, $O(\lambda \cdot |C| + \mathsf{poly}(\lambda))$ for verification, and $O(\mathsf{poly}(\lambda) + |y|)$ for combine.

Ebrahimi et al. [6], UT$_4$ = TFHE$'$+ PZK+ Com This type of UT, uses $\{0,1\}$-LSSS from [5], recalling its cumulative time cost as $O(N^{8.4})$, and space as $O(N^{5.2})$. The FHE of [6] is an instantiation of [16] and [21], which are detailed in the Appendix A.11.

This method uses KHPRF from [17]; KHPRF primitive is δ-almost key homomorphic where for any $k_1, k_2 \in K$, $F(k_1,x) + F(k_2,x) = F(k_1 + k_2, x) + e$, with $|e| \leq \delta$, with analysis spanning several functions. PRF evaluation function $F(k,x)$ requires $O(|k| + |x|)$ time complexity and $O(1)$ space complexity for output. Key generation for KHPRF consumes $O(\lambda)$ time complexity with $O(\lambda)$ space complexity for key storage. Secret sharing of KHPRF keys demands $O(n \cdot \ell)$ time complexity where n is the number of parties and ℓ the number of rows in the secret sharing matrix, with $O(n \cdot \ell)$ space complexity for all shares. Total KHPRF scheme exhibits time complexity $O(\lambda + n \cdot \ell + L + |k| + |x|)$, mostly dominated by secret sharing, with lattice-based

instantiations requiring $O(n\cdot\ell\cdot\mathsf{polylog}(q))$ where q is the lattice modulus. Total space complexity reaches $O(n\cdot\ell+\lambda)$, dominated by storage for key shares.

Combining $\{0,1\}$-LSSS with FHE and KHPRF, the TFHE scheme used in [6] exhibits time complexity $O(2^d\cdot n\cdot m^2\cdot\log q\cdot|C|+N^{8.4})$ dominated by FHE evaluation and LSSS operations, with space complexity $O(n\cdot m+N^{5.2})$ dominated by FHE key storage and LSSS requirements.

Ebrahimi & Yadav (2024) [6] adapt PZK from [14], recalling the overall time and space complexity of the PZK as $O(\lambda\cdot|\Psi|)$ and $O(\lambda+|\Psi|)$, respectively.

Here, we consider the commitment scheme from [15], which is also mentioned in [5]. Since it is not mentioned in [6], it is unclear which scheme they exactly use and whether it is attributed to a specific sub-procedure. So, we consider the commitment scheme's time and space costs as $O(\lambda\cdot|m|)$ and $O(\lambda+|m|)$.

$\mathsf{TFHE}'.\mathsf{Setup}(1^\lambda,1^d,A_{t,N})$

1: $(\mathsf{fpk},\mathsf{fsk})\leftarrow\mathsf{FHE.Setup}(1^\lambda,1^d)$
2: $(K_1,K_2,...,K_N)\leftarrow\mathsf{bSS.Share}(0\in\mathcal{K},A_{t,N})$
3: $(\mathsf{fsk}_1,\mathsf{fsk}_2,...,\mathsf{fsk}_N)\leftarrow\mathsf{bSS.Share}(\mathsf{fsk},A_{t,N})$
4: $\forall i\in[N].\,\mathsf{tfsk}_i:=(\{\mathsf{fhesk}_j\}_{j\in T_i},\{k_j\}_{j\in T_i})$
5: **return** $(\mathsf{tfpk}:=\mathsf{fpk},\{\mathsf{tfsk}_i\}_{i\in[N]})$

$\mathsf{TFHE}'.\mathsf{Encrypt}(\mathsf{tfpk},\mu)$

1: $\mathsf{ct}\leftarrow\mathsf{FHE.Encrypt}(\mathsf{tfpk},\mu)$
2: **return** ct

$\mathsf{TFHE}'.\mathsf{Eval}(\mathsf{tfpk},C,\{\mathsf{ct}_i\}_{i\in[k]})$

1: $\mathsf{ct}_C\leftarrow\mathsf{FHE.Eval}(\mathsf{tfpk},C,\mathsf{ct}_1,...,\mathsf{ct}_k)$
2: **return** ct_C

$\mathsf{TFHE}'.\mathsf{PartDec}(\mathsf{tfpk},\mathsf{tfsk}_i,\mathsf{ct})$

1: $(\{\mathsf{fhesk}_j\}_{j\in T_i},\{k_j\}_{j\in T_i}):=\mathsf{tfsk}_i$
2: $\forall j\in T_i.\,\xi_j\xleftarrow{R}[-B_{\mathrm{sm}},B_{\mathrm{sm}}]$
3: $\forall j\in T_i.\,\eta_j\xleftarrow{R}[-E_{\mathrm{sm}},E_{\mathrm{sm}}]$
4: $\forall j\in T_i.\,y_j:=\mathsf{FHE.decode}_0(\mathsf{fhesk}_j,\mathsf{ct})+\xi_j+F(k_j,H(\mathsf{ct}))+\eta_j$
5: $\mathsf{p}_i:=\{y_j\}_{j\in T_i}$
6: **return** p_i

$\mathsf{TFHE}'.\mathsf{FinDec}(\mathsf{tfpk},S,\{\mathsf{p}_i\}_{i\in S})$

1: $S\subseteq[n]\,|\,S\in A_{t,n}$
2: $\forall i\in S.\,\{y_j\}_{j\in T_i}:=\mathsf{p}_i$
3: $T:=\mathtt{MinimalValidShareSet}(\cup_{i\in S}T_i)$
4: $\mu:=\mathsf{FHE.decode}_1(\sum_{j\in T}y_j)$
5: **return** μ

Fig. 15: TFHE from $\{0,1\}$-LSSS with FHE and KHPRF, Ebrahimi & Yadav [6].

Cheon et al. [8], $\mathsf{UT}_5=\mathsf{TFHE}''+\mathsf{PZK}+\mathsf{Com}$ In this technique, the construction of $\mathsf{TreeSSS}=(\mathsf{TreeSSS.Share},\mathsf{TreeSSS.Combine})$ is inspired by Gupta and Mahajan's work (1996) [18] on constructing majority functions using small majority gates.

– $\mathsf{TreeSSS.Share}(sk,A_{N,t})$ implements tree secret sharing, taking $O((2t-1)^Lt^2\log q)$ time over L iterations, with $O((2t-1)^L\log q)$ space for level-L shares. Distributing shares takes $O((2t-1)^L)$ time and $O((2t-1)^L\log q/N)$ space per party, with $L\geq\log_{ct}N+\log_tN+O(1)$, $(2t-1)^L=O(N^{3+2.3/\log t})$.

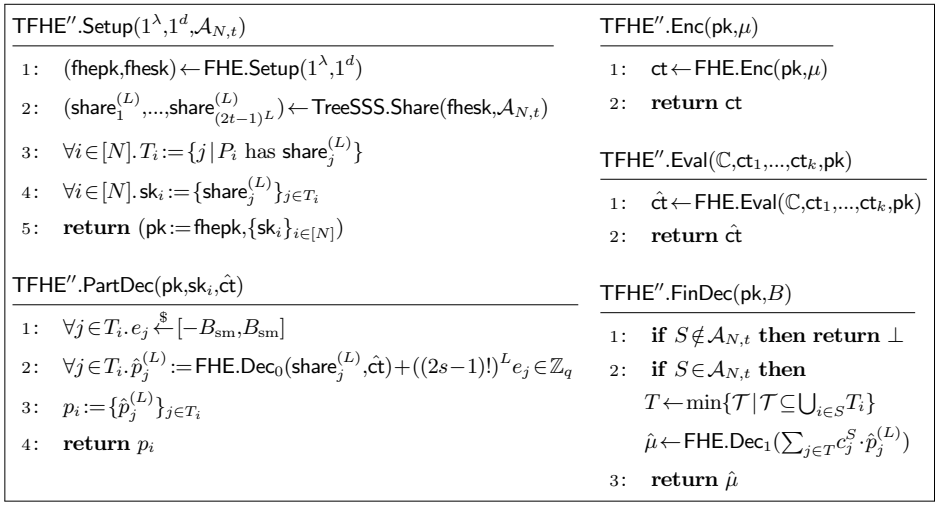

Fig. 16: Threshold Fully Homomorphic from TreeSSS and FHE, Cheon et al. [8]

– TreeSSS.Combine(B) reconstructs shares level by level, taking $O(L(2s-1)^L s^2 \log q)$ time and $O((2s-1)^L \log q)$ space; thus, total TreeSSS time complexity is $O((2s-1)^L s^2 \log q) = O(N^{3+2.3/\log s} s^2 \log q)$, and space cost is $O((2s-1)^L \log q) = O(N^{3+2.3/\log s} \log q)$.

They use Brakerski et al. (2014) [12] as the basis for their LWE-based FHE, which is detailed in Appendix A.11. Cheon et al.'s UT refers to the communication (not computational) efficiency of their construction compared to previous approaches by reducing the number of secret shares from Boneh et al.'s $O(N^5 \cdot 3)$ to approximately $O(N^3)$, Cheon et al. achieve efficiency gains for deployments. While both constructions have exponential offline verification costs, this is a one-time cost during setup.

A.10 Notes on Efficiency Costs of UTs

To give more details on conclusion of our analysis on construction methods' efficiency from a time complexity perspective, the method by Boneh et al. [5] exhibits complexity of $O(N \cdot \lambda \cdot n \cdot \log q + |C| \cdot n^2 \cdot \log q)$, scaling linearly with the number of parties N. Cheon et al. [8] improves efficiency for large numbers of participants with complexity $O(N^{3+2.3/\log s} \cdot s^2 \cdot \log q + |F| \cdot n^2 \cdot \log q)$, which performs better than Boneh's approach when the parameter s is chosen optimally. In contrast, Ebrahimi & Yadav [6] demonstrate significantly higher complexity at $O(2^d \cdot n \cdot m^2 \cdot \log q \cdot |C| + N^{8.4})$, with exponential dependency on circuit depth d and polynomial scaling of $N^{8.4}$, making it less efficient for deeper circuits and larger committee sizes. The Campanelli & Khoshakhlagh [7] TFHE+HS construction achieves $O(N \cdot \ell \cdot n + |C| \cdot n^2 \cdot \log q)$, similar to Boneh's approach but with potential improvements in the circuit evaluation component. Finally, their HSS+HS approach for the restricted (2,2)-threshold

case demonstrates the most favorable time cost at $O(\lambda \cdot |C| \cdot \mathsf{poly}(\lambda))$, though its applicability is limited to two-party settings.

Regarding space complexity, the techniques also exhibit differences in memory requirements. Boneh et al. [5] requires $O(N \cdot n \cdot \log q \cdot (\ell + \lambda))$ space, scaling linearly with the number of parties. Cheon et al. [8] demonstrates space cost of $O(N^{3+\frac{2.3}{\log s}} \cdot \log q + |S| \cdot N^{2+\frac{2.3}{\log s}} \cdot \log q)$, which grows faster with N but can be more efficient than Boneh's method for carefully chosen parameters. Ebrahimi & Yadav [6] exhibits space requirements of $O(N^{5.2} + n \cdot m + n \cdot (\lambda + |\Psi|))$, dominated by the $N^{5.2}$ term for large committees. The Campanelli & Khoshakhlagh [7] $\mathsf{TFHE}+\mathsf{HS}$ approach uses $O(n \cdot \log q \cdot (N + |C|) + |y|)$ space, offering better scaling with N compared to Cheon's and Ebrahimi's methods. Their $\mathsf{HSS}+\mathsf{HS}$ construction provides the most space-efficient solution at $O(\lambda \cdot |C| + |x| + |y| + \mathsf{poly}(\lambda))$, though again limited to the two-party threshold setting, making it suitable only for specialized applications where minimizing memory usage is crucial and only two parties are involved.

A.11 Further Details on the Complexity Analysis

The computational costs have been simplified based on the dominant terms, particularly their polynomial degree, exponential base, or logarithmic base, as terms with higher asymptotic growth rates dominate the expression.

The FHE of UT_4 [6] is an instantiation of [16] and [21] with the following complexity:

- $\mathsf{FHE.Setup}(1^\lambda, 1^d)$ performs key generation and parameter selection $\big(t : O(\lambda \cdot d \cdot n^2 \cdot \log q)\big)$, with $O(n \cdot \log q)$ space for storing pk and $sk \in \mathbb{Z}_q^n$. Thus, $\mathsf{FHE.Setup}$ time is $O(\lambda \cdot d \cdot n^2 \cdot \log q)$ and space is $O(n \cdot \log q)$.
- $\mathsf{FHE.Encrypt}(pk, \mu)$ executes matrix-vector multiplications $\big(t : O(n^2 \cdot \log q)\big)$, with $O(n \cdot \log q)$ space for ciphertext storage. Therefore, $\mathsf{FHE.Encrypt}$ time is $O(n^2 \cdot \log q)$ and space is $O(n \cdot \log q)$.
- $\mathsf{FHE.Eval}(pk, C, ct_1, ..., ct_k)$ processes circuit C on input ciphertexts $\big(t : O(|C| \cdot n^2 \cdot \log q)\big)$, with $O(n \cdot \log q)$ space for the output ciphertext. Hence, $\mathsf{FHE.Eval}$ time is $O(|C| \cdot n^2 \cdot \log q)$ and space is $O(n \cdot \log q)$.
- $\mathsf{FHE.Dec}_0(sk, ct)$ computes inner products $\big(t : O(n \cdot \log q)\big)$, with $O(\log q)$ space for the scalar output. Thus, $\mathsf{FHE.Dec}_0$ time is $O(n \cdot \log q)$ and space is $O(\log q)$.
- $\mathsf{FHE.Dec}_1(p)$ performs threshold checking $\big(t : O(1)\big)$, with $O(1)$ space for single-bit output. Therefore, $\mathsf{FHE.Dec}_1$ time is $O(1)$ and space is $O(1)$.
- In total, the FHE scheme has time complexity $O(\lambda \cdot d \cdot n^2 \cdot \log q + |C| \cdot n^2 \cdot \log q)$ dominated by setup and evaluation, with space complexity $O(n \cdot \log q)$ for ciphertext and key storage.

In UT_5, the LWE-based FHE from [12] has been used as $\mathsf{FHE} = (\mathsf{FHE.Setup}, \mathsf{FHE.KeyGen}, \mathsf{FHE.Enc}, \mathsf{FHE.Dec}, \mathsf{FHE.Add}, \mathsf{FHE.Mult}, \mathsf{FHE.Refresh})$ implementation with bootstrapping.

- $\mathsf{FHE.Setup}(1^\lambda, 1^L, b)$ creates $L+1$ parameter levels, thus, $\mathsf{FHE.Setup}$ time and space are both $O((L+1) \cdot \mathsf{poly}(\lambda, \log q))$.

- $\mathsf{FHE.KeyGen}(\{\mathtt{params}_j\})$ generates keys for all levels, therefore, $\mathsf{FHE.KeyGen}$ time cost is $O(Ln^3\log^2 q)$ and space complexity is $O(Ln^3\log q)$.
- $\mathsf{FHE.Enc}(\mathtt{params},pk,m)$ encrypts at highest level $O(n^2\log q)$ time and $O(n\log q)$ space cost.
- $\mathsf{FHE.Dec}(\mathtt{params},sk,c)$ decrypts with time $O(n\log q)$ and $O(n)$ space complexity.
- $\mathsf{FHE.Add}(pk,c_1,c_2)$ adds and refreshes with a time complexity of $O(n^3\log^2 q)$ and $O(n^2\log q)$ space.
- $\mathsf{FHE.Mult}(pk,c_1,c_2)$ multiplies and refreshes with time cost of $O(n^3\log^2 q)$ and $O(n^2\log q)$ for space.
- $\mathsf{FHE.Refresh}(c,\tau,q_j,q_{j-1})$ consists of expanding operation $\big(t:O(n^2\log^2 q)\big)$, scaling $\big(t:O(n^2\log q)\big)$, SwitchKey $\big(t:O(n^3\log^2 q)\big)$, with $O(n^2\log q)$ space. Thus, $\mathsf{FHE.Refresh}$ time is $O(n^3\log^2 q)$ and space is $O(n^2\log q)$.
- The commutative time and space complexity of FHE of this method (with bootstrapping) are $O(\lambda^4\times|C|)$ and $O(\log\lambda\times\lambda^3)$, respectively.

A.12 Proof of Lemma 1

Proof. Recall the construction:

$$\mathsf{UT.Setup}(1^\lambda,1^d,(\mathbb{S},N),x) := \begin{cases} (\mathsf{pp}',\{\mathsf{sk}_i\}_i) \leftarrow \mathsf{dUT.Setup}(1^\lambda,(\mathbb{S},N),(\mathbb{C},d)) \\ \mathsf{ct} \leftarrow \mathsf{dUT.Encode}(\mathsf{pp}',x) \\ \textbf{return } (\mathsf{pp}=(\mathsf{pp}',\mathsf{ct}),\{\mathsf{sk}_i\}_i) \end{cases}$$

with $\mathsf{UT.Eval}(\mathsf{pp},\mathsf{sk}_i,\mathbb{C})=\mathsf{dUT.Eval}(\mathsf{pp}',\mathsf{sk}_i,(\mathbb{C},d),\mathsf{ct})$, verification $\mathsf{UT.Verify}(\mathsf{pp},y_i,\mathbb{C})=\mathsf{dUT.Verify}(\mathsf{pp}',y_i,(\mathbb{C},d),\mathsf{ct})$, and $\mathsf{UT.Combine}(\mathsf{pp},B)=\mathsf{dUT.Combine}(\mathsf{pp}',B,\mathsf{ct})$. For the security part of the Lemma, given dUT simulator $\mathcal{S}_{\mathrm{dUT}}=(\mathcal{S}_1,\mathcal{S}_2,\mathcal{S}_3)$ from Definition 2, construct UT simulator $\mathcal{S}_{\mathrm{UT}}=(\mathcal{S}'_1,\mathcal{S}'_2)$:

- $\mathcal{S}'_1(1^\lambda,1^d,(\mathbb{S},N))$: Run $(\mathsf{pp}',\{\mathsf{sk}_i\}_i,\mathsf{st}) \leftarrow \mathcal{S}_1(1^\lambda,(\mathbb{S},N),(\mathbb{C},d))$, $(\mathsf{ct},\mathsf{st}') \leftarrow \mathcal{S}_2(\mathsf{st},|x|,\mathsf{pp}')$, output $(\mathsf{pp}=(\mathsf{pp}',\mathsf{ct}),\{\mathsf{sk}_i\}_i)$ and store st'.
- $\mathcal{S}'_2(C,C(x),S)$: Output $\mathcal{S}_3(\mathsf{st}',C,C(x),S)$.

Consider hybrid experiments: $\mathcal{H}_0$ uses real $\mathsf{dUT.Setup}$ and $\mathsf{dUT.Encode}$ with real evaluations; $\mathcal{H}_1$ replaces setup/encoding with $\mathcal{S}_1,\mathcal{S}_2$; $\mathcal{H}_2$ replaces evaluations with $\mathcal{S}_3$. By Definition 2, $|\Pr[\mathcal{H}_0]-\Pr[\mathcal{H}_1]|\le\mathsf{negl}(\lambda)$ and $|\Pr[\mathcal{H}_1]-\Pr[\mathcal{H}_2]|\le\mathsf{negl}(\lambda)$. Since $\mathcal{H}_0$ corresponds to $\mathsf{Exp}^{\mathrm{real}}_{\mathcal{A},\mathsf{UT}}$ and $\mathcal{H}_2$ to $\mathsf{Exp}^{\mathrm{ideal}}_{\mathcal{A},\mathsf{UT}}$, we have $|\Pr[\mathsf{Exp}^{\mathrm{real}}]-\Pr[\mathsf{Exp}^{\mathrm{ideal}}]|\le\mathsf{negl}(\lambda)$.

For the robustness part of the Lemma, assume adversary $\mathcal{A}_{\mathrm{UT}}$ breaks UT robustness with probability ϵ. Construct $\mathcal{A}_{\mathrm{dUT}}$ that:

- Receives $(\mathsf{pp}',\{\mathsf{sk}_i\}_i)$ from dUT challenger.
- Runs $\mathcal{A}_{\mathrm{UT}}$ providing $(\mathsf{pp}=(\mathsf{pp}',\mathsf{ct}),\{\mathsf{sk}_i\}_i)$ where $\mathsf{ct}\leftarrow\mathsf{dUT.Encode}(\mathsf{pp}',x)$ for adversary-chosen x.
- Forwards (x,C^*,i^*) from $\mathcal{A}_{\mathrm{UT}}$ to challenger, receives honest y_{i^*}, provides to $\mathcal{A}_{\mathrm{UT}}$.
- Outputs y^* received from $\mathcal{A}_{\mathrm{UT}}$.

Since $\mathsf{UT.Eval}=\mathsf{dUT.Eval}$ and $\mathsf{UT.Verify}=\mathsf{dUT.Verify}$ on corresponding inputs, if $\mathcal{A}_{\mathrm{UT}}$ produces $y^*\neq y_{i^*}$ with $\mathsf{UT.Verify}(\mathsf{pp},y^*,C^*)=1$, then $\mathcal{A}_{\mathrm{dUT}}$ produces $y^*\neq y_{i^*}$ with $\mathsf{dUT.Verify}(\mathsf{pp}',y^*,(C^*,d),\mathsf{ct})=1$. Hence $\mathcal{A}_{\mathrm{dUT}}$ succeeds with probability ϵ, contradicting Definition 3. Therefore $\epsilon\le\mathsf{negl}(\lambda)$. □

A.13 Proof of Theorem 1

Proof. Fix arbitrary parameters satisfying the constraints and let $S \subseteq [N]$ with $|S| \geq t+\ell_p$. We proceed by induction on the tree levels from L down to 0.

By construction of Share', each level-L share $\mathrm{share}_m^{(L,j)}$ for $m \in [\ell^L], j \in [k]$ is a point evaluation at m of a degree-$(s+\ell_p-2)$ polynomial $f_{\lfloor m/\ell \rfloor}^{(L,j)}(X)$ satisfying $f_{\lfloor m/\ell \rfloor}^{(L,j)}(\alpha) = \mathrm{share}_{\lfloor m/\ell \rfloor,\alpha}^{(L-1,j)}$ for $\alpha \in \{-(\ell_p-1),\ldots,0\}$. The partition $\{T_i\}_{i\in[N]}$ of $[\ell^L]$ is generated uniformly at random subject to $\bigcup_{i\in[N]} T_i = [\ell^L]$. For each parent node at level $L-1$, there exist ℓ children at level L. Since $|S| \geq t+\ell_p$ and each T_i for $i \in S$ contributes shares, with probability $1-2^{-\Omega(L\log\ell)}$, for each polynomial $f_m^{(L,j)}$ where $m \in [\ell^{L-1}]$, at least $s+\ell_p-1$ of its ℓ evaluations are held by parties in S. By Lagrange interpolation, any $s+\ell_p-1$ distinct points on a degree-$(s+\ell_p-2)$ polynomial uniquely determine the polynomial, hence determine $\mathrm{share}_m^{(L-1,j)}$ for all $j \in [k]$.

Assume that for level $\iota+1$, all shares $\{\mathrm{share}_m^{(\iota+1,j)}\}_{m\in T^{(\iota+1)}, j\in[k]}$ in reconstruction set $T^{(\iota+1)}$ are correctly recovered with probability $1-2^{-\Omega((\iota+1)\log\ell)}$. At level ι, each share $\mathrm{share}_m^{(\iota,j)}$ is the evaluation point of a degree-$(s+\ell_p-2)$ polynomial having ℓ children at level $\iota+1$. By the random partition property and $|S| \geq t+\ell_p$, with probability $1-2^{-\Omega(\log\ell)}$, at least $s+\ell_p-1$ of these children are in $T^{(\iota+1)}$, enabling reconstruction of $\mathrm{share}_m^{(\iota,j)}$ via Lagrange interpolation. By union bound over ℓ^ι nodes at level ι, the failure probability at level ι is at most $\ell^\iota \cdot 2^{-\Omega(\log\ell)} + 2^{-\Omega((\iota+1)\log\ell)} = 2^{-\Omega(\iota\log\ell)}$.

Applying the inductive argument through all L levels, we successfully reconstruct $\mathrm{share}_1^{(0,j)} = \mathsf{sk}^{(j)}$ for all $j \in [k]$ with probability at least $1-\sum_{\iota=0}^{L} 2^{-\Omega(\iota\log\ell)} \geq 1-2^{-\Omega(L\log\ell)}$. Since $L \geq \log_{c_s} N + \log_s N + O(1)$ and $\ell = 2s-1 \geq 5$, we have $L\log\ell \geq \Omega(\log N) = \Omega(\lambda)$ for appropriately chosen parameters. Each $\mathsf{sk}^{(j)} = (\mathsf{sk}_{-(\ell_p-1)}^{(j)}, \ldots, \mathsf{sk}_0^{(j)}) \in \mathbb{Z}_q^{\ell_p}$ represents ℓ_p packed secrets, and the Lagrange reconstruction at reconstruction points $\alpha \in \{-(\ell_p-1),\ldots,0\}$ yields $\hat{\mathsf{sk}}_\alpha^{(j)} = f^{(0,j)}(\alpha) = \mathsf{sk}_\alpha^{(j)}$ for all $j \in [k]$, $\alpha \in \{-(\ell_p-1),\ldots,0\}$ with failure probability at most $2^{-\Omega(\lambda)}$. □

A.14 Proof of Theorem 2

Proof. Let $U \subseteq [N]$ with $|U| < t+\ell_p$ be an arbitrary unauthorized subset. We construct a sequence of hybrid distributions $\{\mathcal{H}_\iota\}_{\iota=0}^{L+1}$ where $\mathcal{H}_0$ corresponds to shares generated from $\{\mathsf{sk}_0^{(j)}\}_{j\in[k]}$ and $\mathcal{H}_{L+1}$ corresponds to shares from $\{\mathsf{sk}_1^{(j)}\}_{j\in[k]}$.

$\mathcal{H}_0$: Generate shares $\{\{\mathsf{sk}_{0,i}^{(j)}\}_{j\in[k]}, T_i, \pi_i\}_{i\in U}$ by executing $\mathsf{Share}'(\{\mathsf{sk}_0^{(j)}\}_{j\in[k]}, \mathcal{A}_{N,t})$ and outputting the view of parties in U.

$\mathcal{H}_\iota$ for $\iota \in [L]$: At level $\iota-1$, replace all shares $\mathrm{share}_m^{(\iota-1,j)}$ for $m \in [\ell^{\iota-1}], j \in [k]$ with uniformly random values in $\mathbb{Z}_q$ subject to maintaining consistency with level-ι shares held by parties in U. Specifically, for each polynomial $f_m^{(\iota,j)}(X)$ of degree $s+\ell_p-2$ at level ι, if $|U \cap \{\text{parties holding children of node } m\}| < s+\ell_p-1$, sample $f_m^{(\iota,j)}(X)$ uniformly from the space of degree-$(s+\ell_p-2)$ polynomials in $\mathbb{Z}_q[X]$ consistent with the evaluations held by U. Recompute commitments $\{c_{\ell(m-1)+\kappa}^{(\iota)}\}_{\kappa\in[\ell]}$ and proofs $z_m^{(\iota)}(X)$ accordingly.

For each level $\iota \in [L]$ and each polynomial $f_m^{(\iota,j)}(X)$, since $|U| < t+\ell_p$, by the random partition property, with overwhelming probability $1-2^{-\Omega(L\log\ell)}$, parties in U collectively hold fewer than $s+\ell_p-1$ evaluation points of $f_m^{(\iota,j)}(X)$. By the information-theoretic security of Shamir secret sharing over $\mathbb{Z}_q$, any $s+\ell_p-2$ or fewer points on a degree-$(s+\ell_p-2)$ polynomial reveal no information about the value at reconstruction points $\alpha \in \{-(\ell_p-1),\ldots,0\}$. Hence, the shares $\{\mathsf{share}_m^{(\iota-1,j)}\}_{m,j}$ are information-theoretically hidden from U.

The commitments $\{c_m^{(\iota)}\}_{m\in[\ell^L]}$ are generated using the computationally hiding commitment scheme $\mathcal{C}$. Under the hiding property of $\mathcal{C}$, for any polynomial-time distinguisher $\mathcal{D}$ and any two messages $(f_m^{(L,1)}(\kappa),\ldots,f_m^{(L,k)}(\kappa),r_m^{(L)}(\kappa))$ and $(f_m^{\prime(L,1)}(\kappa),\ldots,f_m^{\prime(L,k)}(\kappa),r_m^{\prime(L)}(\kappa))$, the advantage

$$\left|\Pr[\mathcal{D}(c_m^{(L)})=1]-\Pr[\mathcal{D}(c_m^{\prime(L)})=1]\right| \le \mathsf{negl}(\lambda),$$

considering the values of $c_m^{(L)} \leftarrow \mathcal{C}((f_m^{(L,1)}(\kappa),\ldots,f_m^{(L,k)}(\kappa),r_m^{(L)}(\kappa)),\gamma_m^{(L)})$ and $c_m^{\prime(L)} \leftarrow \mathcal{C}((f_m^{\prime(L,1)}(\kappa),\ldots,f_m^{\prime(L,k)}(\kappa),r_m^{\prime(L)}(\kappa)),\gamma_m^{\prime(L)})$. By a standard hybrid argument over $\ell^L\cdot k$ committed values and invoking the hiding property of $\mathcal{C}$ at each step, $\mathcal{H}_\iota$ and $\mathcal{H}_{\iota+1}$ are computationally indistinguishable with advantage at most $\ell^L\cdot k\cdot\mathsf{negl}(\lambda)=\mathsf{negl}(\lambda)$ for polynomial $\ell^L=\mathsf{poly}(N)$.

The batched challenges $(d_1^{(\iota)},\ldots,d_k^{(\iota)}) \leftarrow \mathcal{H}(c_{\ell(m-1)+1}^{(\iota)},\ldots,c_{\ell\cdot m}^{(\iota)})$ are outputs of random oracle $\mathcal{H}$. In the random oracle model, these challenges are uniformly random and independent of the underlying secrets, providing no additional information to U beyond what is contained in the commitments.

$\mathcal{H}_{L+1}$: Replace the root secret $\mathsf{share}_1^{(0,j)}=\mathsf{sk}_0^{(j)}$ with $\mathsf{sk}_1^{(j)}$ for all $j\in[k]$ and propagate the change through all levels by resampling all polynomials and commitments. By the information-theoretic and computational arguments above, $\mathcal{H}_L$ and $\mathcal{H}_{L+1}$ are computationally indistinguishable.

By transitivity of computational indistinguishability over $L+1$ hybrids, $\mathcal{H}_0 \overset{c}{\approx} \mathcal{H}_{L+1}$, establishing that the view of any unauthorized subset U with $|U|<t+\ell_p$ is computationally indistinguishable between shares generated from $\{\mathsf{sk}_0^{(j)}\}_{j\in[k]}$ and $\{\mathsf{sk}_1^{(j)}\}_{j\in[k]}$, completing the proof. □

A.15 Proof of Theorem 3

Proof. For Encrypt, each TLWE ciphertext $\mathrm{ct}_j=(\mathbf{a},b)\in\mathbb{T}^{n+1}$ has size $|\mathrm{ct}_j|=O(n)$. For Eval, gate bootstrapping maintains ciphertext dimension through BootstrapTLWE outputting TRLWE of dimension $(\bar{k}+1)\bar{N}$ followed by SampleExtract and KeySwitch reducing back to TLWE dimension $n+1$. Hence $|\hat{\mathrm{ct}}|=O(n)=\mathsf{poly}_1(\lambda,d,n)$ independent of circuit size. For PartDec, each party P_i computes partial phase $\varphi_i\in\mathbb{T}$ and adds noise $e_i\in[-B_{\mathrm{sm}},B_{\mathrm{sm}}]$, yielding $p_i\in\mathbb{T}$ with $|p_i|=O(\log B_{\mathrm{sm}})=O(\lambda)$ when represented with sufficient precision. Thus $|p_i|=\mathsf{poly}_2(\lambda,d,n,N)$. □

A.16 Proof of Theorem 4

Proof. By Theorem 1, $\mathsf{TreeSSS'.Combine'}$ reconstructs $\{\hat{\mathsf{sk}}^{(j)}\}_{j\in[k']}=\{(K_1,...,K_n)\}$ from shares $\{\mathsf{sk}_i\}_{i\in S}$ with probability $\geq 1-2^{-\Omega(\lambda)}$. For $\hat{\mathrm{ct}}=(\mathbf{a},b)$ from Eval, the correctness of Torus-FHE evaluation guarantees $b=\mathbf{a}\cdot K+e'+C(\mu_1,...,\mu_\kappa)$ (mod 1) where $|e'|\leq\bar{\sigma}$ with overwhelming probability. Each partial decryption computes $p_i=\varphi_i+((2s-1)!)^L\cdot e_i$ where $\varphi_i=\sum_{m\in T_i}\mathbf{a}_m\cdot\hat{K}_m$ and $e_i\leftarrow\mathcal{U}([-B_{\mathrm{sm}},B_{\mathrm{sm}}])$. In FinDec, the Lagrange reconstruction yields:

$$\begin{aligned}\varphi_S&=b-\sum_{i\in S}c_i^S\cdot p_i\\&=b-\sum_{i\in S}c_i^S(\varphi_i+((2s-1)!)^Le_i)\\&=b-\mathbf{a}\cdot K-\sum_{i\in S}c_i^S((2s-1)!)^Le_i\\&=e'+C(\mu_1,...,\mu_\kappa)-\texttt{noise flooding}.\end{aligned}$$

By choice of B_{sm} satisfying $\bar{\sigma}+\frac{((2s-1)!)^{2L}(2s-1)^LB_{\mathrm{sm}}}{2^\lambda}=\mathsf{negl}(\lambda)$, the total noise $|e'-\sum_{i\in S}c_i^S((2s-1)!)^Le_i|<1/4$ with probability $\geq 1-\mathsf{negl}(\lambda)$. Hence $\hat{\mu}=\lfloor 2\varphi_S\rceil=C(\mu_1,...,\mu_\kappa)$ with failure probability at most $2^{-\Omega(\lambda)}+\mathsf{negl}(\lambda)=\mathsf{negl}(\lambda)$. □

A.17 Proof of Theorem 5

Proof. Let $\mathcal{A}$ be a PPT adversary and $U\notin\mathcal{A}_{N,t}$ with $|U|<t+\ell_p$. By Theorem 2, the shares $\{\mathsf{sk}_i\}_{i\in U}$ reveal no information about the secret key $K\in\mathbb{B}^n$ under the computational hiding of commitments and random oracle model. An encryption $\mathrm{ct}=(\mathbf{a},b)$ where $b=\mathbf{a}\cdot K+e+\mu$ (mod 1) for $\mathbf{a}\leftarrow\mathbb{T}^n$, $e\leftarrow\mathcal{D}_{\mathbb{T},\alpha}$, $\mu\in\mathbb{T}$ is a TLWE sample. Under the decisional $\mathsf{TLWE}(n,\chi)$ assumption, the distribution $(\mathbf{a},\mathbf{a}\cdot K+e+\mu)$ is computationally indistinguishable from $(\mathbf{a},u)$ where $u\leftarrow\mathbb{T}$. Suppose adversary $\mathcal{A}$ has advantage ϵ in distinguishing encryptions of 0 and 1. Construct a TLWE distinguisher $\mathcal{B}$ that receives sample $(\mathbf{a},b^*)$ and simulates the security experiment: generate $(\mathsf{pk},\{\mathsf{sk}_i\}_i)$, receive U from $\mathcal{A}$, sample $\beta\leftarrow\{0,1\}$, set $\mathrm{ct}=(\mathbf{a},b^*+\beta)$, and output 1 iff $\mathcal{A}(\{\mathsf{sk}_i\}_{i\in U},\mathrm{ct})=\beta$. If $(\mathbf{a},b^*)$ is a valid TLWE sample, $\mathcal{B}$ perfectly simulates the real experiment and succeeds with probability $\frac{1}{2}+\epsilon$. If $b^*\leftarrow\mathbb{T}$ uniformly, ct is independent of β and $\mathcal{B}$ succeeds with probability $\frac{1}{2}$. Hence $\mathcal{B}$'s TLWE advantage is ϵ, contradicting the TLWE assumption unless $\epsilon\leq\mathsf{negl}(\lambda)$. □

A.18 Proof of Theorem 6

Proof. Parse partial evaluation $y_i=(p_i,\pi_i)$. By Theorem 3, partial decryption $p_i\in\mathbb{T}$ has size $|p_i|=O(\lambda)$ when represented with sufficient precision. For zero-knowledge proof π_i, the statement Ψ_i asserts existence of witness $((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,r_i,e_i)$ satisfying: (i) commitment opening $\mathsf{com}_i=\mathsf{Com}((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},T_i);r_i)$, (ii) TreeSSS verification $\mathsf{TreeSSS'.Verify}(\pi_i^{\mathrm{tree}},\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']})=1$, (iii) partial decryption correctness $p_i=\mathsf{FHE''.Dec}_0(\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},\hat{\mathsf{ct}})+((2s-1)!)^Le_i$ mod 1.

Witness size $|(\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},T_i,r_i,e_i)| = O(|T_i|\cdot n\cdot\log q+\lambda) = O(|T_i|\cdot n\cdot\lambda)$ where $|T_i| = O((2t-1)^L/N)$. By efficiency of PZK (preprocessing zero-knowledge), proof size $|\pi_i| = \mathsf{poly}(\lambda,|\Psi_i|) = \mathsf{poly}(\lambda,|T_i|\cdot n)$. Hence $|y_i| = |p_i|+|\pi_i| = O(\lambda)+\mathsf{poly}(\lambda,|T_i|\cdot n) = \mathsf{poly}_1(\lambda,d,n,|T_i|)$.

Public parameters $\mathsf{pp} = (\mathsf{fhepk},\{\pi_i^{\mathrm{tree}}\}_{i\in[N]},\{\sigma_{V,i}\}_{i\in[N]},\{\mathsf{com}_i\}_{i\in[N]})$ consist of: FHE public key $\mathsf{fhepk} = (\mathsf{BK},\mathsf{KS})$ with $|\mathsf{fhepk}| = O(n\cdot(\bar{k}+1)^2\cdot\ell_g\cdot N_{\mathrm{ring}}+\bar{n}\cdot t_{\mathrm{ks}}\cdot n) = \mathsf{poly}(\lambda,d,n)$, TreeSSS verification data $\{\pi_i^{\mathrm{tree}}\}_{i\in[N]}$ with total size $O(N\cdot|T_i|\cdot\log q) = O((2t-1)^L\cdot\log q) = \mathsf{poly}(\lambda,N)$, PZK verifier states $\{\sigma_{V,i}\}_{i\in[N]}$ with total size $O(N\cdot\lambda) = \mathsf{poly}(\lambda,N)$, and commitments $\{\mathsf{com}_i\}_{i\in[N]}$ with total size $O(N\cdot\lambda) = \mathsf{poly}(\lambda,N)$. Hence $|\mathsf{pp}| = \mathsf{poly}_2(\lambda,d,n,N)$. □

A.19 Proof of Theorem 7

Proof. Eval computes FHE encryptions $\{\mathsf{ct}_j\leftarrow\mathsf{FHE''.Encrypt}(\mathsf{fhepk},x_j)\}_{j\in[|x|]}$, evaluates $\hat{\mathsf{ct}}\leftarrow\mathsf{FHE''.Eval}(\mathsf{fhepk},\mathbb{C},\{\mathsf{ct}_j\}_j)$, reconstructs partial key bits $\{\hat{K}_m^{(j)}\}_{m\in T_i,j\in[k']}$ from shares $\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']}$, partial phase $\varphi_i = \mathsf{FHE''.Dec}_0(\{\hat{K}_m^{(j)}\}_{m\in T_i,j\in[k']},\hat{\mathsf{ct}})$, samples noise $e_i\leftarrow\mathcal{U}([-B_{\mathrm{sm}},B_{\mathrm{sm}}])$, sets $p_i = \varphi_i+((2s-1)!)^L e_i \bmod 1$, constructs statement Ψ_i asserting witness $((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,r_i,e_i)$ satisfies all three conditions, generates proof $\pi_i\leftarrow\mathsf{PZK.Prove}(\sigma_{P,i},\Psi_i,\mathrm{witness})$, and outputs $y_i = (p_i,\pi_i)$.

Verify recomputes same FHE encryptions and evaluation $\hat{\mathsf{ct}}$, parses $y_i = (p_i,\pi_i)$, constructs identical statement Ψ_i, and verifies $\mathsf{PZK.Verify}(\sigma_{V,i},\Psi_i,\pi_i)$. By completeness of PZK, honest proofs verify with probability $\geq 1-\mathsf{negl}(\lambda)$. Since witness $((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,r_i,e_i)$ genuinely satisfies all conditions in Ψ_i by construction in Eval: (i) com_i was generated as $\mathsf{Com}((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},T_i);r_i)$ in Setup, (ii) $\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']}$ are valid TreeSSS shares from Setup, (iii) p_i correctly computed as specified, the statement Ψ_i is true and PZK.Prove produces accepting proof with overwhelming probability. □

A.20 Proof of Theorem 8

Proof. Construct simulator $\mathcal{S}_6 = (\mathcal{S}_{6,1},\mathcal{S}_{6,2},\mathcal{S}_{6,3})$ as follows.

$\mathcal{S}_{6,1}(1^\lambda,1^d,(\mathbb{S},N),(\mathbb{C},d))$: Execute $(\mathsf{fhepk},\mathsf{fhesk})\leftarrow\mathsf{FHE''.Setup}(1^\lambda,1^d)$ honestly, partition fhesk into k' groups, run $\mathsf{TreeSSS'.Share}'$ to generate the set of shares $\{\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},T_i,\pi_i^{\mathrm{tree}}\}_{i\in[N]}$, generate PZK preprocessing $(\sigma_{V,i},\sigma_{P,i})\leftarrow\mathsf{PZK.Pre}(1^\lambda)$ for each party, sample randomness $r_i\leftarrow\{0,1\}^\lambda$, compute commitments $\mathsf{com}_i\leftarrow\mathsf{Com}((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i,j\in[k']},T_i);r_i)$, set $\mathsf{pp} = (\mathsf{fhepk},\{\pi_i^{\mathrm{tree}}\}_i,\{\sigma_{V,i}\}_i,\{\mathsf{com}_i\}_i)$ and $s_i = ((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_{j\in[k']},T_i,\sigma_{P,i},r_i)$, store state $\mathsf{st} = (\mathsf{pp},\{s_i\}_i,\mathsf{fhesk})$, and output $(\mathsf{pp},\{s_i\}_i,\mathsf{st})$.

$\mathcal{S}_{6,2}(\mathsf{st},|x|,\mathsf{pp})$: Sample random encoding $\mathsf{ct}\leftarrow\{0,1\}^{|x|\cdot(n+1)\cdot\lceil\log_2(2^\lambda)\rceil}$ to simulate FHE ciphertexts of appropriate length, update state $\mathsf{st}' = (\mathsf{st},\mathsf{ct})$, and output $(\mathsf{ct},\mathsf{st}')$.

$\mathcal{S}_{6,3}(\mathsf{st}',C',C'(x),S)$: For each $i\in S$, retrieve $s_i = ((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_i})_j,T_i,\sigma_{P,i},r_i)$ from st', sample target phase $\varphi_{\mathrm{target},i}$ uniformly from distribution consistent with output $C'(x)$, sample noise $e_i\leftarrow\mathcal{U}([-B_{\mathrm{sm}},B_{\mathrm{sm}}])$, set $p_i = \varphi_{\mathrm{target},i}+((2s-1)!)^L e_i \bmod 1$, use zero-knowledge simulator to generate simulated proof $\pi_i\leftarrow\mathsf{PZK.Sim}(\sigma_{V,i},\Psi_i)$

without witness (exploiting ZK property and preprocessing trapdoor), and output $\{y_i=(p_i,\pi_i)\}_{i\in S}$.

Indistinguishability follows from sequence of hybrids: $\mathcal{H}_0$ is real experiment. $\mathcal{H}_1$ replaces real FHE ciphertexts ct with random strings; by semantic security of $\mathsf{TFHE'''}$ (Theorem 5), unauthorized set S^* with shares $\{s_i\}_{i\in S^*}$ learns no information about fhesk, making ciphertexts indistinguishable from random with advantage $\leq \mathsf{negl}(\lambda)$. $\mathcal{H}_2$ replaces commitments $\{\mathsf{com}_i\}_{i\in S^*}$ with commitments to random values; by computational hiding of Com, advantage $\leq N\cdot\mathsf{negl}(\lambda)=\mathsf{negl}(\lambda)$. $\mathcal{H}_3$ replaces real PZK proofs with simulated proofs; by zero-knowledge of PZK with preprocessing, simulated proofs are indistinguishable from real proofs with advantage $\leq |S|\cdot\mathsf{negl}(\lambda)=\mathsf{negl}(\lambda)$. $\mathcal{H}_4$ is ideal experiment. By triangle inequality, total distinguishing advantage $\leq 4\cdot\mathsf{negl}(\lambda)=\mathsf{negl}(\lambda)$. □

A.21 Proof of Theorem 9

Proof. Assume $\mathcal{A}$ succeeds with non-negligible probability ϵ, i.e., produces $y^*=(p^*,\pi^*)$ such that $p^*\neq p_{i^*}$ yet $\mathsf{Verify}(\mathsf{pp},y^*,x,C^*)=1$. Verify computes the value of $\mathsf{PZK.Verify}(\sigma_{V,i^*},\Psi_{i^*},\pi^*)=1$ where Ψ_{i^*} asserts existence of witness as the value $((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_{i^*}})_j,T_{i^*},r_{i^*},e_{i^*})$ satisfying three conditions including the actual $p^*=\mathsf{FHE''.Dec}_0(\{\mathrm{share}_m^{(L,j)}\}_{m\in T_{i^*},j},\hat{\mathsf{ct}})+((2s-1)!)^L e_{i^*} \bmod 1$.

By soundness of PZK, if verification accepts, then with overwhelming probability $1-\mathsf{negl}(\lambda)$, there exists valid witness $(\tilde{\mathrm{shares}},\tilde{T},\tilde{r},\tilde{e})$ satisfying all three conditions. Condition (i) requires $\mathsf{com}_{i^*}=\mathsf{Com}((\tilde{\mathrm{shares}},\tilde{T});\tilde{r})$. However, in Setup, commitment com_{i^*} was generated as $\mathsf{Com}((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_{i^*},j},T_{i^*});r_{i^*})$ with true shares from TreeSSS. By perfect binding of Com, there cannot exist two distinct valid openings $((\{\mathrm{share}_m^{(L,j)}\}_{m\in T_{i^*},j},T_{i^*}),r_{i^*})$ and $((\tilde{\mathrm{shares}},\tilde{T}),\tilde{r})$ to same commitment com_{i^*} except with probability $\mathsf{negl}(\lambda)$. Hence $\tilde{\mathrm{shares}}=\{\mathrm{share}_m^{(L,j)}\}_{m\in T_{i^*},j}$ and $\tilde{T}=T_{i^*}$ and $\tilde{r}=r_{i^*}$.

Condition (iii) then implies $p^*=\mathsf{FHE''.Dec}_0(\{\mathrm{share}_m^{(L,j)}\}_{m\in T_{i^*},j},\hat{\mathsf{ct}})+((2s-1)!)^L\tilde{e} \bmod 1$. However, honest evaluation Eval computed the actual value of $p_{i^*}=\mathsf{FHE''.Dec}_0(\{\mathrm{share}_m^{(L,j)}\}_{m\in T_{i^*},j},\hat{\mathsf{ct}})+((2s-1)!)^L e_{i^*} \bmod 1$ with same shares and same $\hat{\mathsf{ct}}$. For $p^*\neq p_{i^*}$, we require $((2s-1)!)^L(\tilde{e}-e_{i^*})\not\equiv 0 \pmod 1$, i.e., $\tilde{e}\neq e_{i^*}$. However, the partial decryption function $\mathsf{FHE''.Dec}_0$ is deterministic given shares and ciphertext, so adversary must guess different noise $\tilde{e}$ such that resulting p^* still satisfies verification. By information-theoretic argument, noise flooding with parameter B_{sm} ensures any specific $\tilde{e}\neq e_{i^*}$ occurs with probability at most $2^{-\lambda}$ when e_{i^*} is uniform in $[-B_{\mathrm{sm}},B_{\mathrm{sm}}]$. Hence $\epsilon\leq\mathsf{negl}(\lambda)+2^{-\lambda}=\mathsf{negl}(\lambda)$. □

A.22 Proof of Theorem 10

Proof. Parse partial evaluation (y_i,σ_i) where $y_i\in\mathbb{T}$ has size $|y_i|=O(\lambda)$ as in Theorem 3. Homomorphic signature σ_i authenticates circuit $\mathbb{C}_{\mathrm{Dec}}:=\mathsf{TFHE'''.PartDec}(\cdot,\mathsf{ct}')$ where ct' is evaluated ciphertext. By Theorem 18, $|\sigma_i|\leq\mathsf{poly}(\lambda,\mathsf{depth}(\mathbb{C}_{\mathrm{Dec}}))$. Since $\mathbb{C}_{\mathrm{Dec}}$ computes partial phase computation (linear operations over torus) followed by noise addition (addition modulo 1), depth $\mathsf{depth}(\mathbb{C}_{\mathrm{Dec}})=O(\log n)$ for n-dimensional ciphertext. Hence $|\sigma_i|=\mathsf{poly}(\lambda,\log n)=\mathsf{poly}(\lambda,n)$. Total size $|(y_i,\sigma_i)|=O(\lambda)+\mathsf{poly}(\lambda,n)=\mathsf{poly}_1(\lambda,d,n)$.

Public parameters $\mathsf{pp}=(\mathsf{pk_{fhe}},\mathsf{pkhs},\mathsf{ctx},\{\pi_i\}_{i\in[N]})$ consist of: FHE public key $\mathsf{pk_{fhe}}$ with $|\mathsf{pk_{fhe}}|=\mathsf{poly}(\lambda,d,n)$ as before, homomorphic signature verification key pkhs with $|\mathsf{pkhs}|=\mathsf{poly}(\lambda,n)$ by construction of HS' (key generation produces N FHE encryptions and two obfuscated circuits), encoding ctx with $|\mathsf{ctx}|=O(n)$, and TreeSSS verification data $\{\pi_i\}_{i\in[N]}$ with total size $\mathsf{poly}(\lambda,N)$. Hence $|\mathsf{pp}|=\mathsf{poly}_2(\lambda,d,n,N)$. □

A.23 Proof of Theorem 11

Proof. PartEval computes FHE evaluation $\mathsf{ct}'\leftarrow\mathsf{FHE}''.\mathsf{Eval}(\mathsf{pk_{fhe}},\mathbb{C},\mathsf{ctx})$, reconstructs partial key bits from shares, computes partial phase $\varphi_i=\sum_{m\in T_i,j:(j-1)\ell_p<m\le j\ell_p}\mathbf{a}_m\cdot\hat{K}_m^{(j)}$ where $(\mathbf{a},b)=\mathsf{ct}'$, samples noise $e_i\leftarrow\mathcal{U}([-B_{\mathrm{sm}},B_{\mathrm{sm}}])$, outputs partial decryption $y_i=\varphi_i+((2s-1)!)^L e_i \bmod 1$, defines circuit $\mathbb{C}_{\mathrm{Dec}}:=\mathsf{TFHE}'''.\mathsf{PartDec}(\cdot,\mathsf{ct}')$, evaluates homomorphic signature $\sigma_i\leftarrow\mathsf{HS}'.\mathsf{Eval}(\mathsf{pkhs},i,\mathbb{C}_{\mathrm{Dec}},\mathsf{sk}_i[\mathrm{hs}])$, and outputs (y_i,σ_i).

VfyEval parses (y_i,σ_i), recomputes same evaluation $\mathsf{ct}'\leftarrow\mathsf{FHE}''.\mathsf{Eval}(\mathsf{pk_{fhe}},\mathbb{C},\mathsf{ctx})$, constructs same $\mathbb{C}_{\mathrm{Dec}}:=\mathsf{TFHE}'''.\mathsf{PartDec}(\cdot,\mathsf{ct}')$, verifies $\mathsf{HS}'.\mathsf{Verify}(\mathsf{pkhs},\mathbb{C}_{\mathrm{Dec}},y_i,\sigma_i)$. By evaluation correctness of HS' (Theorem 15), signature σ_i produced by $\mathsf{HS}'.\mathsf{Eval}$ on circuit $\mathbb{C}_{\mathrm{Dec}}$ with message-signature pair derived from $\mathsf{sk}_i[\mathrm{hs}]$ (which was signed in UT_7-AuxSetup via $\mathsf{HS}'.\mathsf{Sign}$) verifies correctly under pkhs with probability $\ge 1-\mathsf{negl}(\lambda)$. □

A.24 Proof of Theorem 12

Proof. Construct simulator $\mathcal{S}_7=(\mathcal{S}_{7,1},\mathcal{S}_{7,2},\mathcal{S}_{7,3})$ as follows. $\mathcal{S}_{7,1}(1^\lambda,1^d,(\mathbb{S},N),(\mathbb{C},d))$: Execute Setup honestly to generate $(\mathsf{pk_{fhe}},K,\bar{K})$, $(\mathsf{skhs},\mathsf{pkhs})\leftarrow\mathsf{HS}'.\mathsf{KeyGen}(1^\lambda,1^n)$, TreeSSS shares $\{\{\mathsf{sk}_i^{(j)}\}_j,T_i,\pi_i\}_i$, auxiliary signatures $\{\mathsf{sk}'_i[\mathrm{hs}]\}_i$ via UT_7-AuxSetup, sample random encoding $\mathsf{ctx}\leftarrow\{0,1\}^{(n+1)\cdot\lceil\log_2(2^\lambda)\rceil}$, set $\mathsf{pp}=(\mathsf{pk_{fhe}},\mathsf{pkhs},\mathsf{ctx},\{\pi_i\}_i)$ and $\mathsf{sk}=\{\mathsf{sk}'_i\}_i$, store state $\mathsf{st}=(\mathsf{pp},\mathsf{sk},K,\mathsf{skhs})$, and output $(\mathsf{pp},\mathsf{sk},\mathsf{st})$.

$\mathcal{S}_{7,2}(\mathsf{st},|x|,\mathsf{pp})$: Replace random ctx in pp with properly-sized random string (already done in $\mathcal{S}_{7,1}$), update state $\mathsf{st}'=\mathsf{st}$, and output $(\mathsf{ctx},\mathsf{st}')$.

$\mathcal{S}_{7,3}(\mathsf{st}',C',C'(x),S)$: For each $i\in S$, retrieve $\mathsf{sk}'_i=(\mathsf{sk}'_i[\mathrm{tfhe}],\mathsf{sk}'_i[\mathrm{hs}])$ from st', sample partial phase φ_i uniformly from appropriate distribution, sample noise $e_i\leftarrow\mathcal{U}([-B_{\mathrm{sm}},B_{\mathrm{sm}}])$, set $y_i=\varphi_i+((2s-1)!)^L e_i \bmod 1$, define circuit $\mathbb{C}_{\mathrm{Dec}}:=\mathsf{TFHE}'''.\mathsf{PartDec}(\cdot,\mathsf{ct}')$ where ct' would result from evaluating C' on ctx, use context-hiding simulator for HS' (Theorem 17) to generate simulated $\widetilde{\sigma}_i\leftarrow\mathcal{S}_{\mathrm{ctx}}(\mathsf{skhs},\mathbb{C}_{\mathrm{Dec}},y_i)$ that hides message inputs, and output $\{(y_i,\widetilde{\sigma}_i)\}_{i\in S}$.

Indistinguishability follows from hybrid argument: $\mathcal{H}_0$ is real experiment. $\mathcal{H}_1$ replaces ctx with random encoding; by semantic security of TFHE''' (Theorem 5) and privacy of TreeSSS shares, unauthorized adversary with shares $\{(\mathsf{sk}_i,T_i)\}_{i\in S^*}$ cannot distinguish real encoding from random with advantage $>\mathsf{negl}(\lambda)$. $\mathcal{H}_2$ replaces real homomorphic signatures with simulated signatures; by context-hiding of HS' (Theorem 17), simulated signatures $\widetilde{\sigma}_i\leftarrow\mathcal{S}_{\mathrm{ctx}}(\mathsf{skhs},\mathbb{C}_{\mathrm{Dec}},y_i)$ are indistinguishable from evaluated signatures with advantage $\le|S|\cdot\mathsf{negl}(\lambda)=\mathsf{negl}(\lambda)$. $\mathcal{H}_3$ is ideal experiment. Total distinguishing advantage $\le 3\cdot\mathsf{negl}(\lambda)=\mathsf{negl}(\lambda)$. □

A.25 Proof of Theorem 13

Proof. Assume $\mathcal{A}$ succeeds with non-negligible probability ϵ, producing (y^*, σ^*) such that $y^* \neq y_{i^*}$ yet $\mathsf{VfyEval}(\mathsf{pp}, (y^*, \sigma^*), i^*, C^*) = 1$. $\mathsf{VfyEval}$ computes $\mathsf{ct}' \leftarrow \mathsf{FHE}''.\mathsf{Eval}(\mathsf{pk}_{\mathsf{fhe}}, C^*, \mathsf{ctx})$, constructs circuit $\mathbb{C}_{\mathrm{Dec}} := \mathsf{TFHE}'''.\mathsf{PartDec}(\cdot, \mathsf{ct}')$, and verifies $\mathsf{HS}'.\mathsf{Verify}(\mathsf{pkhs}, \mathbb{C}_{\mathrm{Dec}}, y^*, \sigma^*) = 1$.

Honest $\mathsf{PartEval}$ computed same evaluation ct', same circuit $\mathbb{C}_{\mathrm{Dec}}$, computed honest partial decryption $y_{i^*} = \mathbb{C}_{\mathrm{Dec}}(\text{inputs})$ where inputs include shares $\{\mathsf{sk}_{i^*}^{(j)}\}_j$ and randomness e_{i^*}, and generated signature $\sigma_{i^*} \leftarrow \mathsf{HS}'.\mathsf{Eval}(\mathsf{pkhs}, i^*, \mathbb{C}_{\mathrm{Dec}}, \mathsf{sk}_{i^*}[\mathrm{hs}])$ where $\mathsf{sk}_{i^*}[\mathrm{hs}] = \mathsf{HS}'.\mathsf{Sign}(\mathsf{skhs}, i^*, \mathsf{sk}'_{i^*}[\mathrm{tfhe}])$ was generated in $\mathsf{UT}_7\text{-}\mathsf{AuxSetup}$.

If $y^* \neq y_{i^*}$ but verification accepts σ^*, then adversary produced valid signature authenticating incorrect output y^* for circuit $\mathbb{C}_{\mathrm{Dec}}$. Construct HS' forger $\mathcal{F}$ as follows: $\mathcal{F}$ receives $\mathsf{vk} = \mathsf{pkhs}$ from HS' challenger, embeds pkhs in pp, simulates Setup by generating FHE keys and TreeSSS shares honestly, queries $\mathsf{HS}'.\mathit{Sign}$ oracle to obtain signatures $\{\mathsf{sk}'_i[\mathrm{hs}]\}_i$ on messages $\{(i, \mathsf{sk}'_i[\mathrm{tfhe}])\}_i$, provides $(\mathsf{pp}, \mathsf{sk})$ to $\mathcal{A}$, receives (x, C^*, i^*) from $\mathcal{A}$, computes honest (y_{i^*}, σ_{i^*}) via $\mathsf{PartEval}$, runs $\mathcal{A}$ to obtain (y^*, σ^*), and outputs forgery $(f^* = \mathbb{C}_{\mathrm{Dec}}, y^* = y^*, \sigma^* = \sigma^*, \tau^* = i^*)$ to HS' challenger.

By construction, $\mathcal{F}$ never queried Sign oracle on tuple $(\mathbb{C}_{\mathrm{Dec}}, \cdot)$ for any value, since $\mathsf{PartEval}$ uses $\mathsf{HS}'.\mathsf{Eval}$ (evaluation) not $\mathsf{HS}'.\mathsf{Sign}$ (signing). Circuit $\mathbb{C}_{\mathrm{Dec}}$ applied to signed data $(i^*, \mathsf{sk}'_{i^*}[\mathrm{tfhe}])$ should output y_{i^*} by correctness, i.e., $\mathbb{C}_{\mathrm{Dec}}(i^*, \mathsf{sk}'_{i^*}[\mathrm{tfhe}]) = y_{i^*}$. However, adversary produced $y^* \neq y_{i^*}$ with valid signature verifying under $\mathbb{C}_{\mathrm{Dec}}$, violating unforgeability of HS'. By Theorem 16, $\mathcal{F}$ succeeds with probability $\leq \mathsf{negl}(\lambda)$. Hence $\epsilon \leq \mathsf{negl}(\lambda)$. □